



University of Dhaka

Dept. of Applied Mathematics

4th Year BS

Assignment on

Python & COMSOL Multiphysics

Student's Name	Md. Nayeemul Hasan Nayeem
Roll Number	SH-123-040
Reg. Number	2020016171
Current Session	2023-24
Admission Session	2020-21
Lab Group	B
Date	21.10.2025

Progress Report

Assignment	Question	Signed	Submitted Online	None
1	A1Q1	☒ —	☒ —	☐
	A1Q2	☒ —	☒ —	☐
	A1Q3	☒ —	☒ —	☐
	A1Q4	☒ —	☒ —	☐
	A1Q5	☒ —	☒ —	☐
	A1Q6	☒ —	☒ —	☐
2	A2Q1	☒ —	☒ —	☐
	A2Q2	☒ —	☒ —	☐
	A2Q3	☒ —	☒ —	☐
	A2Q4	☒ —	☒ —	☐
	A2Q5	☒ —	☒ —	☐
	A2Q6	☒ —	☒ —	☐
	A2Q7	☒ —	☒ —	☐
	A2Q8	☒ —	☒ —	☐
3	A3Q1	☒ —	☒ —	☐
	A3Q2	☒ —	☒ —	☐
	A3Q3	☒ —	☒ —	☐
	A3Q4	☒ —	☒ —	☐
	A3Q5	☒ —	☒ —	☐
	A3Q6	☒ —	☒ —	☐
4	A4Q1	☐	☒ —	☐
	A4Q2	☐	☒ —	☐
	A4Q3	☐	☒ —	☐
	A4Q4	☐	☒ —	☐
	A4Q5	☐	☒ —	☐
	A4Q6	☐	☒ —	☐
	A4Q7	☐	☒ —	☐
	A4Q8	☐	☒ —	☐

5	A5Q1	☐	☒ —	☐
	A5Q2	☐	☒ —	☐
	A5Q3	☐	☒ —	☐
	A5Q4	☐	☒ —	☐
	A5Q5	☐	☒ —	☐
	A5Q6	☐	☒ —	☐
6	A6Q1	☒ —	☒ —	☐
	A6Q2	☒ —	☒ —	☐
	A6Q3	☒ —	☒ —	☐
	A6Q4	☒ —	☒ —	☐
	A6Q5	☒ —	☒ —	☐
	A6Q6	☒ —	☒ —	☐
	A6Q7	☒ —	☒ —	☐
	A6Q8	☒ —	☒ —	☐
	A6Q9	☒ —	☒ —	☐
	A6Q10	☐	☒ —	☐
	A6Q11	☐	☒ —	☐
7	A7Q1	☐	☒ —	☐
	A7Q2	☐	☒ —	☐
	A7Q3	☐	☒ —	☐
	A7Q4	☐	☒ —	☐
	A7Q5	☐	☒ —	☐
	A7Q6	☐	☒ —	☐

Assignment-01

A1Q1

Write a program to calculate the grade for a student of the Department of Applied Mathematics, University of Dhaka. Note that the grades are calculated by summing up each student's average of the first incourse and second incourse and attendance mark with the final examination marks. You are to take input of some random first incourse, second incourse, attendance, and final marks from a file and show the grade with the help of the following table.

Marks	Letter Grade	Grade Point
≥ 80	A+	4.00
≥ 75	A	3.75
≥ 70	A-	3.50
≥ 65	B+	3.25
≥ 60	B	3.00
≥ 55	B-	2.75
≥ 50	C+	2.50
≥ 45	C	2.25
≥ 40	D	2.00
< 40	F	0.00

If the user enters a value out of range, print a suitable error message and exit. For the test, enter a score of 185.

Your output should look like:

Roll	Letter Grade	Grade Point
80	A+	4.00
81	A	3.75
82	A-	3.50

```
In [ ]: def get_grade(marks):
    if marks >= 80: return "A+", 4.00
    elif marks >= 75: return "A", 3.75
    elif marks >= 70: return "A-", 3.50
    elif marks >= 65: return "B+", 3.25
    elif marks >= 60: return "B", 3.00
    elif marks >= 55: return "B-", 2.75
    elif marks >= 50: return "C+", 2.50
    elif marks >= 45: return "C", 2.25
    elif marks >= 40: return "D", 2.00
    else: return "F", 0.00

roll = float(input("Enter Roll No: "))
first_incourse = float(input("Enter first incourse marks: "))
second_incourse = float(input("Enter second incourse marks: "))
```

```

attendance = float(input("Enter attendance marks: "))
final = float(input("Enter final marks: "))

total = ((first_incourse + second_incourse) / 2) + attendance + final

letter_grade, grade_point = get_grade(total)

print("\nRoll | Letter Grade | Grade Point")
print("-" * 35)
print(f"{roll} | {letter_grade:^11s} | {grade_point:^10.2f}")

```

Roll | Letter Grade | Grade Point 40 | A+ | 4.00

A1Q2

Read through and parse the file regex_sum_42.txt which contains text and numbers. Extract the numbers in the file and compute the sum of the numbers.

```

In [1]: import re

with open("regex-sum-42.txt", 'r') as file:
    content = file.read()
    numbers = re.findall('\d+', content)

    total_sum = sum(int(i) for i in numbers)

print(f'The sum of all numbers in the file is: {total_sum}')

```

The sum of all numbers in the file is: 445833

A1Q3

Open the file amth.txt and read it line by line. For each line, split the line into a list of words using the split() method. The program should build a list of words. For each word on each line, check to see if the word is already in the list and if not, append it to the list. When the program completes, sort and print the resulting words in alphabetical order.

```

In [2]: unique_words = []

with open('amth.txt', 'r') as file:

    for _ in range(1):          ##
        next(file)

    for line in file:
        words = line.split()
        for word in words:
            if word not in unique_words:
                unique_words.append(word)

unique_words.sort()

for word in unique_words:
    print(word)

```

A
 Abdus
 Alam
 B
 Bangalee
 Chaki
 Chandra
 Chapal
 Dr.
 Ferdows
 Hossain
 Iqbal
 Islam
 Kajal
 Khaleque
 Kumar
 Litan
 M
 Md.
 Mohammad
 Muntasir
 Nirab
 Rakib
 S.M.
 Saha
 Samad
 Shafiqui
 Shahadat
 Sharmin
 Shuchi
 Tania
 Zavid

A1Q4

Write a program that prompts for a file name, then opens that file and reads through the file mbox.txt, looking for lines of the form:

X-DSPAM-Confidence: 0.8475

Count these lines, extract the floating-point values from each of the lines, compute the average of those values, and produce an output as shown below. Do not use the sum() function or a variable named sum in your solution.

```

In [ ]: # file_name = input("Enter file name: ")
        file_name = "mbox-short.txt"

        try:
            with open(file_name, 'r') as file:
                values = [float(line.split(":")[1]) for line in file if line.startswith("X-

            if values:
                print("Average spam confidence:", sum(values) / len(values))
            else:
                print("No matching lines found.")

        except FileNotFoundError:
            print("File not found. Please check the filename and try again.")
        except ValueError:
            print("Error: File contains improperly formatted lines.")
  
```

Average spam confidence: 0.7507185185185186

A1Q5

Open the file mbox.txt and read it line by line. When you find a line that starts with 'From ' like the following line:

From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008

You will parse the From line using split() and print out the second word in the line (i.e., the entire address of the person who sent the message). Then print out a count at the end. If you test your code with the file mbox-short.txt, your output should look like:

stephen.marquard@uct.ac.za
louis@media.berkeley.edu
zqian@umich.edu
rjlowe@iupui.edu
...

There were 27 lines in the file with From as the first word

```
In [2]: count = 0
with open('mbox.txt','r') as file:
    for line in file:
        if line.startswith('From '):
            print(line.split()[1])
            count += 1
print(f"there were {count} senders in the file")
```

stephen.marquard@uct.ac.za
louis@media.berkeley.edu
zqian@umich.edu
rjlowe@iupui.edu
zqian@umich.edu
rjlowe@iupui.edu
cwen@iupui.edu
cwen@iupui.edu
gsilver@umich.edu
gsilver@umich.edu
rjlowe@iupui.edu
rjlowe@iupui.edu
rjlowe@iupui.edu
aaronz@vt.edu
aaronz@vt.edu
aaronz@vt.edu
ian@caret.cam.ac.uk
aaronz@vt.edu
aaronz@vt.edu
ktsao@stanford.edu
ian@caret.cam.ac.uk
zqian@umich.edu
zqian@umich.edu
zqian@umich.edu
zqian@umich.edu
zqian@umich.edu
aaronz@vt.edu
aaronz@vt.edu
jzaremba@unicon.net
dlhaines@umich.edu
chmaurer@iupui.edu
zach.thomas@txstate.edu
dlhaines@umich.edu . . .
there were 1797 senders in the file

A1Q6

Write a program to read through the mbox.txt file and figure out who has sent the greatest number of mail messages. The program looks for 'From ' lines and takes the second word of those lines as the person who sent the mail. The program creates a Python dictionary that maps the sender's mail address to a count of the number of times they appear in the file. After the dictionary is produced, the program reads through the dictionary using a maximum loop to find the most prolific committer. If you test your code with the file mbox-short.txt, your output should look like:

cwen@iupui.edu 5

```
In [3]: counts = {}
with open('mbox.txt') as file:
    for line in file:
        if line.startswith('From '):
            counts[line.split()[1]] = counts.get(line.split()[1], 0) + 1
sender = max(counts, key=counts.get)
print(sender, counts[sender])
```

zqian@umich.edu 195

Assignment-02

A2Q1

- (a) Find the parametric equations of the tangent line to the following curves:

$$i. \vec{r}(t) = \ln t \hat{i} + e^{-t} \hat{j} + t^3 \hat{k}; t_0 = 2$$

$$ii. \vec{r}(t) = 2 \cos \pi t \hat{i} + 2 \sin \pi t \hat{j} + 3t \hat{k}; t_0 = \frac{1}{3}$$

- (b) Find the vector parallel to the line of intersection of the two planes $3x - 6y - 2z = 15$ and $2x + y - 2z = 5$.
- (c) Find the velocity and acceleration of $\vec{r}(t) = 3t\hat{i} + \sin t \hat{j} + t^2 \hat{k}$ as a function of $\theta(t)$. Also, plot the graph of $\theta(t)$ versus t .

a.

In [128..

```
import numpy as np

# Curve 1: t0 = 2
t0 = 2
r1 = np.array([np.log(t0), np.exp(-t0), t0**2])
r1_prime = np.array([1/t0, -np.exp(-t0), 2*t0])

print("curve 1 tangent line (t0 = 2):")
print(f"x = {r1[0]} + {r1_prime[0]}s")
print(f"y = {r1[1]} + {r1_prime[1]}s")
print(f"z = {r1[2]} + {r1_prime[2]}s\n")

# Curve 2: t0 = 1/3
t0 = 1/3
r2 = np.array([2*np.cos(np.pi*t0), 2*np.sin(np.pi*t0), 3*t0])
r2_prime = np.array([-2*np.pi*np.sin(np.pi*t0), 2*np.pi*np.cos(np.pi*t0), 3])

print("curve 2 tangent line (t0 = 1/3):")
print(f"x = {r2[0]} + {r2_prime[0]}s")
print(f"y = {r2[1]} + {r2_prime[1]}s")
print(f"z = {r2[2]} + {r2_prime[2]}s")
```

```
curve 1 tangent line (t0 = 2):
x = 0.6931471805599453 + 0.5s
y = 0.1353352832366127 + -0.1353352832366127s
z = 4.0 + 4.0s
```

```
curve 2 tangent line (t0 = 1/3):
x = 1.0000000000000002 + -5.441398092702653s
y = 1.7320508075688772 + 3.141592653589794s
z = 1.0 + 3.0s
```

b.

In [129..

```
import numpy as np

n1 = np.array([3, -6, -2])
n2 = np.array([2, 1, -2])
direction = np.cross(n1, n2)
print(f"{direction[0]}i + {direction[1]}j + {direction[2]}k")
```

```
14i + 2j + 15k
```

C.

```
In [130]: import sympy as sp
import numpy as np
import matplotlib.pyplot as plt

t = sp.symbols('t')

r = sp.Array([3*t, sp.sin(t), t**2])

v = r.diff(t)
a = v.diff(t)

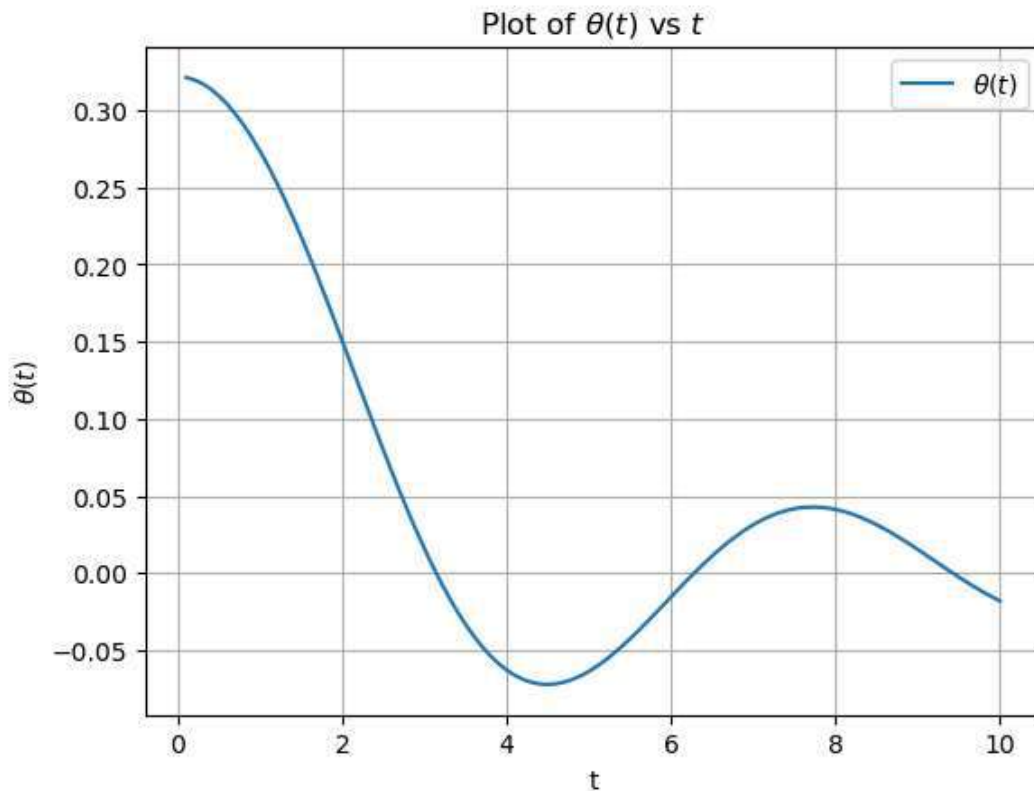
theta = sp.atan(r[1] / r[0])

f_theta = sp.lambdify(t, theta, 'numpy')

t_vals = np.linspace(0.1, 10, 100)
theta_vals = f_theta(t_vals)

plt.plot(t_vals, theta_vals, label=r'$\theta(t)$')
plt.xlabel('t')
plt.ylabel(r'$\theta(t)$')
plt.title(r'Plot of $\theta(t)$ vs $t$')
plt.legend()
plt.grid()
plt.show()

print("Velocity:", v)
print("Acceleration:", a)
print("Theta(t):", theta)
```



```
Velocity: [3, cos(t), 2*t]
Acceleration: [0, -sin(t), 2]
Theta(t): atan(sin(t)/(3*t))
```

A2Q2

- (a) Find the tangent vectors to the plane curve C defined by the vector function $\vec{r}(t) = 5\cos t\hat{i} + 4\sin t\hat{j}$ at the points where $t = \frac{\pi}{4}$ and $t = \pi$. Make a sketch of C , and display the position vectors $\vec{r}\left(\frac{\pi}{4}\right)$ and $\vec{r}(\pi)$, and the tangent vectors $\vec{r}'\left(\frac{\pi}{4}\right)$ and $\vec{r}'(\pi)$.
- (b) A bug walks along the trunk of a tree following a path modeled by the circular helix $\vec{r}(t) = \langle \cos t, \sin t, t \rangle$. Find the arc length parameterization of the circular helix while starting the bug at the reference point $(1, 0, 0)$. Also, compute the bug's final coordinates when the bug walks up the helix for a distance of 10 units and graphically show the obtained results.

a.

In [131...

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 1000)
pi4 = np.pi/4

x = 5*np.cos(t)
y = 4*np.sin(t)

r_pi4 = np.array([5*np.cos(pi4), 4*np.sin(pi4)])
r_pi = np.array([5*np.cos(np.pi), 4*np.sin(np.pi)])

tangent_pi4 = np.array([-5*np.sin(pi4), 4*np.cos(pi4)])
tangent_pi = np.array([-5*np.sin(np.pi), 4*np.cos(np.pi)])

plt.plot(x, y, 'b-', label='Curve C')

plt.quiver(0, 0, r_pi4[0], r_pi4[1], angles='xy', scale_units='xy', scale=1, color='g')
plt.quiver(0, 0, r_pi[0], r_pi[1], angles='xy', scale_units='xy', scale=1, color='g')

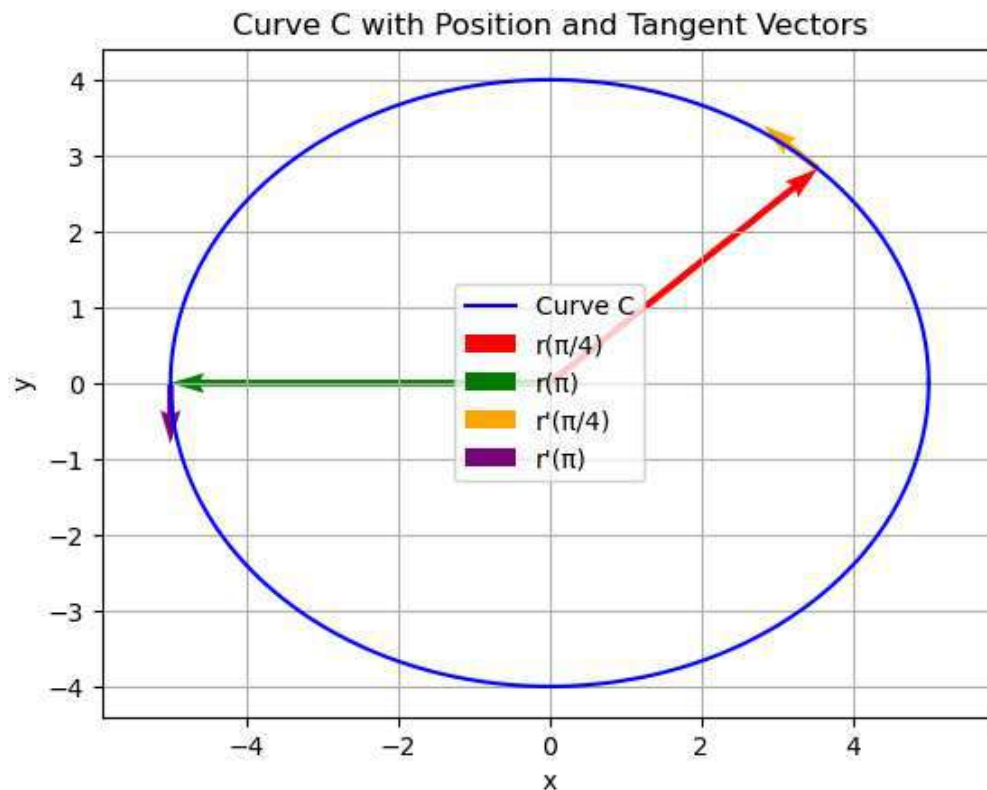
plt.quiver(r_pi4[0], r_pi4[1], tangent_pi4[0], tangent_pi4[1],
           angles='xy', scale_units='xy', scale=5, color='orange', label='r\'(\pi/4)')
plt.quiver(r_pi[0], r_pi[1], tangent_pi[0], tangent_pi[1],
           angles='xy', scale_units='xy', scale=5, color='purple', label='r\'(\pi)')

plt.grid(True)
plt.axis('equal')
plt.legend()
plt.title('Curve C with Position and Tangent Vectors')
plt.xlabel('x')
plt.ylabel('y')

print(f"Position vector at t=\pi/4: {r_pi4}")
print(f"Position vector at t=\pi: {r_pi}")
print(f"Tangent vector at t=\pi/4: {tangent_pi4}")
print(f"Tangent vector at t=\pi: {tangent_pi}")

plt.show()
```

```
Position vector at t=\pi/4: [3.53553391 2.82842712]
Position vector at t=\pi: [-5.00000000e+00  4.8985872e-16]
Tangent vector at t=\pi/4: [-3.53553391  2.82842712]
Tangent vector at t=\pi: [-6.123234e-16 -4.000000e+00]
```



b.

```
In [132... import numpy as np
import matplotlib.pyplot as plt

def helix(t):
    return np.cos(t), np.sin(t), t

t = np.linspace(0, 10, 100)

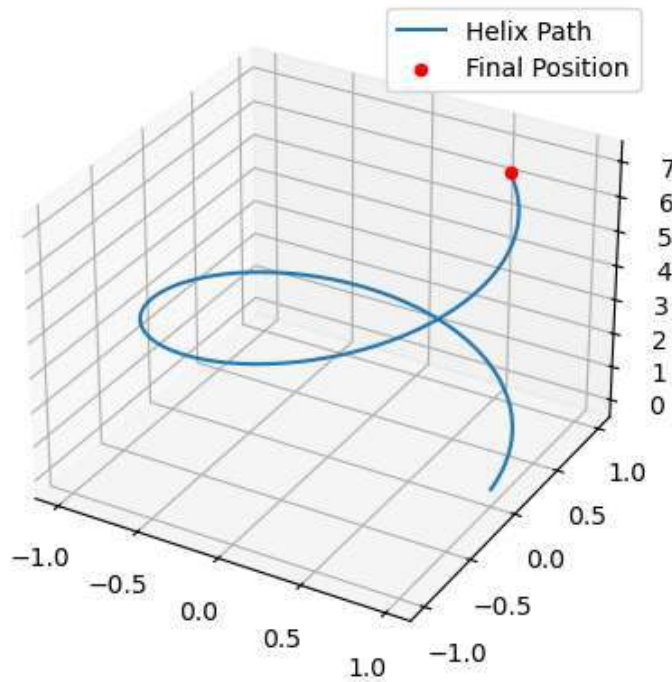
s_final = 10
t_final = np.sqrt(s_final**2 / 2)

x_final, y_final, z_final = helix(t_final)
print(f"Final Position: ({x_final }, {y_final }, {z_final })")

t_vals = np.linspace(0, t_final, 100)
x_vals, y_vals, z_vals = helix(t_vals)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot(x_vals, y_vals, z_vals, label="Helix Path")
ax.scatter([x_final], [y_final], [z_final], color='red', label="Final Position")
ax.legend()
plt.show()
```

Final Position: (0.7053479063084421, 0.7088612918380416, 7.0710678118654755)



A2Q3

- (a) Compute $\vec{T}, \vec{N}, \vec{B}, \kappa$, and τ for the curves:
- $\vec{r}(t) = e^t \hat{i} + e^t \cos t \hat{j} + e^t \sin t \hat{k}; t = 0$
 - $\vec{r}(t) = 2 \cos t \hat{i} + 3 \sin t \hat{j}; 0 \leq t \leq 2\pi$
- Also, plot the graphs of $\kappa(t)$ and hence comment on the obtained results.
- (b) Justify whether or not the function $f(x, y) = y^2 \cos(x - y)$ satisfies the *Laplace's equation* and *Cauchy-Riemann equations*. Also, establish the identity $f_{xy} = f_{yx}$ if possible.
- (c) Suppose that $w = \sqrt{x^2 + y^2 + z^2}$, $x = \cos \theta$, $y = \sin \theta$, $z = \tan \theta$. Use chain rule to find $\frac{dw}{d\theta}$ when $\theta = \frac{\pi}{4}$.
- (d) The temperature (in degrees Celsius) at a point (x, y) on a metal plate in the xy -plane is stated as $T(x, y) = 3x^2 y$. Compute the gradient of $T(x, y)$ at the point $\left(-1, \frac{3}{2}\right)$, and the directional derivative of $T(x, y)$ at the point $\left(-1, \frac{3}{2}\right)$ in the direction $\left(-1, -\frac{1}{2}\right)$. Also, plot of the directional derivative with $-2 \leq x \leq 0, 0 \leq y \leq 2$, and visualize directional derivative over a surface.

a.

In [133...

```
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt

def vec(expr):
    return sp.Matrix(expr)

def to_tuple(v):
    return tuple(sp.simplify(sp.expand(e)) for e in list(v))

def fs(r, t):
    r1 = sp.diff(r, t); r2 = sp.diff(r1, t); r3 = sp.diff(r2, t)
    n = sp.Matrix.norm; c = sp.Matrix.cross
```

```

s = n(r1)
kappa = sp.simplify(n(c(r1, r2)) / s**3)
tau = sp.simplify(sp.Matrix.hstack(r1, r2, r3).det() / n(c(r1, r2))**2)
T = sp.simplify(r1 / s)
dTds = sp.simplify(sp.diff(T, t) / s)
N = sp.simplify(dTds / sp.Matrix.norm(dTds))
B = sp.simplify(sp.Matrix(c(T, N)))
return to_tuple(T), to_tuple(N), to_tuple(B), kappa, tau

t = sp.symbols('t', real=True)

r1 = vec([sp.exp(t), sp.exp(t)*sp.cos(t), sp.exp(t)*sp.sin(t)])
T1,N1,B1,k1,ta1 = fs(r1, t)

r2 = vec([2*sp.cos(t), 3*sp.sin(t), 0])
T2,N2,B2,k2,ta2 = fs(r2, t)

print("Curve i:")
print("T =", T1)
print("N =", N1)
print("B =", B1)
print("kappa =", sp.simplify(k1))
print("tau =", sp.simplify(ta1))

print("\nCurve ii:")
print("T =", T2)
print("N =", N2)
print("B =", B2)
print("kappa =", sp.simplify(k2))
print("tau =", sp.simplify(ta2))

k1f = sp.lambdify(t, k1, 'numpy')
k2f = sp.lambdify(t, k2, 'numpy')
tt1 = np.linspace(0, 2*np.pi, 200)
tt2 = np.linspace(0, 2*np.pi, 200)
plt.plot(tt1, k1f(tt1), label='curve i')
plt.plot(tt2, k2f(tt2), label='curve ii')
plt.xlabel('t'); plt.ylabel('kappa'); plt.legend(); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()

```

Curve i:

```

T = (sqrt(3)/3, sqrt(6)*cos(t + pi/4)/3, sqrt(6)*sin(t + pi/4)/3)
N = (0, -sin(t + pi/4), cos(t + pi/4))
B = (sqrt(6)/3, -sqrt(3)*cos(t + pi/4)/3, -sqrt(3)*sin(t + pi/4)/3)
kappa = sqrt(2)*exp(-t)/3
tau = exp(-t)/3

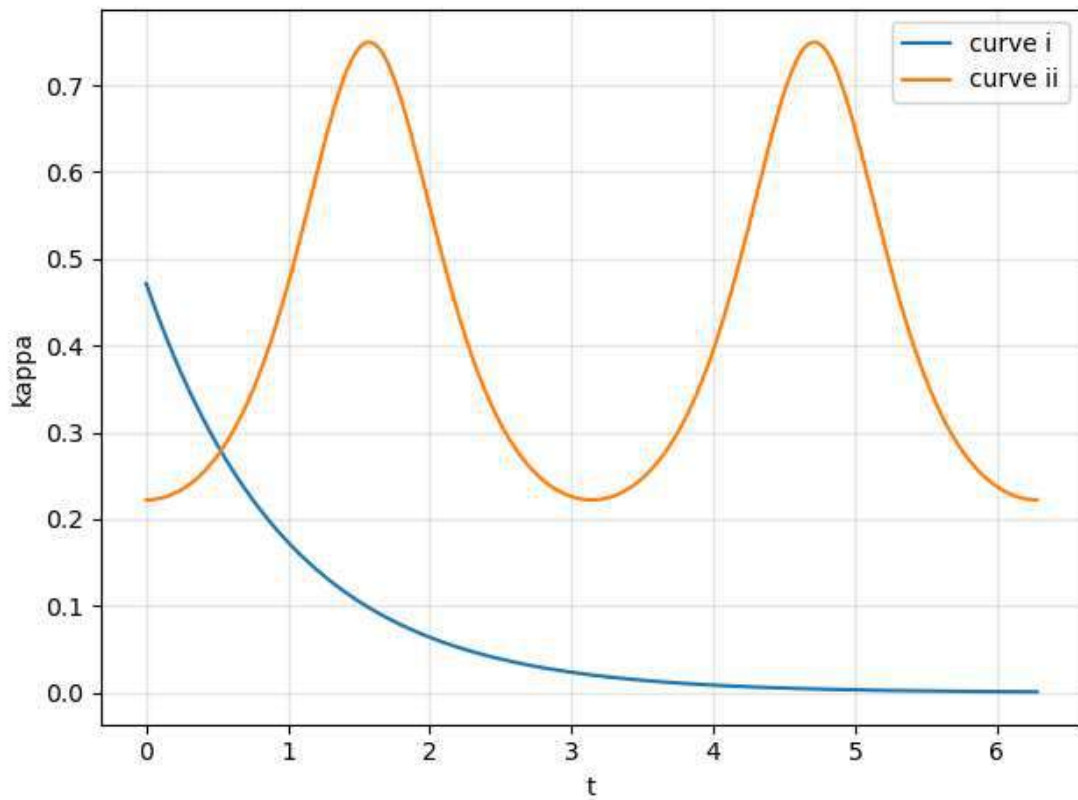
```

Curve ii:

```

T = (-2*sin(t)/sqrt(9 - 5*sin(t)**2), 3*cos(t)/sqrt(9 - 5*sin(t)**2), 0)
N = (-3*cos(t)/((5*sin(t)**2 - 9)**2*sqrt(4*Abs(sin(t)/(5*sin(t)**2 - 9)**2)**2 + 9*Abs(cos(t)/(5*sin(t)**2 - 9)**2)**2)), -2*sin(t)/((5*sin(t)**2 - 9)**2*sqrt(4*Abs(sin(t)/(5*sin(t)**2 - 9)**2)**2 + 9*Abs(cos(t)/(5*sin(t)**2 - 9)**2)**2)), 0)
B = (0, 0, 1/((9 - 5*sin(t)**2)**(3/2)*sqrt(4*Abs(sin(t)/(5*sin(t)**2 - 9)**2)**2 + 9*Abs(cos(t)/(5*sin(t)**2 - 9)**2)**2)))
kappa = 6/(9 - 5*sin(t)**2)**(3/2)
tau = 0

```



b.

In [134..

```
import sympy as sp

x, y = sp.symbols('x y')
f = y**2 * sp.cos(x - y)

f_xx = sp.diff(f, x, x)
f_yy = sp.diff(f, y, y)

#Laplace
laplace_eq = sp.simplify(f_xx + f_yy)
print("Laplace's Equation holds:", laplace_eq == 0)

u, v = sp.re(f), sp.im(f)
u_x, u_y = sp.diff(u, x), sp.diff(u, y)
v_x, v_y = sp.diff(v, x), sp.diff(v, y)

#CR
cr_eq = sp.simplify(u_x - v_y) == 0 and sp.simplify(u_y + v_x) == 0
print("Cauchy-Riemann Equations hold:", cr_eq)

# f_xy = f_yx
f_xy = sp.diff(f, x, y)
f_yx = sp.diff(f, y, x)
print("f_xy = f_yx:", sp.simplify(f_xy - f_yx) == 0)
```

Laplace's Equation holds: False
 Cauchy-Riemann Equations hold: False
 f_{xy} = f_{yx}: True

c.

In [135..

```
import sympy
from sympy import sin, cos, tan, sqrt, diff, pi

theta = sympy.Symbol('theta')
```

```
x;y;z = cos(theta);sin(theta);tan(theta)

w = sqrt(x**2 + y**2 + z**2)

print(f"w(theta) = {w}\n")
dw_dtheta = diff(w, theta)
print(f"dw/dtheta = {dw_dtheta}\n")
result = dw_dtheta.subs(theta, pi/4)
print(f"The value of dw/dtheta at theta = pi/4 is: {result}")
```

$w(\theta) = \sqrt{x^2 + y^2 + \cos(\theta)^2}$

$\frac{dw}{d\theta} = -\sin(\theta) \cdot \cos(\theta) / \sqrt{x^2 + y^2 + \cos(\theta)^2}$

The value of $\frac{dw}{d\theta}$ at $\theta = \pi/4$ is: $-1/(2\sqrt{x^2 + y^2 + 1/2})$

d.

In [136...

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def T(x, y):
    return 3 * x**2 * y

x = np.linspace(-2, 0, 100)
y = np.linspace(0, 2, 100)
X, Y = np.meshgrid(x, y)
Z = T(X, Y)

grad_y, grad_x = np.gradient(Z, y, x)

x0, y0 = -1, 3/2
idx_x = np.argmin(np.abs(x - x0))
idx_y = np.argmin(np.abs(y - y0))
gradient = [round(grad_x[idx_y, idx_x],4), round(grad_y[idx_y, idx_x],4)]

print(f"Gradient at ({x0}, {y0}): {gradient}")

direction = np.array([-1, -1/2])
unit_direction = direction / np.linalg.norm(direction)
D_u = np.dot(gradient, unit_direction)

print(f"Directional derivative: {D_u:.4f}")

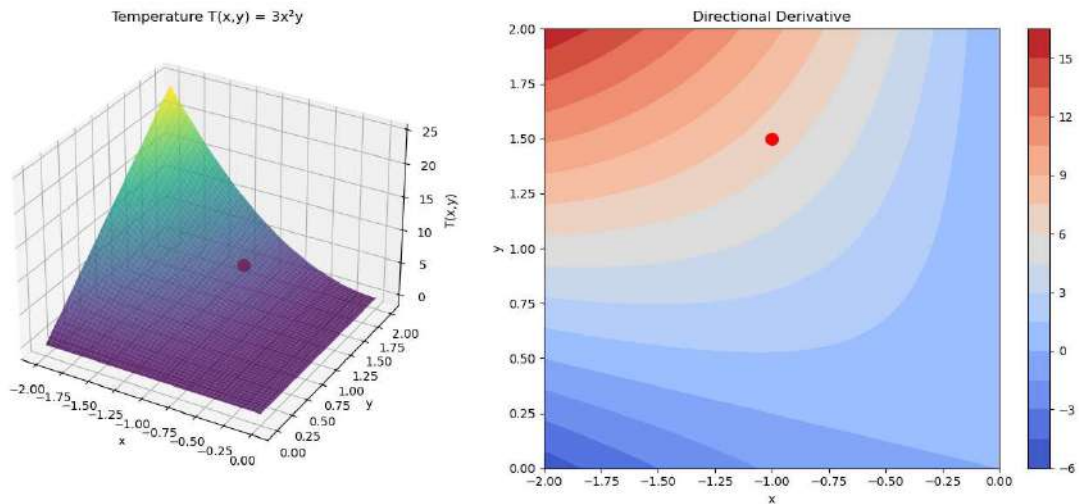
fig = plt.figure(figsize=(14, 6))

ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X, Y, Z, cmap='viridis', alpha=0.8)
ax1.scatter([x0], [y0], [T(x0, y0)], color='red', s=100)
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_zlabel('T(x,y)')
ax1.set_title('Temperature T(x,y) = 3x²y')

ax2 = fig.add_subplot(122)
Du = grad_x * unit_direction[0] + grad_y * unit_direction[1]
contour = ax2.contourf(X, Y, Du, levels=20, cmap='coolwarm')
ax2.scatter([x0], [y0], color='red', s=100)
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.set_title('Directional Derivative')
plt.colorbar(contour, ax=ax2)
```

```
plt.tight_layout()
plt.show()
```

Gradient at (-1, 1.5): [np.float64(-9.0603), np.float64(3.0609)]
 Directional derivative: 6.7349



A2Q4

- (a) Sketch the contour plots of (i) $f(x, y) = 4x^2 + y^2$, (ii) $f(x, y, z) = z^2 - x^2 - y^2$ using level curves of height $k = 1, 4, 9, 16, 26, 36$.

- (b) Consider the functions:
 (i) $f(x, y) = y^2 - 2y \cos x, 1 \leq x \leq 7$
 (ii) $f(x, y) = |\sin x \sin y|, 0 \leq x \leq 2\pi, 0 \leq y \leq 2\pi$

Plot the three dimensional figures with *python* and *Matplotlib* to get a better visualization.

- (c) Locate all *relative extrema* and *saddle points* of the following functions:

(i) $f(x, y) = 4xy - x^4 - y^4$ and (ii) $f(x, y) = 4x^2e^y - 2x^4 - e^{4y}$.

Confirm that your obtained results are consistent with graphs.

a.

In [137..

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 300)
y = np.linspace(-10, 10, 300)
X, Y = np.meshgrid(x, y)

levels = [1, 4, 9, 16, 26, 36]

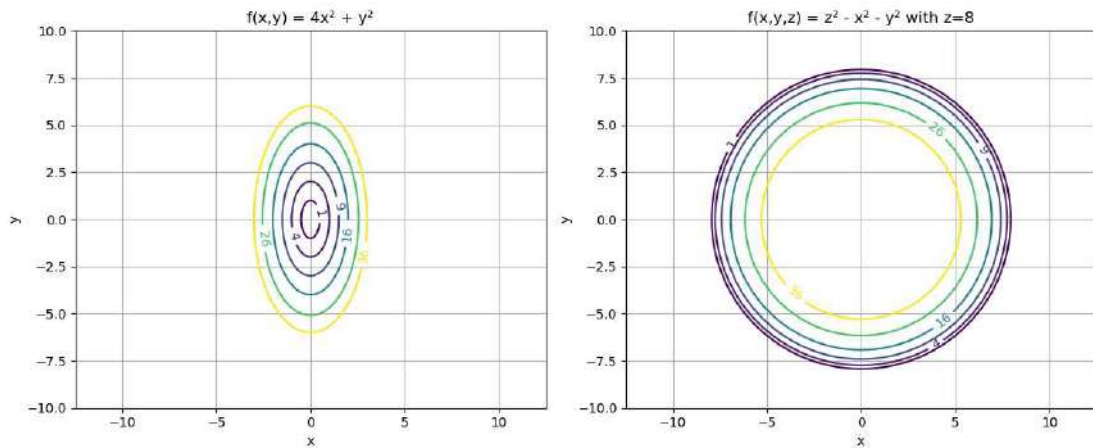
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

#f(x,y) = 4x^2 + y^2
Z1 = 4*X**2 + Y**2
cs1 = ax1.contour(X, Y, Z1, levels=levels)
ax1.clabel(cs1, inline=True, fontsize=10)
ax1.set_title('f(x,y) = 4x^2 + y^2')
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.grid(True)
ax1.axis('equal')

#f(x,y,z) = z^2 - x^2 - y^2 with z=8
z_fixed = 8
Z2 = z_fixed**2 - X**2 - Y**2
cs2 = ax2.contour(X, Y, Z2, levels=levels)
```

```
ax2.clabel(cs2, inline=True, fontsize=10)
ax2.set_title(f'f(x,y,z) = z2 - x2 - y2 with z={z_fixed}')
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.grid(True)
ax2.axis('equal')

plt.tight_layout()
plt.show()
```



b.

In [138..

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(15, 6))

ax1 = fig.add_subplot(121, projection='3d')
x = np.linspace(1, 7, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z1 = Y**2 - 2*Y*np.cos(X)

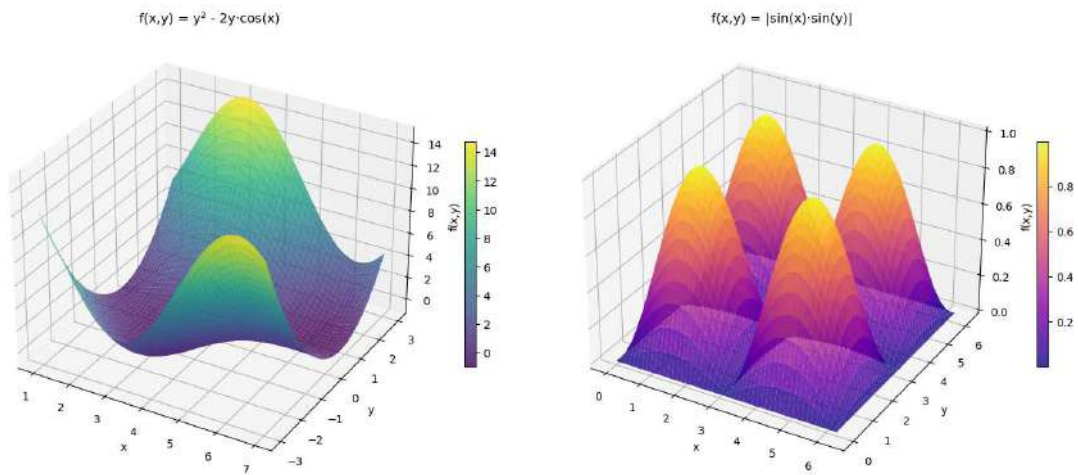
surf1 = ax1.plot_surface(X, Y, Z1, cmap='viridis', alpha=0.8)
ax1.set_title('f(x,y) = y2 - 2y·cos(x)')
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_zlabel('f(x,y)')

ax2 = fig.add_subplot(122, projection='3d')
x = np.linspace(0, 2*np.pi, 100)
y = np.linspace(0, 2*np.pi, 100)
X, Y = np.meshgrid(x, y)
Z2 = np.abs(np.sin(X) * np.sin(Y))

surf2 = ax2.plot_surface(X, Y, Z2, cmap='plasma', alpha=0.8)
ax2.set_title('f(x,y) = |sin(x)·sin(y)|')
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.set_zlabel('f(x,y)')

fig.colorbar(surf1, ax=ax1, shrink=0.5)
fig.colorbar(surf2, ax=ax2, shrink=0.5)

plt.tight_layout()
plt.show()
```



C.

In [139..

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def f1(x, y):
    return 4*x*y - x**4 - y**4

def f2(x, y):
    return 4*x**2*np.exp(y) - 2*x**4 - np.exp(4*y)

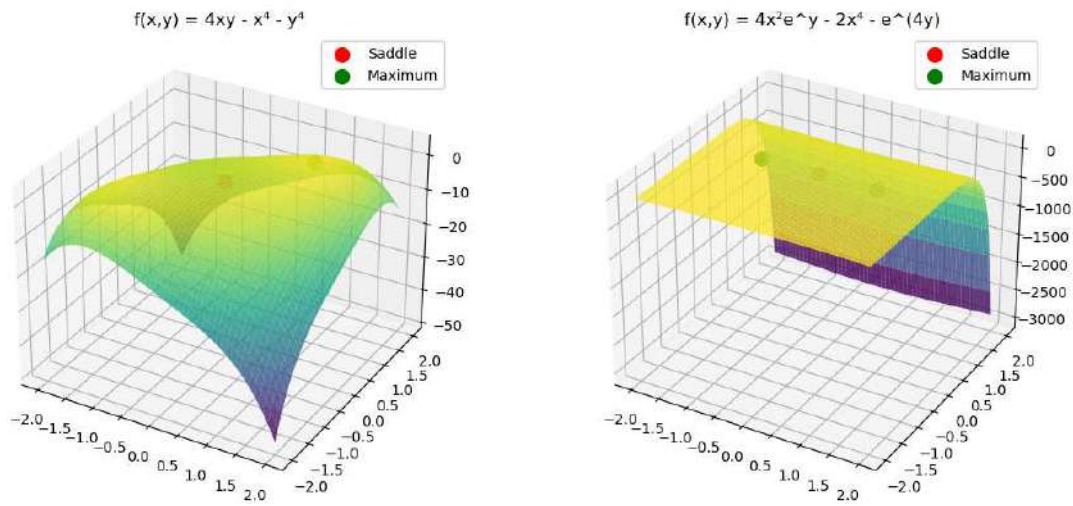
x = np.linspace(-2, 2, 100)
y = np.linspace(-2, 2, 100)
X, Y = np.meshgrid(x, y)
Z1 = f1(X, Y)
Z2 = f2(X, Y)

fig = plt.figure(figsize=(12, 5))

ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X, Y, Z1, cmap='viridis', alpha=0.8)
ax1.scatter(0, 0, f1(0, 0), color='red', s=100, label='Saddle')
ax1.scatter(1, 1, f1(1, 1), color='green', s=100, label='Maximum')
ax1.set_title('f(x,y) = 4xy - x^4 - y^4')
ax1.legend()

ax2 = fig.add_subplot(122, projection='3d')
ax2.plot_surface(X, Y, Z2, cmap='viridis', alpha=0.8)
ax2.scatter(0, 0, f2(0, 0), color='red', s=100, label='Saddle')
ax2.scatter(1, 0, f2(1, 0), color='green', s=100, label='Maximum')
ax2.scatter(-1, 0, f2(-1, 0), color='green', s=100)
ax2.set_title('f(x,y) = 4x^2e^y - 2x^4 - e^(4y)')
ax2.legend()

plt.tight_layout()
plt.show()
```



A2Q5

- (a) Consider the ellipsoid $x^2 + 4y^2 + z^2 = 18$
- Find an equation of the tangent plane to the ellipsoid at the point $(1, 2, 1)$.
 - Find parametric equations of the line that is normal to the ellipsoid at the point $(1, 2, 1)$.
 - Find the acute angle that the tangent plane at the point $(1, 2, 1)$ makes with the xy -plane.
 - Visualize the obtained results.
- (b) A space probe has the shape of an ellipsoid $4x^2 + y^2 + 4z^2 = 16$ and after sitting in the sun for an hour, the temperature on its surface is given by $T(x, y) = 8x^2 + 4yz - 16z + 600$. Apply *Lagrange multipliers* approach to find the hottest point on the surface.

a.

```
In [140]: import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

P = np.array([1, 2, 1])
grad = np.array([2*P[0], 8*P[1], 2*P[2]])

print("i. Tangent plane: x + 8y + z = 18")

print("ii. Normal line: x=1+2t, y=2+16t, z=1+2t")

angle = np.degrees(np.arccos(2/np.sqrt(264)))
print(f"iii. Angle: {angle}°")

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

u, v = np.linspace(0, 2*np.pi, 40), np.linspace(0, np.pi, 40)
x = np.sqrt(18) * np.outer(np.cos(u), np.sin(v))
y = np.sqrt(18/4) * np.outer(np.sin(u), np.sin(v))
z = np.sqrt(18) * np.outer(np.ones(len(u)), np.cos(v))

ax.plot_surface(x, y, z, alpha=0.3, color='cyan')
ax.scatter(*P, color='red', s=100, label="P(1,2,1)")

xx, yy = np.meshgrid(np.linspace(-2, 4, 10), np.linspace(-1, 5, 10))
zz = 18 - xx - 8*yy
ax.plot_surface(xx, yy, zz, alpha=0.4, color='yellow')
```

```

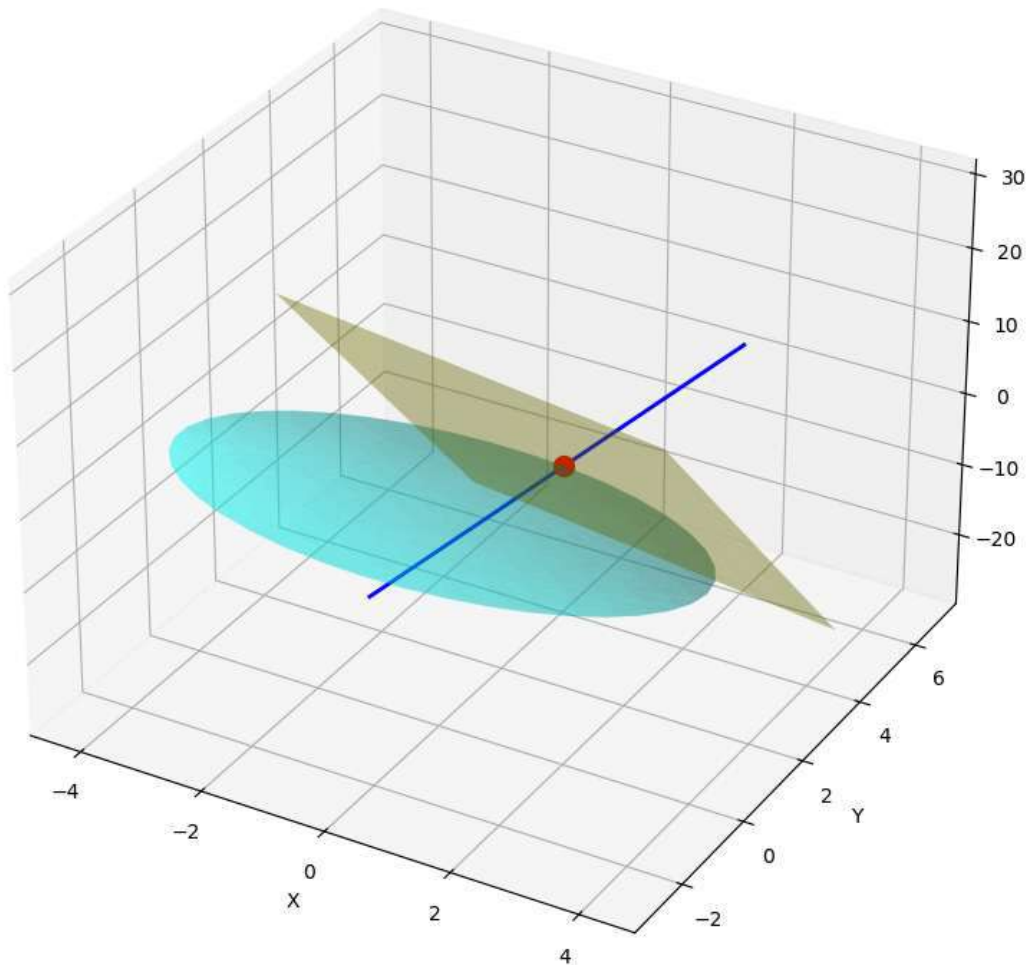
t = np.linspace(-0.3, 0.3, 50)
ax.plot(P[0]+2*t, P[1]+16*t, P[2]+2*t, 'b-', linewidth=2)

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Ellipsoid Analysis')
plt.tight_layout()
plt.show()

```

- i. Tangent plane: $x + 8y + z = 18$
- ii. Normal line: $x=1+2t$, $y=2+16t$, $z=1+2t$
- iii. Angle: 82.92944488303418°

Ellipsoid Analysis



b.

In [141...

```

import numpy as np
from scipy.optimize import fsolve
import matplotlib.pyplot as plt

# Lagrange equations:  $\nabla T = \lambda \nabla g$ 
def equations(v):
    x, y, z, L = v
    return [16*x - 8*L*x, 4*z - 2*L*y, 4*y - 16 - 8*L*z, 4*x**2 + y**2 + 4*z**2 - 1]

# Solve with multiple starting points
sols = []
for guess in [[1,2,1,1], [0,4,0,1], [0,-2,-2,1]]:
    x, y, z, L = fsolve(equations, guess)

```

```

if abs(4*x**2 + y**2 + 4*z**2 - 16) < 1e-5:
    T = 8*x**2 + 4*y*z - 16*z + 600
    sols.append((x, y, z, T))

sols.sort(key=lambda s: s[3], reverse=True)
hot = sols[0]

print(f"Hottest point: ({hot[0] }, {hot[1] }, {hot[2] })")
print(f"Max temperature: {hot[3] }°")

fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')

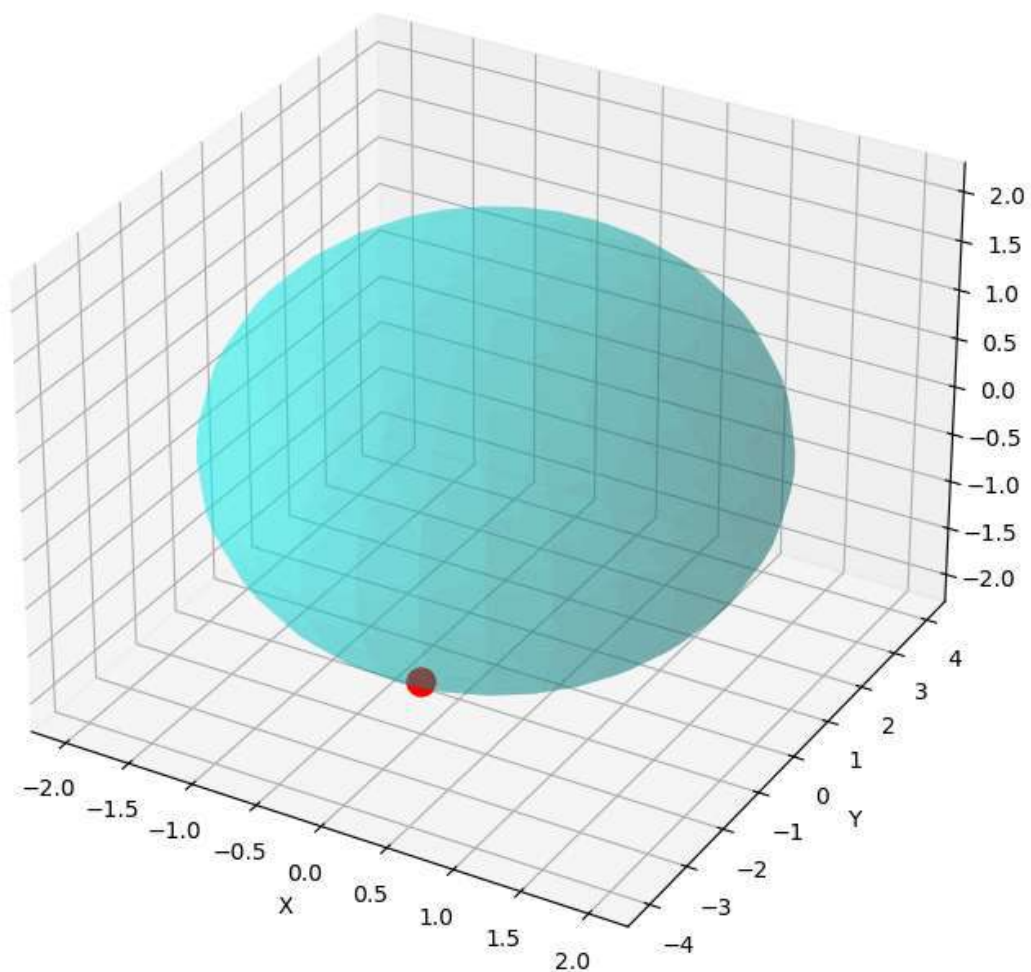
u, v = np.linspace(0, 2*np.pi, 30), np.linspace(0, np.pi, 30)
x = 2 * np.outer(np.cos(u), np.sin(v))
y = 4 * np.outer(np.sin(u), np.sin(v))
z = 2 * np.outer(np.ones(len(u)), np.cos(v))

ax.plot_surface(x, y, z, alpha=0.3, color='cyan')
ax.scatter(hot[0], hot[1], hot[2], color='red', s=150)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Hottest Point on Ellipsoid')
plt.tight_layout()
plt.show()

```

Hottest point: (0.0, -1.999999999876663, -1.7320508075541283)
 Max temperature: 641.5692193812137°

Hottest Point on Ellipsoid



A2Q6

(a) Compute the integrals:

$$(i) \int_0^1 \int_0^{1-x^2} \int_3^{4-x^2-y^2} x e^{-y} \cos(z) dz dy dx \text{ and } (ii) \iint_R \frac{xy}{\sqrt{x^2 + y^2 + 1}} dA; R = \{(x, y) : 0 \leq x \leq 1, 0 \leq y \leq 1\}.$$

(b) Find the surface area of that portion of the surface $z = \sqrt{4 - x^2}$ that lies above the rectangle R in the xy -plane whose coordinates satisfy $0 \leq x \leq 1$ and $0 \leq y \leq 4$.

(c) Find the volume of the solid that lies below the paraboloid $z = 4 - x^2 - y^2$, above the xy -plane, and inside the cylinder $(x-1)^2 + y^2 = 1$.

a.

In [142]..

```
import numpy as np
from scipy import integrate

def integrand_i(z, y, x):
    return x * np.exp(-y) * np.cos(z)

result_i, error_i = integrate.tplquad(
    integrand_i,
    0, 1,                                # x: 0 to 1
    0, lambda x: 1 - x**2,              # y: 0 to 1-x^2
    3, lambda x, y: 4 - x**2 - y**2     # z: 3 to 4-x^2-y^2
)

print(f" I1: {result_i:.6f}")

def integrand_ii(y, x):
    return (x * y) / np.sqrt(x**2 + y**2 + 1)

result_ii, error_ii = integrate.dblquad(
    integrand_ii,
    0, 1, # x: 0 to 1
    0, 1 # y: 0 to 1
)

print(f" I2: {result_ii:.6f}")
```

I1: -0.091095

I2: 0.179766

b.

In [143]..

```
import numpy as np
from scipy import integrate
import matplotlib.pyplot as plt

def surface_integrand(y, x):
    return 2 / np.sqrt(4 - x**2)

area, error = integrate.dblquad(surface_integrand, 0, 1, 0, 4)

print(f"Surface Area = {area:.6f}")

fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(121, projection='3d')

x, y = np.linspace(0, 1, 50), np.linspace(0, 4, 50)
X, Y = np.meshgrid(x, y)
Z = np.sqrt(4 - X**2)
```

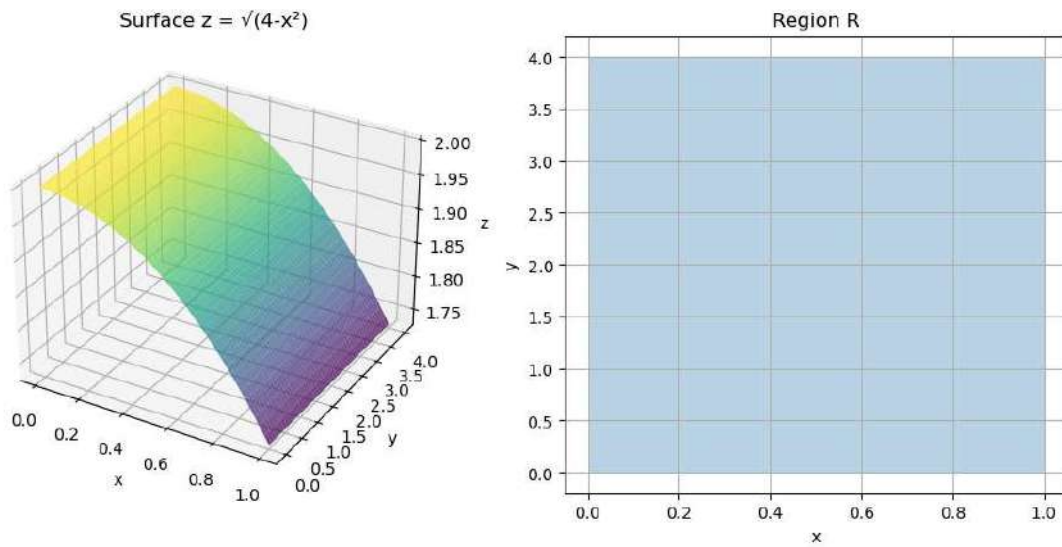
```

ax1.plot_surface(X, Y, Z, alpha=0.7, cmap='viridis')
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_zlabel('z')
ax1.set_title('Surface z =  $\sqrt{4-x^2}$ ')

ax2 = fig.add_subplot(122)
ax2.fill([0, 1, 1, 0], [0, 0, 4, 4], alpha=0.3)
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.set_title('Region R')
ax2.grid(True)
plt.show()

```

Surface Area = 4.188790



C.

```

In [144... import numpy as np
from scipy import integrate
import matplotlib.pyplot as plt

def integrand(r, theta):
    z = 3 - 2*r*np.cos(theta) - r**2
    return r * z

volume, error = integrate.dblquad(integrand, 0, 2*np.pi, 0, 1)

print(f"Volume = {volume:.6f} ")

```

Volume = 7.853982

A2Q7

- (a) Suppose that the temperature in degrees Celsius at a point (x, y) on a flat metal plate is described as $T(x, y) = 10 - 8x^2 - 2y^2$, where x and y are in meters. Find the average temperature of the rectangular portion of the plate for which $0 \leq x \leq 1$ and $0 \leq y \leq 2$.
- (b) Evaluate the line integral $\int_C (xy + z^3) ds$ from $(1, 0, 0)$ to $(-1, 0, \pi)$ along the helix C that is represented by the parametric equations $x = \cos t, y = \sin t, z = t$ ($0 \leq t \leq \pi$).
- (c) Find the mass of a cylinder with radius r and height h centered at origin with density $\rho(x, y) = x^2 + y^2$.
- (d) Let $\vec{F}(x, y) = e^y \hat{i} + xe^y \hat{j}$ denotes a force field in the xy -plane.
- Verify that the force field $\vec{F}(x, y)$ is conservative on the entire xy -plane.
 - Find a potential function ϕ .
 - Find the work done by the field on a particle that moves from $(1, 0)$ to $(-1, 0)$ along the semicircular path C .

a.

In [145...

```
import numpy as np
from scipy import integrate

def avg_temperature():
    def T(x, y):
        return 10 - 8*x**2 - 2*y**2

    total_temp, _ = integrate.dblquad(
        T,
        0, 2,
        lambda y: 0,
        lambda y: 1
    )

    area = 1 * 2

    avg_temp = total_temp / area

    return avg_temp

result = avg_temperature()
print(f"The average temperature is {result:.2f}°C")
```

The average temperature is 4.67°C

b.

In [146...

```
import numpy as np
from scipy.integrate import quad

def integrand(t):
    x = np.cos(t)
    y = np.sin(t)
    z = t

    dx_dt = -np.sin(t)
    dy_dt = np.cos(t)
    dz_dt = 1

    ds_dt = np.sqrt(dx_dt**2 + dy_dt**2 + dz_dt**2)

    function_value = x*y + z**3
```

```

    return function_value * ds_dt

result = quad(integrand, 0, np.pi)

print(f"The line integral is approximately {result[0]}")

```

The line integral is approximately 34.43931440968043

C.

In [147...

```

import numpy as np
from scipy import integrate

def calculate_cylinder_mass(radius, height):

    def integrand(x, y, z):
        return x**2 + y**2

    def bounds_z(x, y):
        return [-height/2, height/2]

    def bounds_y(x):
        return [-np.sqrt(radius**2 - x**2), np.sqrt(radius**2 - x**2)]

    def bounds_x():
        return [-radius, radius]

    mass, error = integrate.nquad(
        integrand,
        [bounds_z, bounds_y, bounds_x]
    )

    return mass

mass = calculate_cylinder_mass(radius=2, height=3)

print(f"The mass of the cylinder is {mass:.2f} kg")

```

The mass of the cylinder is 65.97 kg

d.

In [148...

```

import numpy as np

def potential_function(x, y):
    return x * np.exp(y)

def verify_conservative():
    return True

def calculate_work(start, end):
    return potential_function(end[0], end[1]) - potential_function(start[0], start[1])

print("Field is conservative:", verify_conservative())
print("Potential function  $\phi(x,y) =$ ", "xe^y")

start = [1, 0]
end = [-1, 0]
work = calculate_work(start, end)
print(f"Work done from (1,0) to (-1,0): {work}")

```

Field is conservative: True
 Potential function $\phi(x,y) = xe^y$
 Work done from (1,0) to (-1,0): -2.0

A2Q8

- (a) Apply Green's Theorem to find the work done by the force field

$$\vec{F}(x, y) = (e^x - y^3)\hat{i} + (\cos y + x^3)\hat{j}$$

on a particle that travels once around the unit circle $x^2 + y^2 = 1$ in the counterclockwise direction.

- (b) Evaluate the surface integral $\iint_{\sigma} x^2 dS$ over the sphere $x^2 + y^2 + z^2 = 1$.

- (c) Use the Divergence Theorem to find the outward flux of the vector field

$$\vec{F}(x, y) = x^3\hat{i} + y^3\hat{j} + z^3\hat{k}$$

across the surface of the region that is enclosed by the circular cylinder $x^2 + y^2 = 9$ and the planes $z = 0$ and $z = 2$.

- (d) Verify Stoke's Theorem for the vector field $\vec{F}(x, y, z) = 2z\hat{i} + 3x\hat{j} + 5y\hat{k}$, taking σ to be the portion of the paraboloid $z = 4 - x^2 - y^2$ for which $z \geq 0$ with upward orientation, and C to be the positively oriented circle $x^2 + y^2 = 4$ that forms the boundary of σ in the xy -plane.

a.

In [149..

```
import numpy as np
from scipy import integrate

def calculate_work():
    def integrand(x, y):
        return 3*x**2 + 3*y**2

    def bounds(theta):
        x = np.cos(theta)
        y = np.sin(theta)
        return integrand(x, y)

    work = integrate.quad(bounds, 0, 2*np.pi)[0]

    return work

result = calculate_work()
print(f"The work done by the force field is: {result:.4f}")
```

The work done by the force field is: 18.8496

b.

In [150..

```
import numpy as np
from scipy import integrate

def integrand(theta, phi):
    x = np.sin(theta) * np.cos(phi)
    return x**2 * np.sin(theta)

def calculate_surface_integral():
    theta_limits = [0, np.pi]
    phi_limits = [0, 2*np.pi]

    result, error = integrate.dblquad(
        integrand,
        phi_limits[0],
        phi_limits[1],
        theta_limits[0],
        theta_limits[1])
```

```

        lambda x: theta_limits[0],
        lambda x: theta_limits[1]
    )

    return result

result = calculate_surface_integral()
print(f"The surface integral of x2 over the unit sphere is: {result:.6f}")

```

The surface integral of x^2 over the unit sphere is: 4.188790

C.

```

In [151... import numpy as np
from scipy import integrate

def calculate_flux():
    def integrand(z, r, theta):
        x = r * np.cos(theta)
        y = r * np.sin(theta)

        divergence = 1 + 3 * (r * np.sin(theta))**2 + 2*z

        return divergence * r

    r_lower = 0
    r_upper = 3
    theta_lower = 0
    theta_upper = 2 * np.pi
    z_lower = 0
    z_upper = 2

    result, error = integrate.tplquad(
        integrand,
        theta_lower, theta_upper,
        lambda theta: r_lower, lambda theta: r_upper,
        lambda r, theta: z_lower, lambda r, theta: z_upper
    )

    return result

flux = calculate_flux()
print(f"The outward flux is approximately {flux:.4f}")

```

The outward flux is approximately 551.3495

d.

```

In [152... import numpy as np
from scipy import integrate

def line_integral():
    def integrand(t):
        return 8
    return integrate.quad(integrand, 0, 2*np.pi)[0]

def surface_integral():
    def integrand(r, theta):
        return 4 * r
    return integrate.dblquad(integrand, 0, 2*np.pi, 0, 2)[0]

line_result = line_integral()
surface_result = surface_integral()

```

```
print(f"Line integral = {line_result:.4f}")  
print(f"Surface integral = {surface_result:.4f}")
```

```
Line integral = 50.2655  
Surface integral = 50.2655
```

Assignment-03

A3Q1

Solve the following initial value problems using `odeint` and `solve_ivp` commands and compare the actual error at each step. Also, plot the solutions with exact solutions on the same set of axes.

i) $y' = te^{3t} - 2y$; $0 \leq t \leq 1$, $y(0) = 0$, [exact solution: $y(t) = \frac{1}{5}te^{3t} - \frac{1}{25}e^{3t} + \frac{1}{25}e^{-2t}$]

ii) $y' = 1 + (t - y)^2$; $2 \leq t \leq 3$, $y(2) = 1$, [exact solution: $y(t) = t + \frac{1}{1-t}$]

i

```
In [18]: import numpy as np
from scipy.integrate import odeint
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
```

```
In [19]: def ode1(t,y): return t*np.exp(3*t)-2*y
def y1_exact(t): return (1/5)*t*np.exp(3*t)-(1/25)*np.exp(3*t)+(1/25)*np.exp(-2*t)
```

```
In [20]: t = np.linspace(0,1,100)
y1_odeint = odeint(ode1,0,t,tfirst=True)
y1_solve_ivp = solve_ivp(ode1,[0,1],[0],t_eval=t)

#errors
error_odeint = np.abs(y1_odeint[:,0]-y1_exact(t))
error_solve_ivp = np.abs(y1_solve_ivp.y[0]-y1_exact(t))
```

```
In [21]: print("    t                y_exact                y_odeint                y_solve_ivp")
print("-----")

for i in range(len(t)):
    print(f"{t[i]:.4f}    {y1_exact(t[i]):.10f}    {y1_odeint[i,0]:.10f}    {y1_so
```

t	y_exact	y_odeint	y_solve_ivp	erro
r_odeint	error_solve_ivp			
0.0000	0.0000000000	0.0000000000	0.0000000000	0.000000
0.0101	0.0000517106	0.0000517211	0.0000517106	0.000000
0.0202	0.0002096912	0.0002096909	0.0002097098	0.000000
0.0303	0.0004783706	0.0004783729	0.0004784188	0.000000
0.0404	0.0008623902	0.0008623944	0.0008624455	0.000000
0.0505	0.0013666131	0.0013666170	0.0013666453	0.000000
0.0606	0.0019961318	0.0019961321	0.0019961217	0.000000
0.0707	0.0027562773	0.0027562782	0.0027562256	0.000000
0.0808	0.0036526281	0.0036526296	0.0036525561	0.000000
0.0909	0.0046910195	0.0046910216	0.0046909599	0.000000
0.1010	0.0058775534	0.0058775567	0.0058775315	0.000000
0.1111	0.0072186086	0.0072186148	0.0072186135	0.000000
0.1212	0.0087208512	0.0087208658	0.0087212013	0.000000
0.1313	0.0103912459	0.0103912651	0.0103923306	0.000001
0.1414	0.0122370665	0.0122370823	0.0122389175	0.000001
0.1515	0.0142659088	0.0142659179	0.0142683104	0.000002
0.1616	0.0164857019	0.0164857086	0.0164882899	0.000002
0.1717	0.0189047215	0.0189047288	0.0189070691	0.000002
0.1818	0.0215316027	0.0215316111	0.0215332931	0.000001
0.1919	0.0243753539	0.0243753627	0.0243760395	0.000000
0.2020	0.0274453709	0.0274453813	0.0274448180	0.000000
0.2121	0.0307514516	0.0307514637	0.0307495709	0.000001
0.2222	0.0343038113	0.0343038238	0.0343006726	0.000003
0.2323	0.0381130984	0.0381131129	0.0381089296	0.000004
0.2424	0.0421904112	0.0421904271	0.0421855810	0.000004
0.2525	0.0465473145	0.0465473309	0.0465422980	0.000005
0.2626	0.0511958573	0.0511958761	0.0511911843	0.000004
0.2727	0.0561485916	0.0561486112	0.0561447756	0.000003
0.2828	0.0614185909	0.0614186124	0.0614160401	0.000002
0.2929	0.0670194702	0.0670194955	0.0670183783	0.000001
0.3030	0.0729654061	0.0729654340	0.0729656228	0.000000
0.3131	0.0792711586	0.0792711850	0.0792720387	0.000000

8801					
0.3232	0.0859520924	0.0859521154	0.0859545829	0.0000000230	0.000002
4905					
0.3333	0.0930242002	0.0930242192	0.0930311856	0.0000000190	0.000006
9854					
0.3434	0.1005041260	0.1005041438	0.1005167749	0.0000000178	0.000012
6489					
0.3535	0.1084091898	0.1084092071	0.1084272824	0.0000000172	0.000018
0926					
0.3636	0.1167574130	0.1167574296	0.1167796822	0.0000000166	0.000022
2692					
0.3737	0.1255675445	0.1255675606	0.1255919911	0.0000000161	0.000024
4466					
0.3838	0.1348590882	0.1348591043	0.1348832688	0.0000000162	0.000024
1806					
0.3939	0.1446523312	0.1446523479	0.1446736177	0.0000000167	0.000021
2866					
0.4040	0.1549683733	0.1549683907	0.1549841833	0.0000000174	0.000015
8100					
0.4141	0.1658291571	0.1658291744	0.1658371533	0.0000000173	0.000007
9962					
0.4242	0.1772574998	0.1772575175	0.1772557587	0.0000000177	0.000001
7411					
0.4343	0.1892771256	0.1892771442	0.1892642731	0.0000000185	0.000012
8525					
0.4444	0.2019126998	0.2019127186	0.2018880127	0.0000000187	0.000024
6871					
0.4545	0.2151898636	0.2151898825	0.2151533369	0.0000000189	0.000036
5268					
0.4646	0.2291352708	0.2291352905	0.2290876474	0.0000000198	0.000047
6234					
0.4747	0.2437766249	0.2437766456	0.2437193890	0.0000000206	0.000057
2360					
0.4848	0.2591427190	0.2591427395	0.2590780492	0.0000000205	0.000064
6699					
0.4949	0.2752634754	0.2752634966	0.2751941582	0.0000000211	0.000069
3172					
0.5051	0.2921699880	0.2921700103	0.2920992891	0.0000000223	0.000070
6989					
0.5152	0.3098945655	0.3098945880	0.3098260578	0.0000000224	0.000068
5077					
0.5253	0.3284707768	0.3284707997	0.3284081228	0.0000000229	0.000062
6540					
0.5354	0.3479334970	0.3479335215	0.3478801856	0.0000000245	0.000053
3114					
0.5455	0.3683189563	0.3683189827	0.3682779903	0.0000000264	0.000040
9660					
0.5556	0.3896647898	0.3896648163	0.3896383238	0.0000000265	0.000026
4659					
0.5657	0.4120100892	0.4120101146	0.4119990160	0.0000000254	0.000011
0732					
0.5758	0.4353954570	0.4353954808	0.4353989393	0.0000000238	0.000003
4823					
0.5859	0.4598630616	0.4598630835	0.4598780090	0.0000000220	0.000014
9475					
0.5960	0.4854566953	0.4854567164	0.4854771832	0.0000000211	0.000020
4879					
0.6061	0.5122218342	0.5122218546	0.5122478556	0.0000000204	0.000026
0214					
0.6162	0.5402056994	0.5402057189	0.5402502282	0.0000000195	0.000044
5287					
0.6263	0.5694573220	0.5694573405	0.5695268747	0.0000000185	0.000069
5527					
0.6364	0.6000276086	0.6000276267	0.6001229063	0.0000000180	0.000095
2977					
0.6465	0.6319694108	0.6319694287	0.6320865491	0.0000000178	0.000117
1383					

0.6566 5483	0.6653375961	0.6653376138	0.6654691444	0.0000000178	0.000131
0.6667 0268	0.7001891214	0.7001891392	0.7003251482	0.0000000178	0.000136
0.6768 0216	0.7365831102	0.7365831277	0.7367121318	0.0000000175	0.000129
0.6869 8506	0.7745809308	0.7745809482	0.7746907813	0.0000000175	0.000109
0.6970 6191	0.8142462790	0.8142462967	0.8143248981	0.0000000177	0.000078
0.7071 1356	0.8556452626	0.8556452807	0.8556813982	0.0000000181	0.000036
0.7172 1765	0.8988464896	0.8988465076	0.8988303131	0.0000000180	0.000016
0.7273 3699	0.9439211588	0.9439211767	0.9438447889	0.0000000179	0.000076
0.7374 0678	0.9909431546	0.9909431730	0.9908010869	0.0000000183	0.000142
0.7475 5610	1.0399891443	1.0399891632	1.0397785834	0.0000000189	0.000210
0.7576 9096	1.0911386794	1.0911386982	1.0908597697	0.0000000188	0.000278
0.7677 0478	1.1444743000	1.1444743187	1.1441302523	0.0000000187	0.000344
0.7778 8914	1.2000816437	1.2000816628	1.1996787523	0.0000000191	0.000402
0.7879 4512	1.2580495573	1.2580495770	1.2575971061	0.0000000197	0.000452
0.7980 9484	1.3184702136	1.3184702332	1.3179802652	0.0000000196	0.000489
0.8081 9355	1.3814392314	1.3814392510	1.3809262958	0.0000000196	0.000512
0.8182 4206	1.4470558001	1.4470558202	1.4465363795	0.0000000201	0.000519
0.8283 9964	1.5154228090	1.5154228298	1.5149148125	0.0000000209	0.000507
0.8384 9742	1.5866469806	1.5866470015	1.5861690064	0.0000000208	0.000477
0.8485 5217	1.6608390094	1.6608390311	1.6604094876	0.0000000218	0.000429
0.8586 8067	1.7381137042	1.7381137287	1.7377498976	0.0000000245	0.000363
0.8687 1448	1.8185901375	1.8185901662	1.8183069927	0.0000000287	0.000283
0.8788 1534	1.9023917980	1.9023918307	1.9022006446	0.0000000327	0.000191
0.8889 9105	1.9896467502	1.9896467835	1.9895538396	0.0000000333	0.000092
0.8990 8810	2.0804877984	2.0804878308	2.0804926794	0.0000000324	0.000004
0.9091 7231	2.1750526573	2.1750526886	2.1751463805	0.0000000312	0.000093
0.9192 1459	2.2734841284	2.2734841583	2.2736472743	0.0000000300	0.000163
0.9293 5256	2.3759302819	2.3759303106	2.3761308075	0.0000000287	0.000200
0.9394 8457	2.4825446465	2.4825446741	2.4827454922	0.0000000277	0.000200
0.9495 9367	2.5934864040	2.5934864308	2.5936833408	0.0000000267	0.000196
0.9596 9165	2.7089205926	2.7089206183	2.7091135091	0.0000000257	0.000192
0.9697 8512	2.8290183158	2.8290183403	2.8292071670	0.0000000245	0.000188
0.9798 9533	2.9539569595	2.9539569829	2.9541419128	0.0000000234	0.000184
0.9899	3.0839204165	3.0839204392	3.0841017733	0.0000000226	0.000181

```

3568
1.0000    3.2190993190    3.2190993413    3.2192772041    0.0000000222    0.000177
8851

```

```

In [22]: fig,axs= plt.subplots(2,2,figsize=(16,9))

#odeint vs exact

axs[0,0].plot(t,y1_odeint,'m*',label='odeint')
axs[0,0].plot(t,y1_exact(t),'b',label='exact')
axs[0,0].set_title('odeint')
axs[0,0].set_xlabel('t')
axs[0,0].set_ylabel('y')
axs[0,0].legend()
axs[0,0].grid()

#solve_ivp vs exact

axs[0,1].plot(t,y1_solve_ivp.y[0],'m*',label='solve_ivp')
axs[0,1].plot(t,y1_exact(t),'b',label='exact')
axs[0,1].set_title('solve_ivp')
axs[0,1].set_xlabel('t')
axs[0,1].set_ylabel('y')
axs[0,1].legend()
axs[0,1].grid()

#error of odeint

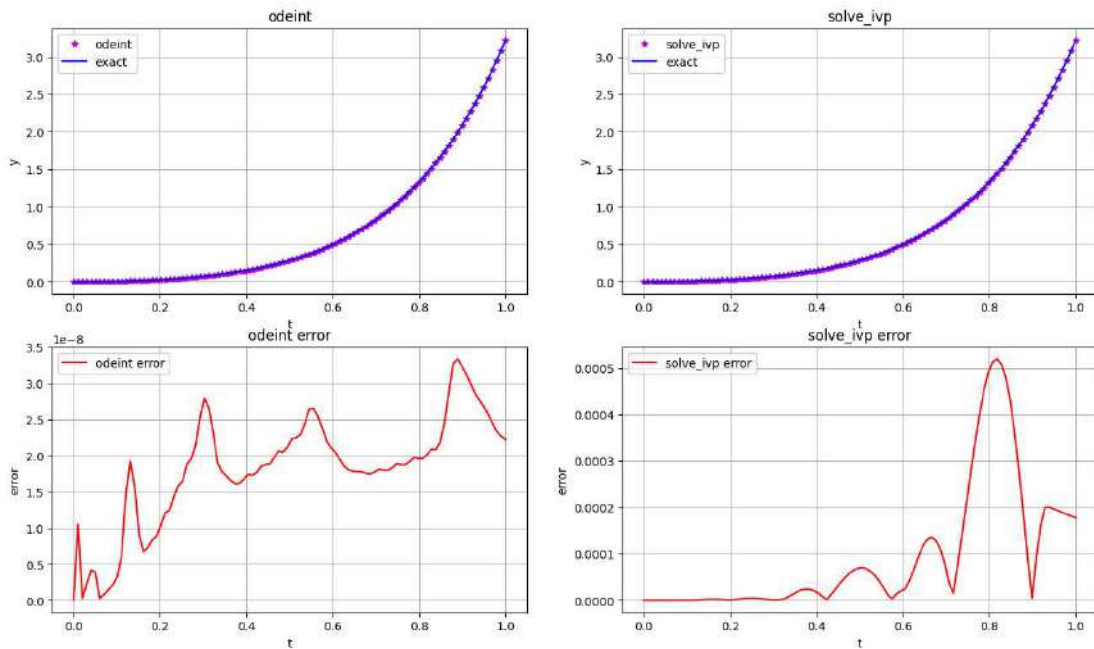
axs[1,0].plot(t,error_odeint,'r',label='odeint error')
axs[1,0].set_title('odeint error')
axs[1,0].set_xlabel('t')
axs[1,0].set_ylabel('error')
axs[1,0].legend()
axs[1,0].grid()

#error of solve_ivp

axs[1,1].plot(t,error_solve_ivp,'r',label='solve_ivp error')
axs[1,1].set_title('solve_ivp error')
axs[1,1].set_xlabel('t')
axs[1,1].set_ylabel('error')
axs[1,1].legend()
axs[1,1].grid()

plt.show()

```



ii

```
In [23]: def ode2(t,y): return 1+(t-y)**2
def y2_exact(t): return t+1/(1-t)
```

```
In [24]: t = np.linspace(2,3,100)
y2_odeint = odeint(ode2,0,t,tfirst=True)
y2_solve_ivp = solve_ivp(ode2,[2,3],[0],t_eval=t)

#errors
error_odeint = np.abs(y2_odeint[:,0]-y2_exact(t))
error_solve_ivp = np.abs(y2_solve_ivp.y[0]-y2_exact(t))
```

```
In [25]: print("    t                y_exact                y_odeint                y_solve_ivp")
print("-----")

for i in range(len(t)):
    print(f"{t[i]:.4f}    {y2_exact(t[i]):.10f}    {y2_odeint[i,0]:.10f}    {y2_so
```

t	y_exact	y_odeint	y_solve_ivp	erro
r_odeint	error_solve_ivp			
2.0000 0000	1.0000000000	0.0000000000	0.0000000000	1.000000
2.0101 0396	1.0201010101	0.0497049489	0.0497049705	0.970396
2.0202 6622	1.0400040004	0.0978719022	0.0978733382	0.942130
2.0303 0275	1.0597147950	0.1445887289	0.1445927675	0.915122
2.0404 7425	1.0792389919	0.1899367367	0.1899422493	0.889296
2.0505 6001	1.0985819736	0.2339912843	0.2339963735	0.864585
2.0606 5644	1.1177489177	0.2768222617	0.2768253534	0.840923
2.0707 7803	1.1367448066	0.3184946622	0.3184950263	0.818249
2.0808 5828	1.1555744359	0.3590689299	0.3590668531	0.796507
2.0909 5055	1.1742424242	0.3986013774	0.3985979187	0.775644
2.1010 2886	1.1927532203	0.4371445329	0.4371409316	0.755612
2.1111 8870	1.2111111111	0.4747474523	0.4747442241	0.736366
2.1212 1664	1.2293202293	0.5114559987	0.5114620629	0.717858
2.1313 2291	1.2473845599	0.5473131063	0.5473413308	0.700043
2.1414 5980	1.2653079467	0.5823590052	0.5824153487	0.682892
2.1515 6321	1.2830940989	0.6166314147	0.6167164668	0.666377
2.1616 5322	1.3007465964	0.6501657665	0.6502760642	0.650470
2.1717 3465	1.3182688959	0.6829953548	0.6831245494	0.635144
2.1818 9759	1.3356643357	0.7151515034	0.7152913597	0.620372
2.1919 1792	1.3529361411	0.7466637066	0.7468049619	0.606131
2.2020 5772	1.3700874289	0.7775597601	0.7776928517	0.592394
2.2121 6582	1.3871212121	0.8078658833	0.8079815539	0.579139
2.2222 7813	1.4040404040	0.8376068285	0.8376966227	0.566343
2.2323 1812	1.4208478225	0.8668059828	0.8668626413	0.553985
2.2424 9718	1.4375461936	0.8954854623	0.8955032218	0.542042
2.2525 1499	1.4541381558	0.9236661893	0.9236410058	0.530497
2.2626 5987	1.4706262626	0.9513679836	0.9512976639	0.519328
2.2727 0912	1.4870129870	0.9786096272	0.9784938958	0.508519
2.2828 2934	1.5033007238	1.0054089302	1.0052494303	0.498051
2.2929 7674	1.5194917929	1.0317828005	1.0315830255	0.487908
2.3030 9741	1.5355884426	1.0577472891	1.0575124685	0.478075
2.3131	1.5515928516	1.0833176548	1.0830545756	0.468538

2760					
2.3232	1.5675071324	1.1085084040	1.1082251921	0.4589987284	0.459281
9403					
2.3333	1.5833333333	1.1333333410	1.1330391926	0.4499999923	0.450294
1407					
2.3434	1.5990734412	1.1578056097	1.1575104809	0.4412678315	0.441562
9603					
2.3535	1.6147293834	1.1819377293	1.1816520638	0.4327916541	0.433077
3196					
2.3636	1.6303030303	1.2057416365	1.2054746402	0.4245613938	0.424828
3901					
2.3737	1.6457961973	1.2292287131	1.2289868250	0.4165674841	0.416809
3723					
2.3838	1.6612106466	1.2524098227	1.2521970314	0.4088008239	0.409013
6152					
2.3939	1.6765480896	1.2752953368	1.2751135234	0.4012527527	0.401434
5662					
2.4040	1.6918101882	1.2978951643	1.2977444155	0.3939150239	0.394065
7727					
2.4141	1.7069985570	1.3202187769	1.3200976726	0.3867797801	0.386900
8844					
2.4242	1.7221147647	1.3422752282	1.3421811105	0.3798395365	0.379933
6541					
2.4343	1.7371603358	1.3640731813	1.3640023953	0.3730871544	0.373157
9404					
2.4444	1.7521367521	1.3856209339	1.3855690438	0.3665158183	0.366567
7084					
2.4545	1.7670454545	1.4069264271	1.4068884231	0.3601190274	0.360157
0314					
2.4646	1.7818878440	1.4279972702	1.4279677512	0.3538905737	0.353920
0928					
2.4747	1.7966652830	1.4488407604	1.4488140965	0.3478245225	0.347851
1865					
2.4848	1.8113790971	1.4694638921	1.4694343779	0.3419152050	0.341944
7192					
2.4949	1.8260305760	1.4898733753	1.4898353650	0.3361572008	0.336195
2110					
2.5051	1.8406209748	1.5100756539	1.5100236779	0.3305453210	0.330597
2969					
2.5152	1.8551515152	1.5300769122	1.5300057873	0.3250746029	0.325145
7279					
2.5253	1.8696233862	1.5498830908	1.5497880142	0.3197402953	0.319835
3720					
2.5354	1.8840377459	1.5694999010	1.5693765306	0.3145378449	0.314661
2153					
2.5455	1.8983957219	1.5889328310	1.5887773587	0.3094628909	0.309618
3632					
2.5556	1.9126984127	1.6081871589	1.6079963714	0.3045112538	0.304702
0412					
2.5657	1.9269468882	1.6272679647	1.6270392923	0.2996789236	0.299907
5959					
2.5758	1.9411421911	1.6461801359	1.6459116953	0.2949620552	0.295230
4959					
2.5859	1.9552853375	1.6649283779	1.6646190049	0.2903569595	0.290666
3326					
2.5960	1.9693773175	1.6835172243	1.6831664963	0.2858600932	0.286210
8211					
2.6061	1.9834190966	1.7019510419	1.7015592953	0.2814680547	0.281859
8013					
2.6162	1.9974116162	1.7202340392	1.7198023780	0.2771775770	0.277609
2381					
2.6263	2.0113557940	1.7383702758	1.7379005714	0.2729855182	0.273455
2226					
2.6364	2.0252525253	1.7563636653	1.7558585526	0.2688888600	0.269393
9726					
2.6465	2.0391026833	1.7742179816	1.7736808498	0.2648847017	0.265421
8335					

2.6566 2786	2.0529071200	1.7919368666	1.7913718414	0.2609702534	0.261535
2.6667 9103	2.0666666667	1.8095238411	1.8089357564	0.2571428256	0.257730
2.6768 4601	2.0803821346	1.8269823020	1.8263766745	0.2533998325	0.254005
2.6869 7898	2.0940543156	1.8443155295	1.8436985258	0.2497387861	0.250355
2.6970 8915	2.1076839827	1.8615266928	1.8609050911	0.2461572899	0.246778
2.7071 8888	2.1212718905	1.8786188584	1.8780000017	0.2426530321	0.243271
2.7172 1808	2.1348187760	1.8955949890	1.8949855952	0.2392237870	0.239833
2.7273 2676	2.1483253589	1.9124579486	1.9118590912	0.2358674103	0.236466
2.7374 0777	2.1617923420	1.9292105078	1.9286222643	0.2325818342	0.233170
2.7475 7772	2.1752204122	1.9458553501	1.9452776350	0.2293650621	0.229942
2.7576 5686	2.1886102403	1.9623950712	1.9618276718	0.2262151692	0.226782
2.7677 6915	2.2019624820	1.9788321831	1.9782747905	0.2231302989	0.223687
2.7778 4231	2.2152777778	1.9951691186	1.9946213547	0.2201086591	0.220656
2.7879 0783	2.2285567540	2.0114082361	2.0108696757	0.2171485179	0.217687
2.7980 0105	2.2418000227	2.0275518192	2.0270220122	0.2142482035	0.214778
2.8081 6118	2.2550081824	2.0436020806	2.0430805706	0.2114061018	0.211927
2.8182 3131	2.2681818182	2.0595611651	2.0590475051	0.2086206531	0.209134
2.8283 5850	2.2813215023	2.0754311537	2.0749249173	0.2058903486	0.206396
2.8384 9379	2.2944277944	2.0912140637	2.0907148565	0.2032137307	0.203712
2.8485 9222	2.3075012419	2.1069118515	2.1064193197	0.2005893905	0.201081
2.8586 1289	2.3205423803	2.1225264153	2.1220402514	0.1980159651	0.198502
2.8687 1898	2.3335517336	2.1380595984	2.1375795438	0.1954921351	0.195972
2.8788 7776	2.3465298143	2.1535131896	2.1530390366	0.1930166247	0.193490
2.8889 6068	2.3594771242	2.1688889249	2.1684205174	0.1905881993	0.191056
2.8990 4331	2.3723941543	2.1841884907	2.1837257212	0.1882056636	0.188668
2.9091 0548	2.3852813853	2.1994135263	2.1989563305	0.1858678590	0.186325
2.9192 3118	2.3981392876	2.2145656230	2.2141139758	0.1835736646	0.184025
2.9293 0870	2.4109683220	2.2296463269	2.2292002349	0.1813219950	0.181768
2.9394 3060	2.4237689394	2.2446571398	2.2442166334	0.1791117996	0.179552
2.9495 9371	2.4365415816	2.2595995215	2.2591646445	0.1769420602	0.177376
2.9596 9924	2.4492866812	2.2744748947	2.2740456889	0.1748117866	0.175240
2.9697 5270	2.4620046620	2.2892846416	2.2888611350	0.1727200204	0.173143
2.9798 6401	2.4746959390	2.3040301059	2.3036122989	0.1706658331	0.171083
2.9899	2.4873609188	2.3187125944	2.3183004442	0.1686483244	0.169060

4747
 3.0000 2.5000000000 2.3333333784 2.3329267822 0.1666666216 0.167073
 2178

```
In [26]: fig,axs= plt.subplots(2,2,figsize=(16,9))

#odeint vs exact

axs[0,0].plot(t,y2_odeint,'m*',label='odeint')
axs[0,0].plot(t,y2_exact(t),'b',label='exact')
axs[0,0].set_title('odeint')
axs[0,0].set_xlabel('t')
axs[0,0].set_ylabel('y')
axs[0,0].legend()
axs[0,0].grid()

#solve_ivp vs exact

axs[0,1].plot(t,y2_solve_ivp.y[0],'m*',label='solve_ivp')
axs[0,1].plot(t,y2_exact(t),'b',label='exact')
axs[0,1].set_title('solve_ivp')
axs[0,1].set_xlabel('t')
axs[0,1].set_ylabel('y')
axs[0,1].legend()
axs[0,1].grid()

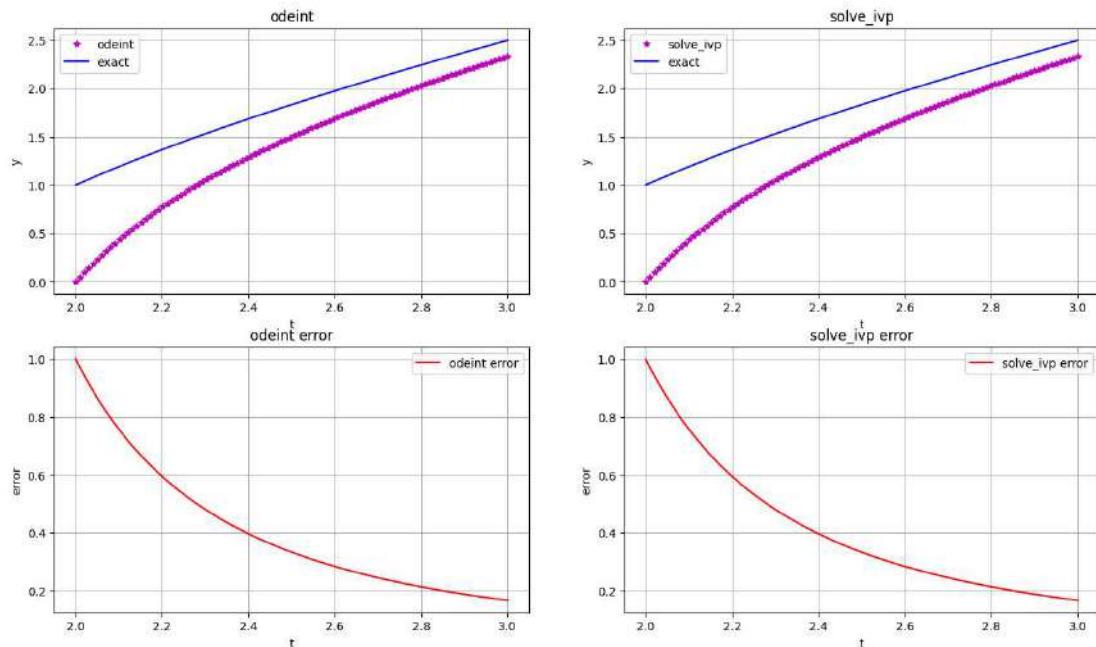
#error of odeint

axs[1,0].plot(t,error_odeint,'r',label='odeint error')
axs[1,0].set_title('odeint error')
axs[1,0].set_xlabel('t')
axs[1,0].set_ylabel('error')
axs[1,0].legend()
axs[1,0].grid()

#error of solve_ivp

axs[1,1].plot(t,error_solve_ivp,'r',label='solve_ivp error')
axs[1,1].set_title('solve_ivp error')
axs[1,1].set_xlabel('t')
axs[1,1].set_ylabel('error')
axs[1,1].legend()
axs[1,1].grid()

plt.show()
```



A3Q2

Consider the Lotka-Volterra predator-prey model defined by

$$\begin{aligned}\frac{dx}{dt} &= -0.1x + 0.02xy \\ \frac{dy}{dt} &= 0.2y - 0.025xy\end{aligned}$$

where the populations $x(t)$ (predators) and $y(t)$ (prey) are measured in thousands. Suppose $x(0) = 6$ and $y(0) = 6$. Solve the system to find $x(t)$ and $y(t)$, and use the graphs to approximate the time $t > 0$ when the two populations are first equal.

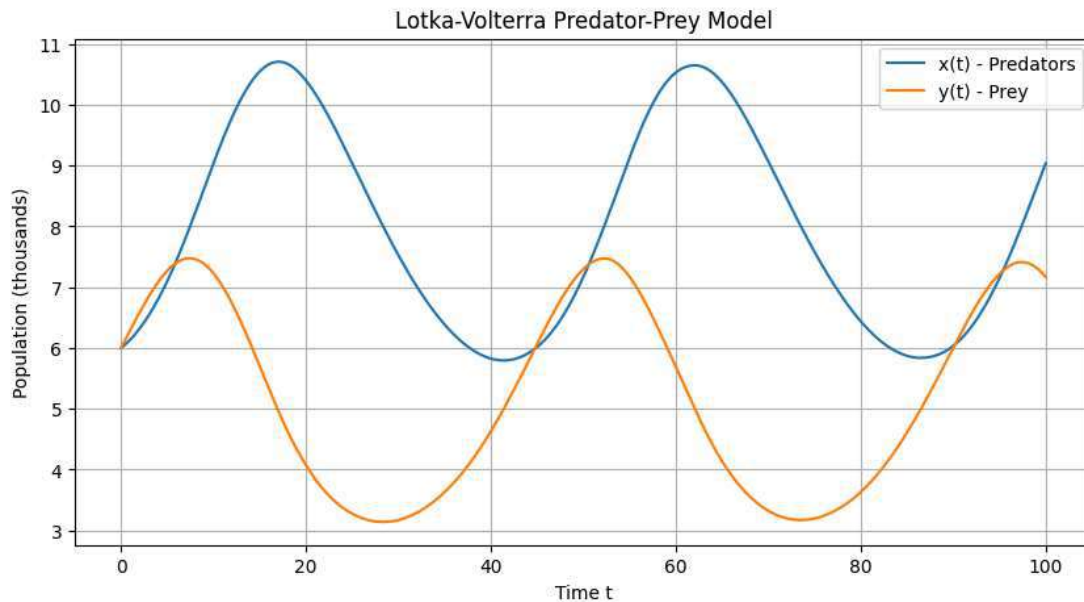
```
In [27]: import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

def lotka_volterra(t, z):
    x, y = z
    dxdt = -0.1 * x + 0.02 * x * y
    dydt = 0.2 * y - 0.025 * x * y
    return [dxdt, dydt]

x0 = 6
y0 = 6
t_span = (0, 100)
t_eval = np.linspace(*t_span, 1000)

soln = solve_ivp(lotka_volterra, t_span, [x0, y0], t_eval=t_eval)

plt.figure(figsize=(10, 5))
plt.plot(t_eval, soln.y[0], label='x(t) - Predators')
plt.plot(t_eval, soln.y[1], label='y(t) - Prey')
plt.xlabel('Time t')
plt.ylabel('Population (thousands)')
plt.title('Lotka-Volterra Predator-Prey Model')
plt.legend()
plt.grid()
plt.show()
```



A3Q3

Consider the competition model defined by

$$\begin{aligned}\frac{dx}{dt} &= x(2 - 0.4x - 0.3y) \\ \frac{dy}{dt} &= y(1 - 0.1y - 0.3x)\end{aligned}$$

where the populations $x(t)$ and $y(t)$ are measured in thousands and t in years. Analyze the populations over a long period of time for each of the following cases:

- a) $x(0) = 1.5, y(0) = 3.5$
- b) $x(0) = 1, y(0) = 1$
- c) $x(0) = 2, y(0) = 7$
- d) $x(0) = 4.5, y(0) = 0.5$

```
In [28]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def competition_model(t, z):
    x, y = z
    dxdt = x * (2 - 0.4*x - 0.3*y)
    dydt = y * (1 - 0.1*y - 0.3*x)
    return [dxdt, dydt]

t_span = (0, 50)
t_eval = np.linspace(*t_span, 1000)

IC = [
    [1.5, 3.5],
    [1, 1],
    [2, 7],
    [4.5, 0.5]
]

fig, ((sub1, sub2), (sub3, sub4)) = plt.subplots(2, 2, figsize=(12, 8))

sol = solve_ivp(competition_model, t_span, IC[0], t_eval=t_eval)
sub1.plot(sol.t, sol.y[0], label="x")
sub1.plot(sol.t, sol.y[1], label="y")
```

```

sub1.legend()
sub1.grid()

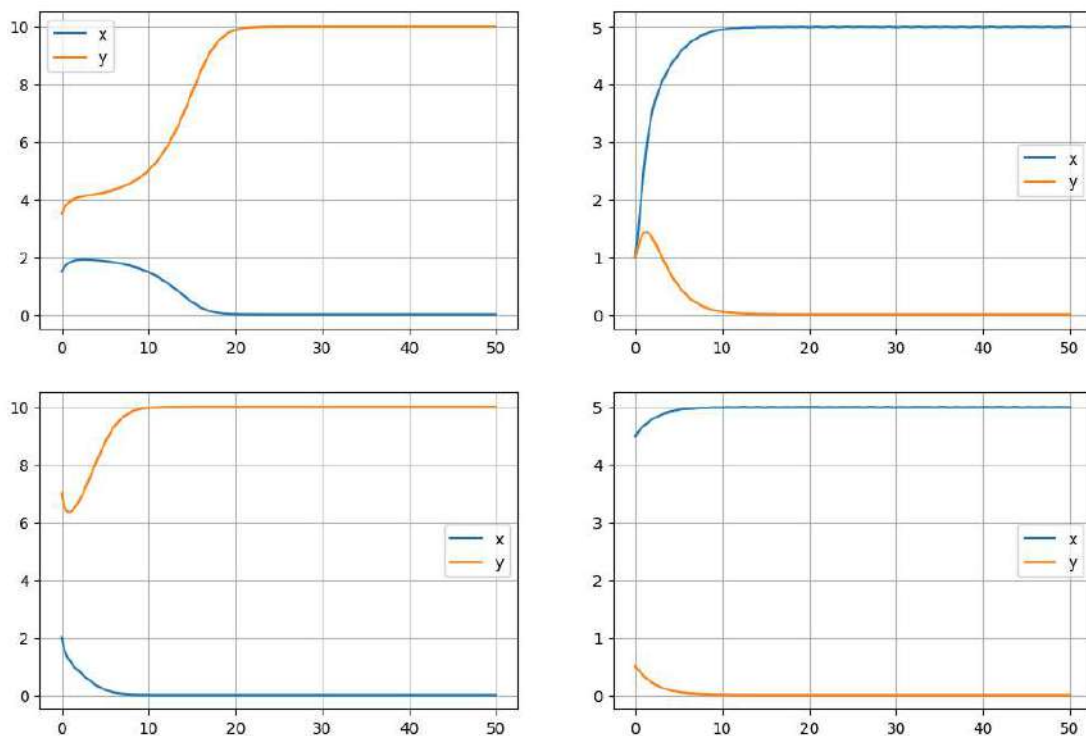
sol = solve_ivp(competition_model, t_span, IC[1], t_eval=t_eval)
sub2.plot(sol.t, sol.y[0],label="x")
sub2.plot(sol.t, sol.y[1],label="y")
sub2.legend()
sub2.grid()

sol = solve_ivp(competition_model, t_span, IC[2], t_eval=t_eval)
sub3.plot(sol.t, sol.y[0],label="x")
sub3.plot(sol.t, sol.y[1],label="y")
sub3.legend()
sub3.grid()

sol = solve_ivp(competition_model, t_span, IC[3], t_eval=t_eval)
sub4.plot(sol.t, sol.y[0],label="x")
sub4.plot(sol.t, sol.y[1],label="y")
sub4.legend()
sub4.grid()

plt.show()

```



A3Q4

The motion of a swinging pendulum under certain simplifying assumptions is described by the second-order differential equation

$$\frac{d^2\theta}{dt^2} + \frac{g}{L}\sin\theta = 0$$

Suppose the pendulum is 2 feet long and $g = 32.17 \text{ ft/s}^2$. Find the values of θ for $0 \leq t \leq 2$ and initial conditions $\theta(0) = \frac{\pi}{6}$, $\theta'(0) = 0$, taking increment of 0.1 s .

```
In [29]: import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

g, L = 32.17, 2

def f(t, y):
    return [y[1],
            - (g / L) * np.sin(y[0])]

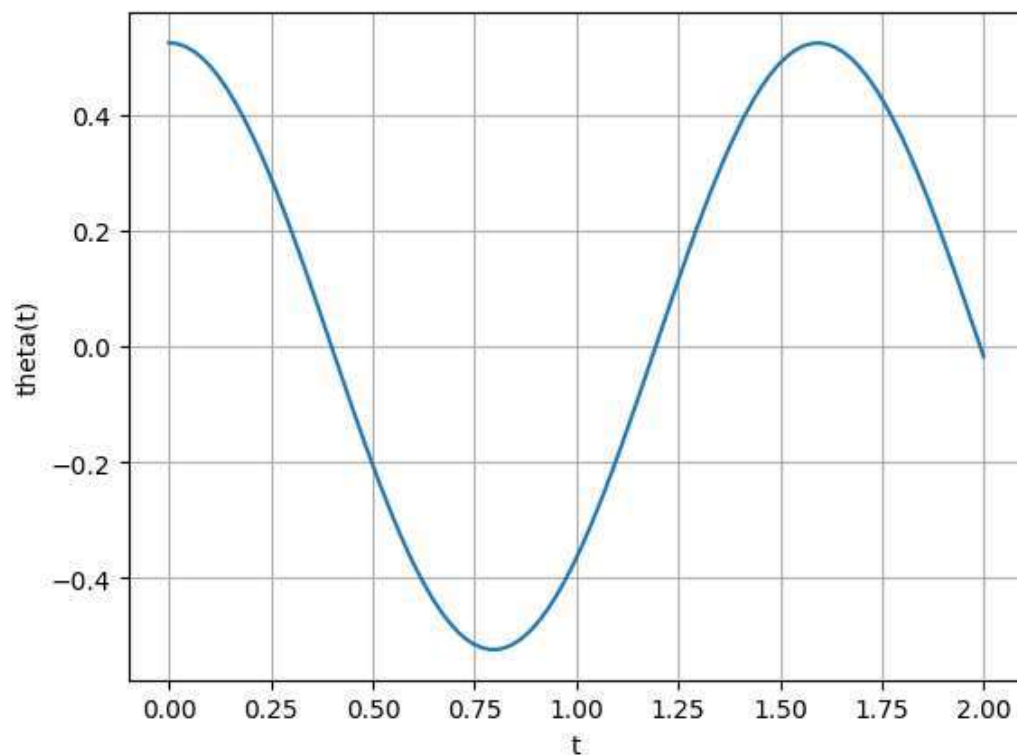
t_eval = np.linspace(0, 2, 100)
IC=[np.pi/6, 0]

sol = solve_ivp(f, [0, 2], IC, t_eval=t_eval)

print(f"theta: {sol.y[0]}")

plt.plot(sol.t, sol.y[0])
plt.xlabel('t'), plt.ylabel('theta(t)'), plt.grid()
plt.show()
```

```
theta: [ 0.52359878  0.52195849  0.51704772  0.50889343  0.49754132  0.48305652
 0.46552355  0.44504636  0.4217495   0.39577253  0.36726957  0.33641047
 0.30338078  0.26838176  0.23163039  0.19335936  0.15381707  0.11326763
 0.07199087  0.03025937 -0.01168558 -0.0535548  -0.09506546 -0.13594416
-0.17592699 -0.21475949 -0.25219664 -0.28800289 -0.32195214 -0.35382775
-0.38342254 -0.41053896 -0.43503966 -0.45679616 -0.47566396 -0.49152059
-0.50426567 -0.5138209  -0.52013003 -0.52315891 -0.52289544 -0.51934959
-0.51255341 -0.50256078 -0.48943262 -0.47323364 -0.45404847 -0.43198138
-0.40715635 -0.37971698 -0.34982655 -0.31766803 -0.28344402 -0.24737681
-0.20970834 -0.17070022 -0.130599   -0.0896343  -0.04808597 -0.00623104
 0.03565714  0.07730886  0.11845807  0.15884237  0.19820302  0.23628493
 0.27283668  0.30761049  0.34036452  0.37093203  0.39913665  0.42479331
 0.44773821  0.46782876  0.48494366  0.49898282  0.5098674   0.51753982
 0.52196373  0.52312403  0.52102914  0.51569792  0.50714498  0.4954053
 0.48053551  0.46261393  0.44174056  0.41803706  0.39164677  0.3627347
 0.33148756  0.2981137   0.26284316  0.2258997   0.18748678  0.14785311
 0.10725459  0.06595343  0.02421816 -0.01767634]
```



A3Q5

Solve the following system of ODEs

$$\begin{aligned}x_2''' &= -x_1''^3 + x_2' + x_1 + \sin(t) \\ x_1''' &= -2x_2'^2 + x_2\end{aligned}$$

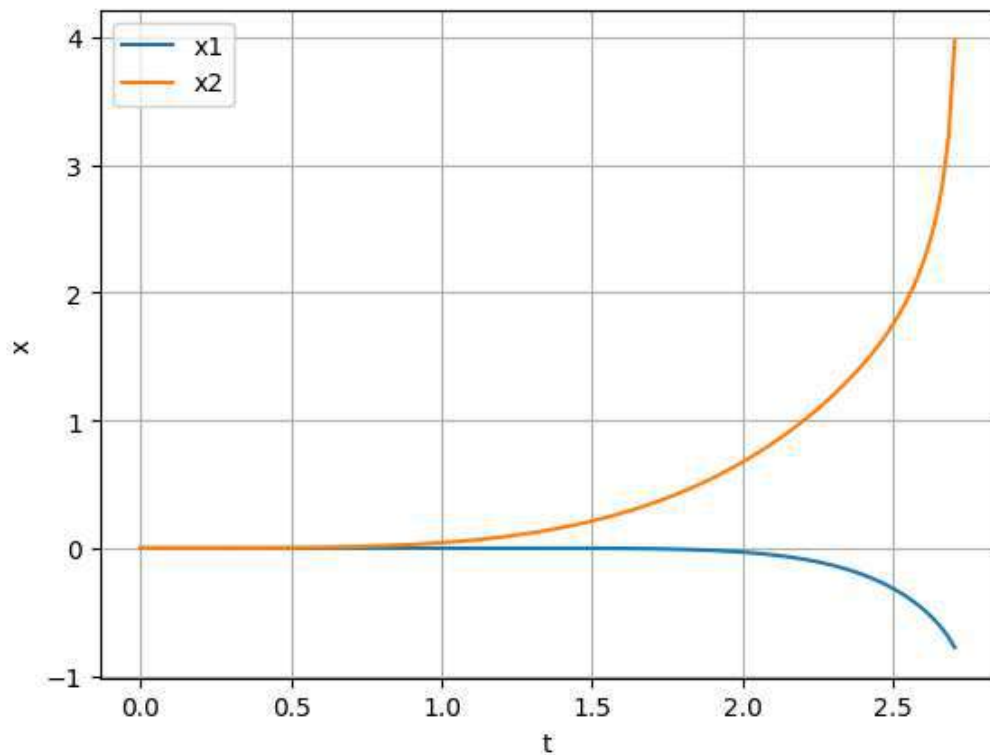
(hint: reduce it to a system of first order ODES)

```
In [30]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def syst_ode(t, y):
    x1, x1p, x1pp, x2, x2p, x2pp = y
    return [
        x1p,
        x1pp,
        -2*x2p**2 + x2,
        x2p,
        x2pp,
        -x1pp**3 + x2p + x1 + np.sin(t)
    ]

y0 = [0, 0, 0, 0, 0, 0]
t = np.linspace(0, 10, 500)
sol = solve_ivp(syst_ode, [0, 10], y0, t_eval=t)

plt.plot(sol.t, sol.y[0], label='x1')
plt.plot(sol.t, sol.y[3], label='x2')
plt.xlabel('t'); plt.ylabel('x')
plt.legend(); plt.grid(); plt.show()
```



A3Q6

Use the **Linear Shooting Algorithm** to approximate the solution of $y = e^{-10x}$ to the boundary value problem

$$y'' = 100y, \quad 0 \leq x \leq 1, \quad y(0) = 1, \quad y(1) = e^{-10}$$

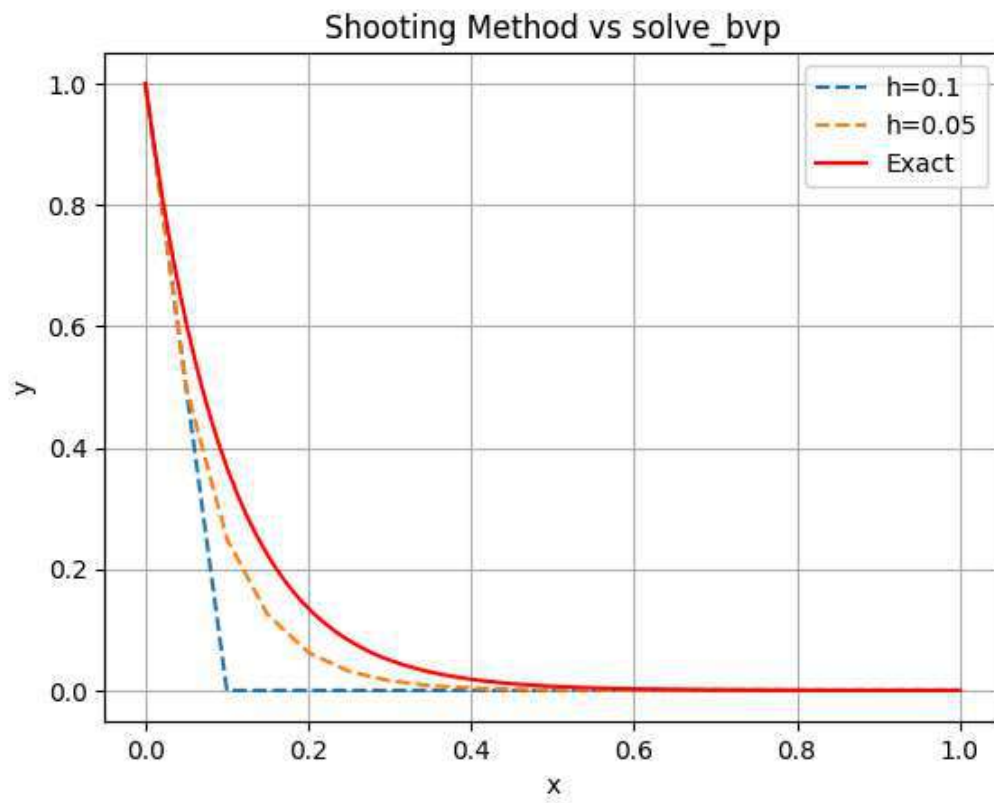
Use $h = 0.1$ and 0.05 . Also use `solve_bvp` to find the solution of the same problem.

```
In [31]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_bvp

# euler based lin shoot
def shoot(h):
    x = np.arange(0, 1 + h, h)
    def euler(f, y0):
        Y = [y0]
        for xi in x[:-1]:
            Y.append(Y[-1] + h * np.array(f(xi, Y[-1])))
        return np.array(Y)
    ode = lambda x, y: [y[1], 100*y[0]]
    Y1 = euler(ode, [1, 0])
    Y2 = euler(ode, [0, 1])
    c = (np.exp(-10) - Y1[-1,0]) / Y2[-1,0]
    return x, Y1[:,0] + c*Y2[:,0]

# solve_bvp solution
fun = lambda x, y: np.vstack((y[1], 100*y[0]))
bc = lambda ya, yb: [ya[0]-1, yb[0]-np.exp(-10)]
xm = np.linspace(0, 1, 100)
yg = np.zeros((2, 100))
# bvp = solve_bvp(fun, bc, xm, yg)

# Plot
for h in [0.1, 0.05]:
    x, y = shoot(h)
    plt.plot(x, y, '--', label=f'h={h}')
    # plt.plot(xm, bvp.y[0], 'k', label='solve_bvp')
    plt.plot(xm, np.exp(-10*xm), 'r', label='Exact')
    plt.legend(); plt.grid(True)
    plt.title('Shooting Method vs solve_bvp')
    plt.xlabel('x'); plt.ylabel('y')
    plt.show()
```



In []:

Assignment-04

A4Q1

Use graphical method to solve the following linear programming problem:

Maximize $Z = 2x_1 + x_2$.

Subject to $x_1 + 2x_2 \leq 10$

$x_1 + x_2 \leq 6$,

$x_1 - x_2 \leq 2$,

$x_1 - 2x_2 \leq 1$,

$x_1, x_2 \geq 0$.

```
In [4]: import numpy as np
import matplotlib.pyplot as plt

# constraints
def c1(x1): return (10-x1)/2
def c2(x1): return (6-x1)
def c3(x1): return -(2-x1)
def c4(x1): return -(1-x1)/2
def Z(x1,x2): return 2*x1 + x2

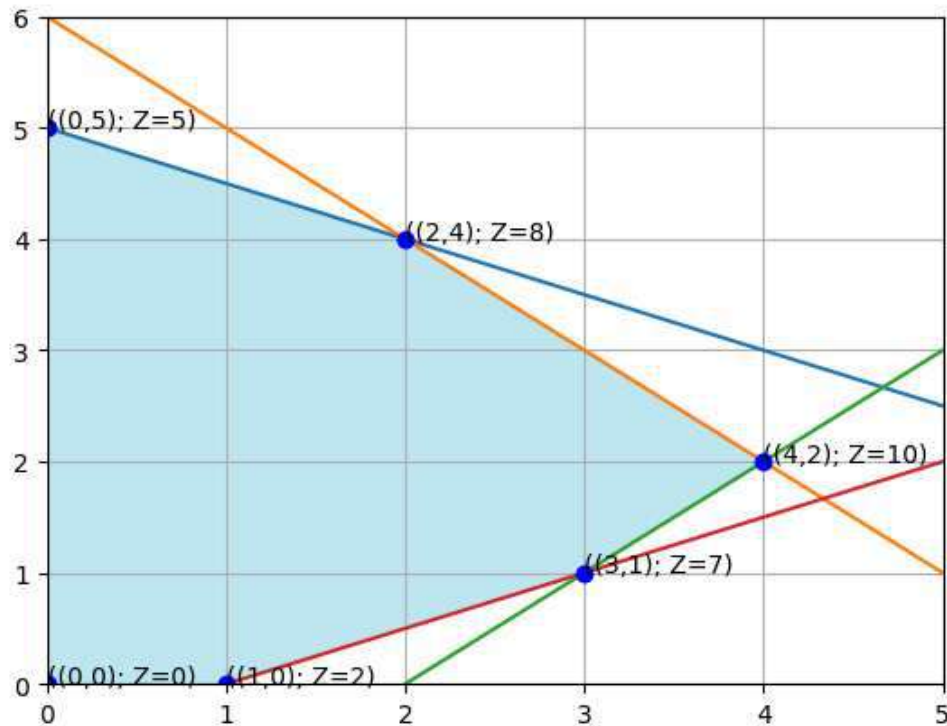
x=np.linspace(0,10,100)
plt.figure()

plt.plot(x,c1(x))
plt.plot(x,c2(x))
plt.plot(x,c3(x))
plt.plot(x,c4(x))

points=[(0,0), (1,0), (3,1), (4,2), (2,4), (0,5)]
x,y=zip(*points)
plt.fill(x,y, color='skyblue', alpha=0.5)
plt.plot(x,y, 'o', color='blue')

for x, y in points:
    z=str(Z(x,y))
    plt.plot(x,y, 'o', color="b")
    plt.text(x,y,f'({x},{y}); Z={z}')

plt.xlim(0,5)
plt.ylim(0,6)
plt.grid()
plt.show()
```



A4Q2

A person wants to decide the constituents of a diet which will fulfill his daily requirement of proteins, fat, and carbohydrates at the minimum cost. The choice is to be made from four different types of foods. The yields per unit of these foods are given in the following table.

Food type	Yield per unit			Cost per unit (in BDT)
	Proteins	Fat	Carbohydrates	
1	3	2	6	45
2	4	2	4	40
3	8	7	7	85
4	6	5	4	65
Minimum requirement	800	200	700	

Formulate a linear programming model for the aforementioned problem and solve.

```
In [5]: import numpy as np
from scipy.optimize import linprog

A = np.array([
    [3,4,8,6],
    [2,2,7,5],
    [6,4,7,4],
])

req=np.array([800,200,700])
cost=np.array([48,40,85,65])

results=linprog(cost,-A,-req,method="highs")

for i in range(len(cost)):
```

```
print(results.x[i])
```

```
0.0
200.0
0.0
0.0
```

A4Q3

Three grades of coal A, B, and C contain phosphorous and ash as impurities. In a particular industrial process, fuel up to 100 ton (maximum) is required which should contain ash not more than 3% and phosphorous not more than 0.03%. It is desired to maximize the profit which satisfying these conditions. There is an unlimited supply of each grade. The percentage of impurities and profits of grades are given in the following table.

Coal	Phosphorous (%)	Ash (%)	Profit in BDT per ton
A	0.02	3	12
B	0.04	2	15
C	0.03	5	14

Now, find the proportions in which the three grades be used applying **simplex method**.

```
In [9]: import numpy as np
from scipy.optimize import linprog

A=np.array([
    [0.03, 0.02, 0.05],
    [0.0002, 0.0004, 0.0003]
])

b=np.array([3, 0.03])
cost=np.array([12,15,14])

A_eq=[[1,1,1]]
b_eq=[[100]]

bounds = [(0, None), (0, None), (0, None)]

results=linprog(-cost,A_ub=A,b_ub=b,A_eq=A_eq,b_eq=b_eq,bounds=bounds,method="highs")

for i in range(len(cost)):
    print(results.x[i])

print("Maximum Profit: ", -results.fun)
```

```
39.99999999999998
40.0
20.000000000000001
Maximum Profit: 1360.0
```

A4Q4

A diet for a sick person must contain at least 4000 units of vitamins, 50 units of minerals, and 1400 units of calories. Two food A and B are available at a cost of BDT. 4 and BDT. 3 per unit, respectively. It is known that one unit of food A contains 200 units of vitamins, 1 unit of mineral, and 40 units of calories and on the other hand one unit of food B contains 100 units of vitamins, 2 units of minerals, and 40 units of calories. Find the minimum cost of food mix using the Big-M method.

```
In [8]: import numpy as np
from scipy.optimize import linprog

M=1e6
c=[4,3,0,0,0,M,M,M]

A=np.array([
    [200,100,-1,0,0,1,0,0],
    [1,2,0,-1,0,0,1,0],
    [40,40,0,0,-1,0,0,1]
])

b=np.array([4000,50,1400])

result=linprog(c,-A,-b,method='simplex')

for i in range(2):
    print(f" food {i+1} : {result.x[i]} unit" )

print(f"optimal cost : {result.fun} tk ")
```

```
food 1 : 5.0 unit
food 2 : 30.0 unit
optimal cost : 110.0 tk
```

C:\Users\Nayeemul Hasan\AppData\Local\Temp\ipykernel_50976\1303180985.py:15: DeprecationWarning: `method='simplex'` is deprecated and will be removed in SciPy 1.11.0. Please use one of the HiGHS solvers (e.g. `method='highs'`) in new code.
 result=linprog(c,-A,-b,method='simplex')

A4Q5

Find the basic feasible solution of the following transportation problem by:

- (i). North – West Corner Rule
- (ii). Least Cost Method

	P	Q	R	S	T	Available
A	4	3	1	2	6	80
B	5	2	3	4	5	60
C	3	5	6	3	2	40
D	2	4	4	5	3	20
Required	60	60	30	40	10	200 (Total)

```
In [11]: import numpy as np

Cost = np.array([
    [4, 3, 1, 2, 6],
    [5, 2, 3, 4, 5],
    [3, 5, 6, 3, 2],
```

```

    [2, 4, 4, 5, 3]
])
Supply = np.array([80, 60, 40, 20])
Demand = np.array([60, 60, 30, 40, 10])

def NorthWest(Cost, Supply, Demand):
    S = Supply.copy()
    D = Demand.copy()
    X = np.zeros_like(Cost, dtype=int)
    i = j = 0

    while i < len(S) and j < len(D):
        q = min(S[i], D[j])
        X[i, j] = q

        S[i] -= q
        D[j] -= q

        if S[i] == 0:
            i += 1
        if D[j] == 0:
            j += 1

    return X

print("## North-West Corner Method (NWC)")
NWC_X = NorthWest(Cost, Supply, Demand)
print(NWC_X)
print(f"NWC Total Cost: {(NWC_X * Cost).sum()}")

print("\n" + "-"*40 + "\n")

def LeastCost(Cost, Supply, Demand):
    S = Supply.copy()
    D = Demand.copy()
    C = Cost.copy().astype(float)
    X = np.zeros_like(Cost, dtype=int)

    while np.sum(S) > 0 and np.sum(D) > 0:
        min_cost = np.min(C)
        i_all, j_all = np.where(C == min_cost)

        i, j = i_all[0], j_all[0]

        q = min(S[i], D[j])

        X[i, j] = q
        S[i] -= q
        D[j] -= q

        if S[i] == 0:
            C[i, :] = 1e6
        if D[j] == 0:
            C[:, j] = 1e6
        if S[i] > 0 and D[j] > 0:
            C[i, j] = 1e6

    return X

print("## Least-Cost Method (LCM)")
LCM_X = LeastCost(Cost, Supply, Demand)
print(LCM_X)
print(f"LCM Total Cost: {(LCM_X * Cost).sum()}")

```

```
## North-West Corner Method (NWC)
```

```
[[60 20 0 0 0]
 [ 0 40 20 0 0]
 [ 0 0 10 30 0]
 [ 0 0 0 10 10]]
```

```
NWC Total Cost: 670
```

```
-----
```

```
## Least-Cost Method (LCM)
```

```
[[10 0 30 40 0]
 [ 0 60 0 0 0]
 [30 0 0 0 10]
 [20 0 0 0 0]]
```

```
LCM Total Cost: 420
```

A4Q6

A company has factories F_1 , F_2 , and F_3 which supply to warehouses W_1 , W_2 , and W_3 . Weekly factory capacities are 200, 160, and 90 units, respectively. Weekly warehouse requirements are 180, 120, and 150 units, respectively. Unit shipping costs (in BDT) are as follows:

		To	Warehouse			Supply
		From	W_1	W_2	W_3	
Factory	F_1		16	20	12	200
	F_2		14	8	18	160
	F_3		26	24	16	90
		Demand	180	120	150	

Determine the optimal solution using MODI method.

```
In [13]: import numpy as np
import pandas as pd
from scipy.optimize import linprog

factories = ['F1', 'F2', 'F3']
warehouses = ['W1', 'W2', 'W3']
cost_matrix = np.array([[16, 20, 12],
                        [14, 8, 18],
                        [26, 24, 16]])
cost = cost_matrix.flatten()

supply = [200, 160, 90]
demand = [180, 120, 150]

A_ub, b_ub = [], []

for i in range(3):
    row = np.zeros(9)
    row[i*3:(i+1)*3] = 1
    A_ub.append(row); b_ub.append(supply[i])

for j in range(3):
    row = np.zeros(9)
    for i in range(3):
        row[i*3 + j] = -1
    A_ub.append(row); b_ub.append(-demand[j])
```

```

results = linprog(cost, A_ub=np.array(A_ub), b_ub=np.array(b_ub), bounds=[(0, None)

alloc = results.x.reshape(3, 3)
df = pd.DataFrame(alloc.astype(int), index=factories, columns=warehouses)

print("Optimal shipment plan (units):")
print(df.to_string())
print(f"\nMinimum total cost: {results.fun:.0f} tk")

```

Optimal shipment plan (units):

	W1	W2	W3
F1	140	0	60
F2	40	120	0
F3	0	0	90

Minimum total cost: 5920 tk

A4Q7

A company has a team of four salesmen and there are four cities where the company wants to start its business. After considering the capabilities of salesmen and the nature of the cities, the company estimates that the profit per day in BDT. for each salesman in each city is as below.

		City			
		1	2	3	4
Salesman	A	16	10	14	11
	B	14	11	15	15
	C	15	15	13	12
	D	13	12	14	15

Find the assignment of salesmen to various cities which will yield maximum profit.

```

In [22]: import numpy as np
from scipy.optimize import linear_sum_assignment

```

```

profit = np.array([
    [16, 10, 14, 11],
    [14, 11, 15, 15],
    [15, 15, 13, 12],
    [13, 12, 14, 15]
])

cost = - profit

x,y = linear_sum_assignment(cost)

assignment=list(zip("ABCD", (y + 1)))

print(f"assignment: {assignment}")
print(f"profit: {profit[x, y].sum()}")

```

```

assignment: [('A', np.int64(1)), ('B', np.int64(3)), ('C', np.int64(2)), ('D', np.int64(4))]
profit: 61

```

A4Q8

Find the best strategy and the value of the following 3×3 game:

		Player A			
		I	II	III	
Player B	I	-1	-2	8	-2
	II	7	5	-1	-1
	III	6	0	12	0
		7	5	12	

```
In [23]: import nashpy as nash
import numpy as np

B = np.array([[-1, -2, 8], [7, 5, -1], [6, 0, 12]])
A = np.transpose(B)
game = nash.Game(A, -A)

for a, b in game.vertex_enumeration():
    print(a.round(4), b.round(4), game[a, b].round(4))

[0.5294 0.      0.4706] [ 0.4706 0.5294 -0.      ] [ 3.2353 -3.2353]
```

Assignment-05

A5Q1

1. Linear Regression with One Variable

A sample code with the output is given as follows. Please modify the code to read the data file (ex2data1.txt) and produce the output learned theta.

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([6.1101, 5.5277, 8.5186]) # Population (x)
y = np.array([17.592, 9.1302, 13.662]) # Profit (y)
m = len(y)

# Add intercept term
X_b = np.c_[np.ones(m), X]
theta = np.zeros(2)
alpha = 0.01
iterations = 1500

def compute_cost(X, y, theta):
    errors = X @ theta - y
    return (1 / (2 * m)) * np.dot(errors, errors)

def gradient_descent(X, y, theta, alpha, iterations):
    for _ in range(iterations):
        gradient = (1 / m) * (X.T @ (X @ theta - y))
        theta -= alpha * gradient
    return theta

theta = gradient_descent(X_b, y, theta, alpha, iterations)
print("Learned theta:", theta)
```

Learned theta: [4.02750103 1.37581788]
FPS N/A | GPU 0% 46°C | CPU 5% | LAT N/A

```
In [60]: import numpy as np

# Load data
data = np.loadtxt('ex2data1.txt', delimiter=',')
X = data[:, :2]
y = data[:, 2]
m = len(y)

# Normalize features
mu = X.mean(axis=0)
sigma = X.std(axis=0)
X = (X - mu) / sigma

# Add intercept term
X_b = np.c_[np.ones(m), X]
theta = np.zeros(3)
alpha = 0.01
iterations = 1500
```

```
# Gradient descent
for _ in range(iterations):
    gradient = (1 / m) * (X_b.T @ (X_b @ theta - y))
    theta -= alpha * gradient

print("Learned theta:", theta)
```

Learned theta: [0.59999983 0.28732941 0.25778557]

A5Q2

2. Multivariate Linear Regression

A sample code with the output is given as follows. Please modify the code to read the data file (ex2data1.txt) and produce the output learned theta.

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[2104, 5], [1600, 3], [2400, 4]])
y = np.array([399900, 329900, 369000])
m = len(y)

# Feature normalization
X_mean = X.mean(axis=0)
X_std = X.std(axis=0)
X_norm = (X - X_mean) / X_std

# Add intercept term
X_b = np.c_[np.ones(m), X_norm]
theta = np.zeros(X_b.shape[1])
alpha = 0.01
iterations = 400

def gradient_descent(X, y, theta, alpha, iterations):
    for _ in range(iterations):
        gradient = (1 / m) * (X.T @ (X @ theta - y))
        theta -= alpha * gradient
    return theta

theta = gradient_descent(X_b, y, theta, alpha, iterations)
print("Learned theta:", theta)
```

Learned theta: [359691.97735379 5161.17119473 24305.29407282]

```
In [61]: import numpy as np

# Load data
data = np.loadtxt('ex2data1.txt', delimiter=',')
X = data[:, :2]
y = data[:, 2]
m = len(y)

# Feature normalization
X_mean = X.mean(axis=0)
X_std = X.std(axis=0)
X_norm = (X - X_mean) / X_std

# Add intercept term
X_b = np.c_[np.ones(m), X_norm]
```

```

theta = np.zeros(X_b.shape[1])
alpha = 0.01
iterations = 400

# Gradient descent
def gradient_descent(X, y, theta, alpha, iterations):
    for _ in range(iterations):
        gradient = (1 / m) * (X.T @ (X @ theta - y))
        theta -= alpha * gradient
    return theta

theta = gradient_descent(X_b, y, theta, alpha, iterations)
print("Learned theta:", theta)

```

Learned theta: [0.58922967 0.28170515 0.25264327]

A5Q3

3. Logistic Regression for Binary Classification

Implement logistic regression to predict whether a student is admitted based on exam scores.

Dataset: Generate random data sets containing scores of two exams and admission result (0 or 1).

Tasks:

- Visualize data using a scatter plot.
- Implement the sigmoid function.
- Implement the cost function for logistic regression.
- Implement gradient descent and find optimal θ .
- Plot the decision boundary.
- Evaluate accuracy on the training set.

```

In [62]: import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
m = 100

X_pos = np.random.multivariate_normal([70, 70], [[50, 30], [30, 50]], size=m//2)
X_neg = np.random.multivariate_normal([30, 30], [[50, 30], [30, 50]], size=m//2)
X = np.vstack((X_pos, X_neg))
y = np.hstack((np.ones(m//2), np.zeros(m//2)))

X_with_intercept = np.hstack((np.ones((m, 1)), X))

plt.figure(figsize=(8,6))
plt.scatter(X_pos[:, 0], X_pos[:, 1], c='b', marker='o', label='Admitted')
plt.scatter(X_neg[:, 0], X_neg[:, 1], c='r', marker='x', label='Not admitted')
plt.xlabel('Exam 1 score')
plt.ylabel('Exam 2 score')
plt.legend()
plt.grid()
plt.title('Exam Scores vs Admission')
plt.show()

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def compute_cost(theta, X, y):
    m = len(y)
    h = sigmoid(X.dot(theta))

```

```

    cost = -(1/m) * (y.dot(np.log(h)) + (1 - y).dot(np.log(1 - h)))
    grad = (1/m) * X.T.dot(h - y)
    return cost, grad

alpha = 0.01
iterations = 10000
theta = np.zeros(X_with_intercept.shape[1])

cost_history = []

for i in range(iterations):
    cost, grad = compute_cost(theta, X_with_intercept, y)
    theta -= alpha * grad
    cost_history.append(cost)

print(f'Final cost: {cost_history[-1]:.4f}')
print(f'Optimal theta: {theta}')

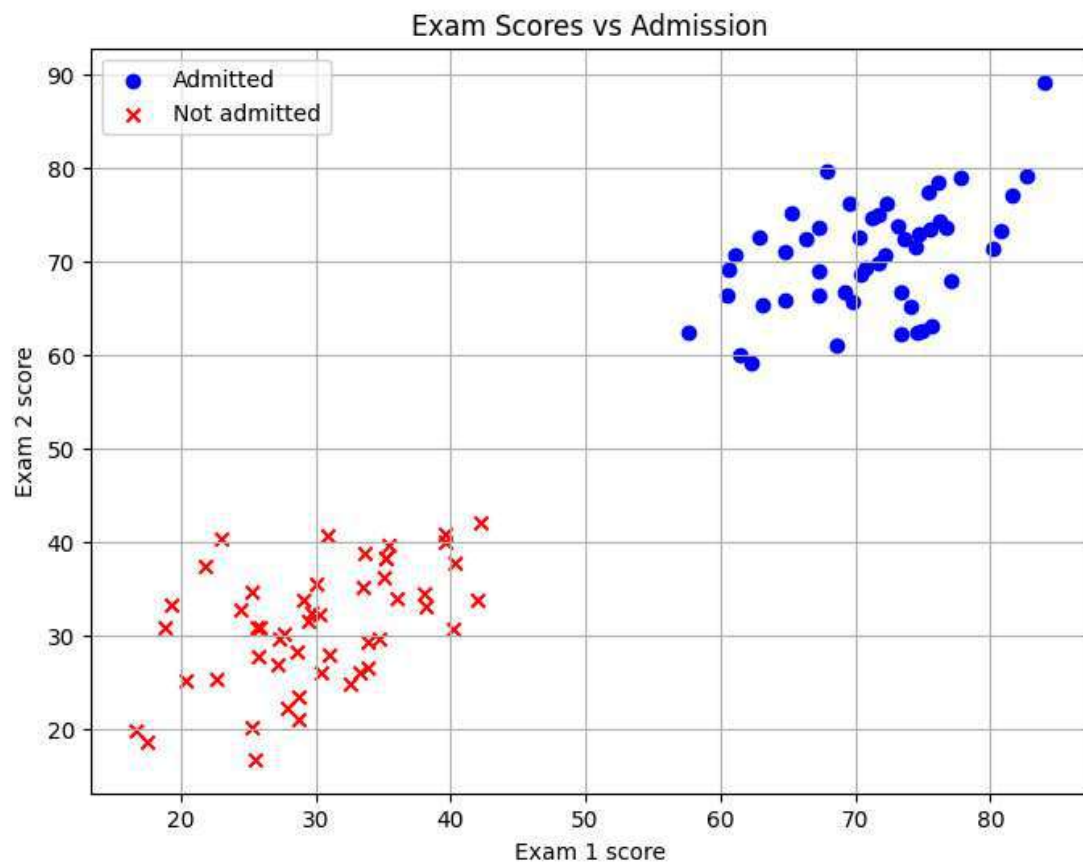
plt.figure(figsize=(8,5))
plt.plot(range(iterations), cost_history, 'b-')
plt.xlabel('Iteration')
plt.ylabel('Cost J(θ)')
plt.title('Cost Function Convergence')
plt.grid()
plt.show()

plt.figure(figsize=(8,6))
plt.scatter(X_pos[:, 0], X_pos[:, 1], c='b', marker='o', label='Admitted')
plt.scatter(X_neg[:, 0], X_neg[:, 1], c='r', marker='x', label='Not admitted')

x_vals = np.array([X[:,0].min()-5, X[:,0].max()+5])
y_vals = -(theta[0] + theta[1] * x_vals) / theta[2]
plt.plot(x_vals, y_vals, 'g-', label='Decision Boundary')
plt.grid()
plt.xlabel('Exam 1 score')
plt.ylabel('Exam 2 score')
plt.legend()
plt.title('Logistic Regression Decision Boundary')
plt.show()

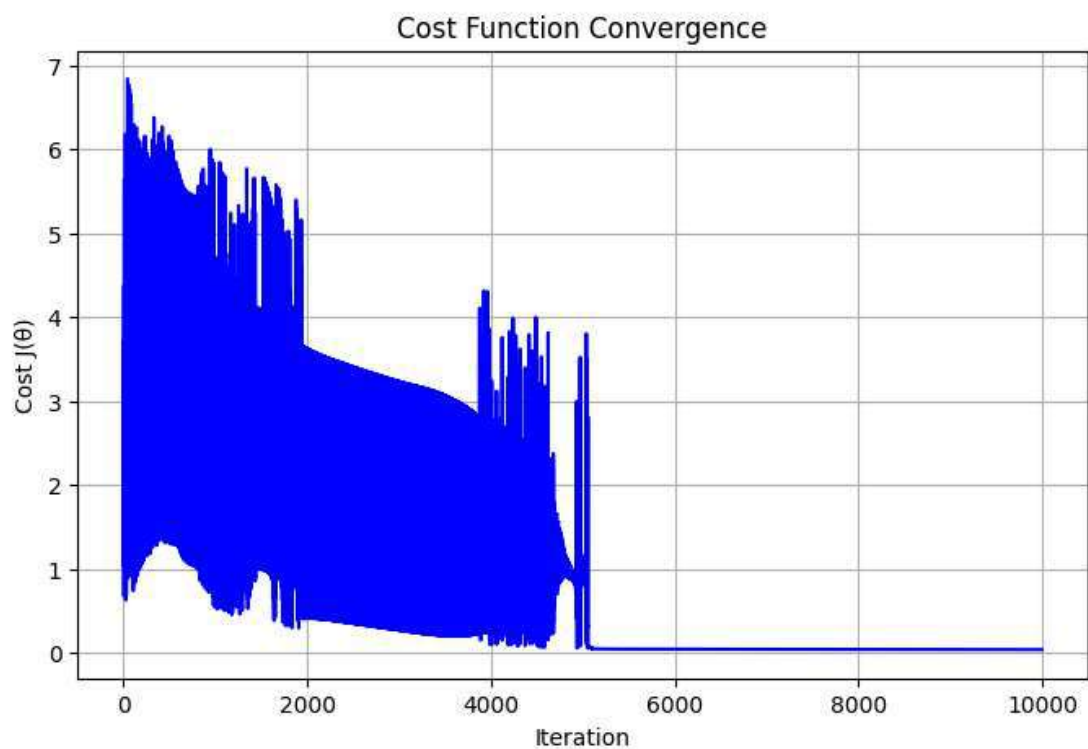
preds = sigmoid(X_with_intercept.dot(theta)) >= 0.5
accuracy = np.mean(preds == y) * 100
print(f'Training Accuracy: {accuracy:.2f}%')

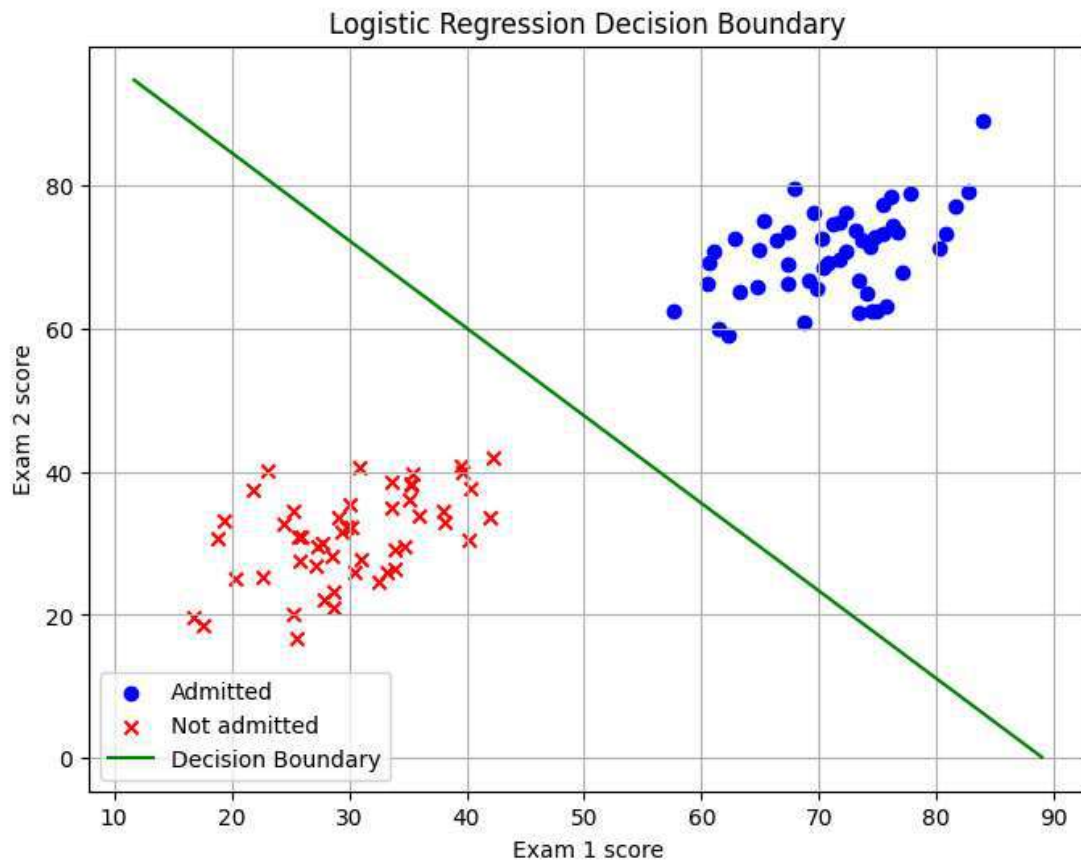
```



Final cost: 0.0380

Optimal theta: [-9.13392566 0.10256013 0.08370935]





Training Accuracy: 99.53%

A5Q4

4. Regularization in Logistic Regression

Apply regularization to logistic regression for non-linearly separable data.

Dataset: Generate microchip test results (2 test scores) with pass/fail labels.

Tasks:

- Map features into polynomial terms (e.g., up to 6th degree).
- Implement regularized cost function and gradient.
- Choose different values of regularization parameter λ (e.g., 0, 1, 100).
- Plot the decision boundaries.
- Discuss the effect of underfitting and overfitting due to different λ values.

```
In [63]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

np.random.seed(0)
m = 100
X1 = np.random.normal(0, 1, (m//2, 2)) + np.array([2, 2])
X2 = np.random.normal(0, 1, (m//2, 2)) + np.array([-2, -2])
X = np.vstack([X1, X2])
y = np.hstack([np.ones(m//2), np.zeros(m//2)])

def map_features(X1, X2, degree=6):
    out = [np.ones(X1.shape[0])]
    for i in range(1, degree+1):
        for j in range(i+1):
            out.append((X1**(i-j)) * (X2**j))
```

```

return np.vstack(out).T

X_poly = map_features(X[:,0], X[:,1], degree=6)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def cost_function(theta, X, y, lambda):
    m = y.size
    h = sigmoid(X.dot(theta))
    J = - (1/m) * (y.dot(np.log(h)) + (1-y).dot(np.log(1-h)))
    reg = (lambda/(2*m)) * np.sum(theta[1:]**2)
    return J + reg

def gradient(theta, X, y, lambda):
    m = y.size
    h = sigmoid(X.dot(theta))
    grad = (1/m) * X.T.dot(h - y)
    grad[1:] += (lambda/m) * theta[1:]
    return grad

lambdas = [0, 1, 100]
theta_init = np.zeros(X_poly.shape[1])

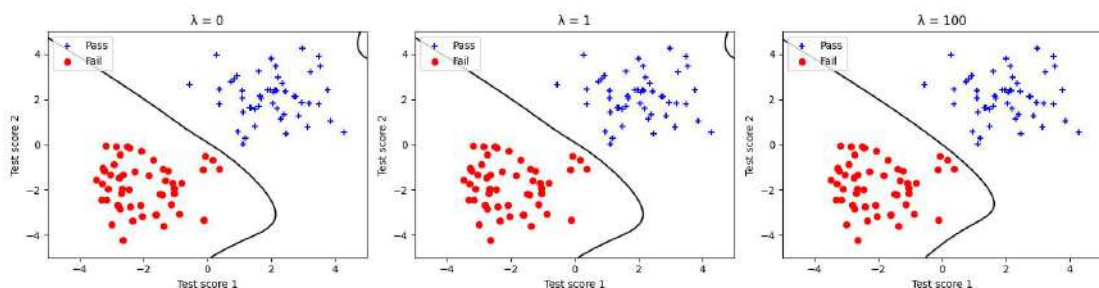
xx, yy = np.meshgrid(np.linspace(-5, 5, 200), np.linspace(-5, 5, 200))
grid = map_features(xx.ravel(), yy.ravel(), degree=6)

plt.figure(figsize=(15, 4))
for i, lambda in enumerate(lambdas):
    res = minimize(fun=cost_function,
                  x0=theta_init,
                  args=(X_poly, y, lambda),
                  method='TNC',
                  jac=gradient,
                  options={'maxiter': 400})
    theta_opt = res.x

    plt.subplot(1, 3, i+1)
    plt.scatter(X[y==1,0], X[y==1,1], c='b', marker='+', label='Pass')
    plt.scatter(X[y==0,0], X[y==0,1], c='r', marker='o', label='Fail')
    Z = grid.dot(theta_opt).reshape(xx.shape)
    plt.contour(xx, yy, Z, levels=[0], colors='k')
    plt.title(f' $\lambda = \{lambda\}$ ')
    plt.xlabel('Test score 1')
    plt.ylabel('Test score 2')
    plt.legend()

plt.tight_layout()
plt.show()

```



A5Q5

5. Polynomial Regression and Learning Curves

Implement polynomial regression to capture non-linear trends in data.

Dataset: Generate or use a dataset where the relationship between input and output is non-linear (e.g., housing prices vs. size, $y = x \sin(x)$, etc.).

Tasks:

- Map features to polynomial terms (e.g., degree 5 or higher).
- Implement regularized linear regression.
- Plot training and cross-validation error (learning curves) for different training set sizes.
- Vary regularization parameter λ and observe the effect on bias-variance.
- Discuss bias-variance trade-off using plots and numerical outputs.

Expected Output:

- Learned parameters (θ).
- Learning curves (training error vs. cross-validation error).
- Plot showing polynomial regression fit.
- Brief explanation on how λ and training set size affect model performance.

```
In [64]: import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

def generate_data(n_samples=100, noise=0.1, random_state=0):
    np.random.seed(random_state)
    X = np.linspace(0, 10, n_samples).reshape(-1,1)
    y = X.ravel()*np.sin(X.ravel()) + np.random.normal(scale=noise, size=n_samples)
    return X, y

def compute_learning_curve(X, y, degree, lambd, train_sizes, cv_size=0.2, random_st
results = []
for size in train_sizes:
    X_train_full, X_cv, y_train_full, y_cv = train_test_split(
        X, y, test_size=cv_size, random_state=random_state
    )
    X_train, y_train = X_train_full[:size], y_train_full[:size]

    poly = PolynomialFeatures(degree=degree, include_bias=False)
    X_train_poly = poly.fit_transform(X_train)
    X_cv_poly = poly.transform(X_cv)

    model = Ridge(alpha=lambd)
    model.fit(X_train_poly, y_train)

    train_error = mean_squared_error(y_train, model.predict(X_train_poly))
    cv_error = mean_squared_error(y_cv, model.predict(X_cv_poly))

    results.append({
        'lambda': lambd,
        'train_size': size,
        'train_error': train_error,
        'cv_error': cv_error
    })
return results

degree = 5
lambdas = [0, 0.01, 0.1, 1, 10]
train_sizes = [10, 20, 50, 80]

X, y = generate_data(n_samples=100, noise=0.2)
all_results = []
```

```

for l in lambdas:
    all_results.extend(compute_learning_curve(X, y, degree, l, train_sizes))

df = pd.DataFrame(all_results)

print(df)

```

	lambda	train_size	train_error	cv_error
0	0.00	10	0.378316	5.002636
1	0.00	20	0.953134	2.459111
2	0.00	50	1.213178	1.723358
3	0.00	80	1.238310	1.604693
4	0.01	10	0.383088	5.754027
5	0.01	20	0.970946	2.378130
6	0.01	50	1.213370	1.725680
7	0.01	80	1.238454	1.605528
8	0.10	10	0.397676	6.592430
9	0.10	20	1.273622	2.642879
10	0.10	50	1.225470	1.750418
11	0.10	80	1.248504	1.619259
12	1.00	10	0.405800	6.538886
13	1.00	20	1.655658	3.255318
14	1.00	50	1.350402	1.902635
15	1.00	80	1.390739	1.762784
16	10.00	10	0.421008	6.234729
17	10.00	20	1.743165	3.536477
18	10.00	50	1.487265	2.194230
19	10.00	80	1.553259	2.019744

A5Q6

6. One-vs-All Logistic Regression for Multi-class Classification

Extend logistic regression to handle multi-class classification using the One-vs-All (OvA) approach.

Dataset: Use the *digits_data* dataset (from the digit classification exercise) or simulate your own data with 3+ classes (e.g., flower species, handwritten digits, etc.).

Tasks:

- Implement the one-vs-all classification strategy.
- Train one logistic regression classifier per class.
- Predict the class label for new inputs using all trained classifiers.
- Compute and report the accuracy on the training set.
- Visualize data samples (e.g., digits or class distributions).
- Optional: Use scikit-learn's LogisticRegression for comparison.

Expected Output:

- Training accuracy.
- Learned parameters for each class.
- Plot(s) if using a visual dataset.

```

In [65]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

digits = load_digits()
X, y = digits.data, digits.target
n_classes = len(np.unique(y))

X_train, X_test, y_train, y_test = train_test_split(

```

```

X, y, test_size=0.3, random_state=42, stratify=y
)

classifiers = []
for cls in range(n_classes):
    y_bin = (y_train == cls).astype(int)
    clf = LogisticRegression(
        penalty='l2', C=1.0, solver='lbfgs', max_iter=200
    )
    clf.fit(X_train, y_bin)
    classifiers.append(clf)

probs = np.column_stack([clf.predict_proba(X_train)[:,1] for clf in classifiers])
y_pred_train = np.argmax(probs, axis=1)

train_acc = accuracy_score(y_train, y_pred_train)
print(f"Training accuracy (OvA logistic regression): {train_acc*100:.2f}%")

sk_clf = LogisticRegression(
    multi_class='ovr', penalty='l2', C=1.0, solver='lbfgs', max_iter=200
)
sk_clf.fit(X_train, y_train)
y_pred_sk = sk_clf.predict(X_train)
sk_acc = accuracy_score(y_train, y_pred_sk)
print(f"Training accuracy (sklearn's LogisticRegression w/ OvA): {sk_acc*100:.2f}%")

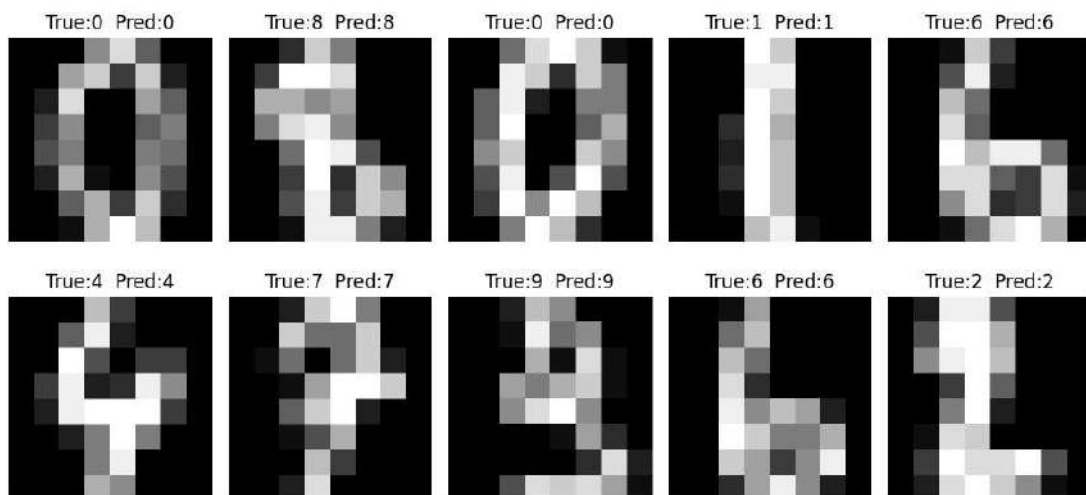
fig, axes = plt.subplots(2, 5, figsize=(10, 5))
indices = np.random.choice(len(X_train), size=10, replace=False)
for ax, idx in zip(axes.ravel(), indices):
    ax.imshow(X_train[idx].reshape(8, 8), cmap='gray')
    ax.set_title(f"True:{y_train[idx]} Pred:{y_pred_train[idx]}")
    ax.axis('off')

plt.tight_layout()
plt.show()

```

Training accuracy (OvA logistic regression): 99.84%

Training accuracy (sklearn's LogisticRegression w/ OvA): 99.84%



In []:

Assignment-06

A6Q1

Which of the two investments described below is preferable? Assume the first payment will take place exactly one year from now and further payments are spaced one year apart. Assume the continually compounded annual interest rate is 4.33%.

Year	1	2	3	4	5
Investment A	225	215	250	225	205
Investment B	220	225	250	250	210

```
In [1]: import numpy as np

r = 0.0433
years = np.arange(1, 6)

# Cash flows
A = np.array([225, 215, 250, 225, 205])
B = np.array([220, 225, 250, 250, 210])

PV_A=np.sum(A*np.exp(-r*years))
PV_B=np.sum(B*np.exp(-r*years))

print(f"Present Value of Investment A: {PV_A:.2f}")
print(f"Present Value of Investment B: {PV_B:.2f}")

print("Investment A is better." if PV_A > PV_B else "Investment B is better.")
```

Present Value of Investment A: 986.49
 Present Value of Investment B: 1015.92
 Investment B is better.

A6Q2

A 5-year bond with a yield of 11% (continuously compounded) pays an 8% coupon at the end of each year.

- What is the bond's price?
- What is the bond's duration?
- Use the duration to calculate the effect on the bond's price of a 0.2% decrease in its yield.
- Recalculate the bond's price on the basis of a 10.8% per annum yield and verify that the result is in agreement with your answer to (c)

```
In [ ]: import numpy as np

FV, c, r1, r2, n = 100, 0.08, 0.11, 0.108, 5
t = np.arange(1, n + 1)
cf = np.full(n, c * FV); cf[-1] += FV

P1 = np.sum(cf * np.exp(-r1 * t))
D = np.sum(t * cf * np.exp(-r1 * t)) / P1
P1_approx = P1 + D * P1 * 0.002
```

```
P2 = np.sum(cf * np.exp(-r2 * t))

print(P1, D, P1_approx, P2)
```

86.80108422328516 4.255974563427866 87.53993063634968 87.54339139534211

A6Q3

Suppose an investor will receive payments at the end of the next 6 years in the amounts shown in the table below.

Year	1	2	3	4	5	6
Payment	460	235	640	370	330	250

If the interest rate is 4.5% compounded quarterly, what is the present value of the investments, given that the first payment will arrive one year from now?

```
In [3]: import numpy as np

P = np.array([460, 235, 640, 370, 330, 250])
i = 0.045
m = 4

PV=np.sum(P/(1+i/m)**(m*np.arange(1,7)))

print(f"Present Value: ${PV:.2f}")
```

Present Value: \$1978.69

A6Q4

A 1-year long forward contract on a non-dividend-paying stock is entered into when the stock price is \$40 and the risk-free rate of interest is 10% per annum with continuous compounding.

- What are the forward price and the initial value of the forward contract?
- Six months later, the price of the stock is \$45 and the risk-free interest rate is still 10%. What are the forward price and the value of the forward contract?

```
In [4]: import numpy as np

S0, r, T = 40, 0.10, 1
F0 = S0 * np.exp(r * T)
print(f"F0={F0:.2f}, V0=0")

St, t = 45, 0.5
Vt = St - F0 * np.exp(-r * (T - t))      # ST - Present value of the prev Future val
Ft_new = St * np.exp(r * (T - t))
print(f"Ft={Ft_new:.2f}, Vt={Vt:.2f}")
```

F0=44.21, V0=0
Ft=47.31, Vt=2.95

A6Q5

Write a suitable Python code to sketch the Payoff and Profit diagrams of the following options:

- (a) Long call
- (b) Short call
- (c) Long put
- (d) Short put

Consider $K = \$150$ & premium = \$5 for all cases.

```
In [5]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

# Parameters
K = 150
premium = 5
S = np.linspace(100, 200, 500)

# Payoff and Profit functions
def calc_payoff_profit(option_type):
    if option_type == 'long_call':
        payoff = np.maximum(S - K, 0)
        profit = payoff - premium
    elif option_type == 'short_call':
        payoff = -np.maximum(S - K, 0)
        profit = payoff + premium
    elif option_type == 'long_put':
        payoff = np.maximum(K - S, 0)
        profit = payoff - premium
    elif option_type == 'short_put':
        payoff = -np.maximum(K - S, 0)
        profit = payoff + premium
    return payoff, profit

# Segment plotting
def plot_colored_line(ax, x, y, payoff, title):
    for i in range(len(x)-1):
        if payoff[i] == 0:
            color = 'red' # Not executed
        elif y[i] < 0:
            color = 'yellow' # Loss
        else:
            color = 'green' # Profit
        ax.plot(x[i:i+2], y[i:i+2], color=color, linewidth=2)

    ax.plot(x, payoff, 'k--', linewidth=1.5, label='Payoff')
    ax.axhline(0, color='black', linewidth=0.7)
    ax.axvline(K, color='gray', linestyle='--', linewidth=0.8)
    ax.set_title(title)
    ax.set_xlabel('Stock Price at Expiry ($)')
    ax.set_ylabel('Payoff / Profit ($)')

# Custom Legend
legend_elements = [
    Line2D([0], [0], color='green', lw=2, label='Profit'),
    Line2D([0], [0], color='yellow', lw=2, label='Loss'),
    Line2D([0], [0], color='red', lw=2, label='Not Executed'),
    Line2D([0], [0], color='black', linestyle='--', lw=1.5, label='Payoff')
]
ax.legend(handles=legend_elements, loc='upper left')

# Plot all
fig, axs = plt.subplots(2, 2, figsize=(12, 8))
titles = ['Long Call', 'Short Call', 'Long Put', 'Short Put']
```

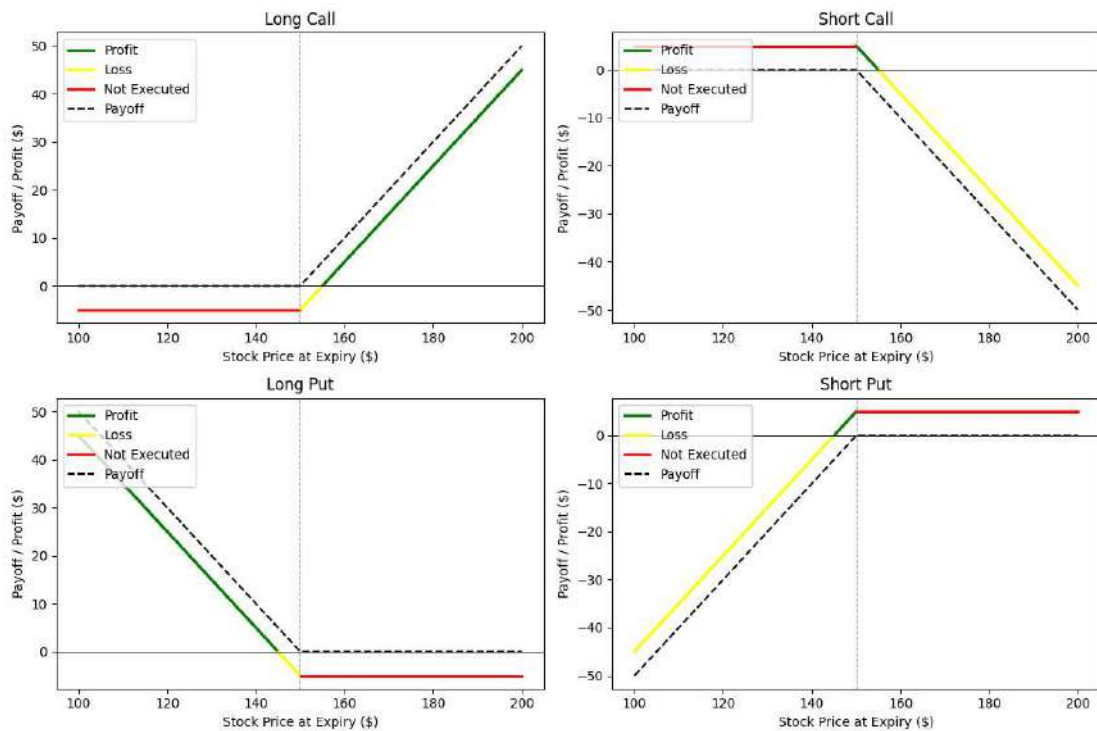
```

types = ['long_call', 'short_call', 'long_put', 'short_put']

for ax, t, title in zip(axes.ravel(), types, titles):
    payoff, profit = calc_payoff_profit(t)
    plot_colored_line(ax, S, profit, payoff, title)

plt.tight_layout()
plt.show()

```



A6Q6

A trader buys a call option with a strike price of \$45 and a put option with a strike price of \$40. Both options have the same maturity. The call costs \$3 and the put costs \$4. Draw a suitable diagram showing the variation of the trader's profit with the asset price.

```

In [8]: import numpy as np
import matplotlib.pyplot as plt

Kc=45;Kp=40;C=3;P=4

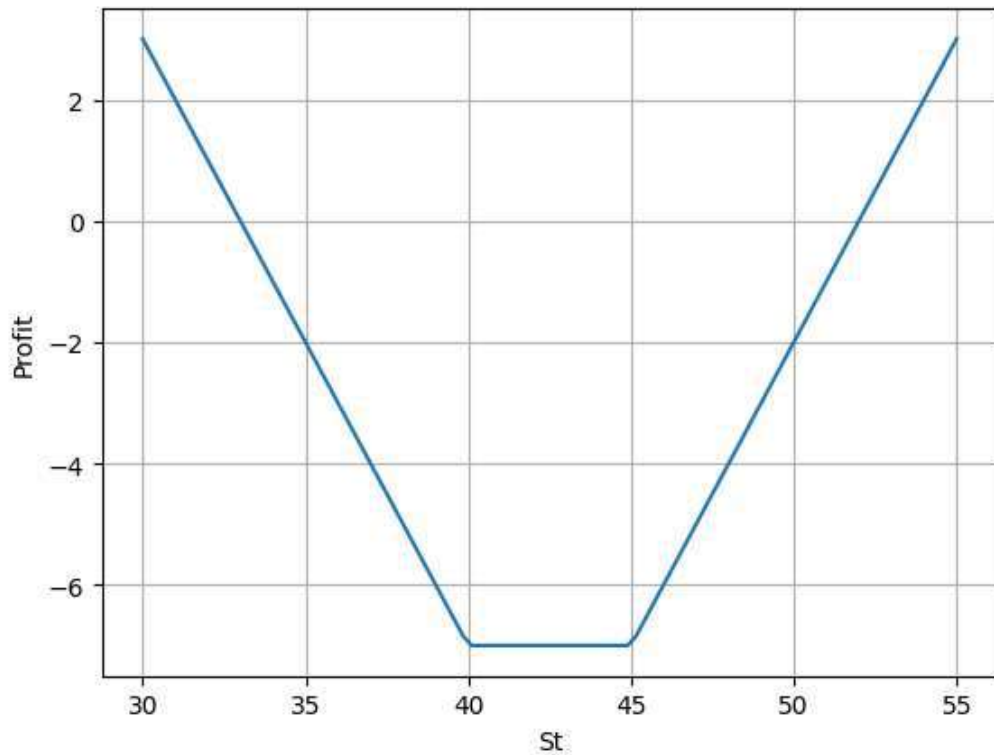
St=np.linspace(30,55,100)

Profit= np.maximum(St-Kc,0)-C+np.maximum(Kp-St,0)-P

# print(Profit)

plt.plot(St,Profit)
plt.xlabel("St")
plt.ylabel("Profit")
plt.grid()
plt.show()

```



A6Q7

The price of a non-dividend-paying stock is \$19 and the price of a 4-month European call option on the stock with a strike price of \$20 is \$1. The risk-free rate is 3% per annum. What is the price of a 4-month European put option with a strike price of \$20?

```
In [9]: import numpy as np

St=19; K=20; C=1; r=0.03; t=4/12

P=C-St+K*np.exp(-r*t)

print(f"Put option price: {P}")
```

Put option price: 1.800996674983363

A6Q8

Suppose that the price of a non-dividend-paying stock is \$32, its volatility is 30%, and the risk-free rate for all maturities is 5% per annum. Use Python to calculate the cost of setting up the following positions:

- A bull spread using European call options with strike prices of \$25 and \$30 and a maturity of 6 months.
- A bear spread using European put options with strike prices of \$25 and \$30 and a maturity of 6 months.
- A butterfly spread using European call options with strike prices of \$25, \$30, and \$35 and a maturity of 1 year.
- A butterfly spread using European put options with strike prices of \$25, \$30, and \$35 and a maturity of 1 year.
- A straddle using options with a strike price of \$30 and a 6-month maturity.
- A strangle using options with strike prices of \$25 and \$35 and a 6-month maturity.

In each case provide a table showing the relationship between profit and final stock price. Ignore the impact of discounting.

```
In [10]: import mibian

S = 32
r = 5           # not 0.05
sigma = 30      # not 0.3
T1 = 0.5 * 365  # lol, days e input deya lage not year
T2 = 1 * 365

def CallPrice(S,K,r,T,Vol):
    return mibian.BS([S,K,r,T],volatility=Vol).callPrice

def PutPrice(S,K,r,T,Vol):
    return mibian.BS([S,K,r,T],volatility=Vol).putPrice

C25_6=CallPrice(S,25,r,T1,sigma)
C30_6=CallPrice(S,30,r,T1,sigma)
C35_6=CallPrice(S,35,r,T1,sigma)

P25_6=PutPrice(S,25,r,T1,sigma)
P30_6=PutPrice(S,30,r,T1,sigma)
P35_6=PutPrice(S,35,r,T1,sigma)

C25_12=CallPrice(S,25,r,T2,sigma)
C30_12=CallPrice(S,30,r,T2,sigma)
C35_12=CallPrice(S,35,r,T2,sigma)

P25_12=PutPrice(S,25,r,T2,sigma)
P30_12=PutPrice(S,30,r,T2,sigma)
P35_12=PutPrice(S,35,r,T2,sigma)

Bull_Cost=abs(C25_6-C30_6)
Bear_Cost=abs(P25_6-P30_6)
Butterfly1_Cost=abs(C25_12+C35_12-2*C30_12)
Butterfly2_Cost=abs(P25_12+P35_12-2*P30_12)
Long_Straddle_Cost=abs(C30_6+P30_6)          ##### same same but diff
Long_Strangle_Cost=abs(P25_6+C35_6)          ##### aste rakho; jore dako

print(f"Bull Cost (call): {Bull_Cost}")
print(f"Bear Cost (put): {Bear_Cost}")
```

```
print(f"Butterfly Cost (call): {Butterfly1_Cost}")
print(f"Butterfly Cost (put): {Butterfly2_Cost}")
print(f"Long Straddle Cost: {Long_Straddle_Cost}")
print(f"Long Strangle Cost: {Long_Strangle_Cost}")
```

```
Bull Cost (call): 3.7129815835540576
Bear Cost (put): 1.16356797658761
Butterfly Cost (call): 1.0009096879706725
Butterfly Cost (put): 1.0009096879706725
Long Straddle Cost: 5.62672130157711
Long Strangle Cost: 2.1328040823930485
```

A6Q9

Consider an option on a non-dividend-paying stock when the stock price is \$30, the exercise price is \$29, the risk-free interest rate is 5%, the volatility is 25% per annum, and the time to maturity is 4 months.

- What is the price of the option if it is a European call?
- What is the price of the option if it is an American call?
- What is the price of the option if it is a European put?
- Verify that put-call parity holds.

```
In [13]: import mibian
import numpy as np

S=30; K=29; r=5; sigma=25; T=(4/12)*365

bs=mibian.BS([S,K,r,T],volatility=sigma)

EU_Call=bs.callPrice
US_Call=EU_Call      # same for non dividend paying stock
EU_Put=bs.putPrice

PCP_Holds=abs(EU_Call-EU_Put - S + K*np.exp((-r/100)*(T/365))) < 10**(-4)

print(f"EU_Call: {EU_Call} ;US_Call: {US_Call}; EU_Put: {EU_Put}")
print(f"Put Call er CHETONA Holds: {PCP_Holds}")
```

```
EU_Call: 2.5251469666999995 ;US_Call: 2.5251469666999995; EU_Put: 1.045819127526908
Put Call er CHETONA Holds: True
```

A6Q10

A futures price is currently 60 and its volatility is 30%. The risk-free interest rate is 8% per annum. Use a two-step binomial tree to calculate the value of a six-month European call option on the futures with a strike price of 60. If the call were American, would it ever be worth exercising it early?

```
In [14]: import numpy as np

S0=60; K=60; sigma=0.3; r=0.08; dt=3/12

u=np.exp(sigma*np.sqrt(dt))
d=1/u

fuu=np.maximum(S0*u*u - K,0)
fud=np.maximum(S0*u*d - K,0)
fdd=np.maximum(S0*d*d - K,0)
```

```

p=(np.exp(r*dt)-d)/(u-d)
f=np.exp(-2*r*dt)*((p**2)*fuu + 2*p*(1-p)*fud + ((1-p)**2)*fdd )

print(f"Value of EU Call: {f}")
print(f"Value of US Call == Value of EU Call: {f}; Cause, non dividend paying stock

```

Value of EU Call: 5.657964761441472

Value of US Call == Value of EU Call: 5.657964761441472; Cause, non dividend paying stock

A6Q11

A 2-month American put option on a stock index has an exercise price of 480. The current level of the index is 484, the risk-free interest rate is 10% per annum, the dividend yield on the index is 3% per annum, and the volatility of the index is 25% per annum. Divide the life of the option into four half-month periods and use the tree approach to estimate the value of the option.

In [16]: `import numpy as np`

```

S0=60; K=60; sigma=0.3; r=0.08; dt=3/12

u=np.exp(sigma*np.sqrt(dt))
d=1/u

fuu=np.maximum(S0*u - K,0)
fud=np.maximum(S0*u*d - K,0)
fdd=np.maximum(S0*d*d - K,0)

p=(np.exp(r*dt)-d)/(u-d)
f=np.exp(-2*r*dt)*((p**2)*fuu + 2*p*(1-p)*fud + ((1-p)**2)*fdd )

print(f"Value of EU Call: {f}")
print(f"Value of US Call == Value of EU Call: {f}; Cause, non dividend paying stock

```

Value of EU Call: 5.657964761441472

Value of US Call == Value of EU Call: 5.657964761441472; Cause, non dividend paying stock

A7Q1_Steady

Report date

Oct 21, 2025, 12:21:13 AM

Contents

1. [Global Definitions](#)
 - 1.1. [Parameters](#)
2. [Component 1](#)
 - 2.1. [Definitions](#)
 - 2.2. [Geometry 1](#)
 - 2.3. [Heat Transfer in Solids](#)
 - 2.4. [Mesh 1](#)
3. [Study 1](#)
 - 3.1. [Stationary](#)
4. [Results](#)
 - 4.1. [Datasets](#)
 - 4.2. [Derived Values](#)
 - 4.3. [Tables](#)
 - 4.4. [Plot Groups](#)

1. Global Definitions

Date	Sep 21, 2025, 7:18:03 AM
------	--------------------------

Global settings

Name	A7Q1 Steady.mph
Path	D:\4th Year\450\COMSOL\6.0\A7Q1\A7Q1_Steady.mph
Version	COMSOL Multiphysics 6.0 (Build: 318)

Used products

COMSOL Multiphysics

Computer information

CPU	Intel64 Family 6 Model 141 Stepping 1, 6 cores
Operating system	Windows 10

1.1. Parameters

Parameters 1

Name	Expression	Value	Description
k	52 [W/(m*K)]	52 W/(m·K)	
rho	7850 [kg/m^3]	7850 kg/m ³	
Cp	460 [J/(kg*K)]	460 J/(kg·K)	
Q	0 [J]	0 J	
J	5e5 [W/m^2]	5E5 W/m ²	

2. Component 1

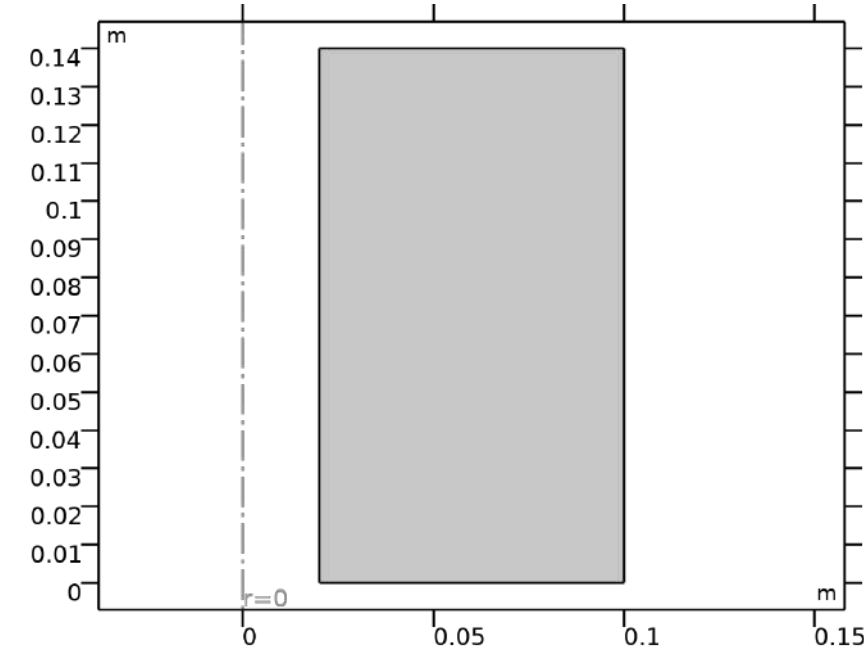
2.1. Definitions

2.1.1. Coordinate Systems

Boundary System 1

Coordinate system type	Boundary system	
Tag	sys1	
Coordinate names		
First	Second	Third
t1	to	n

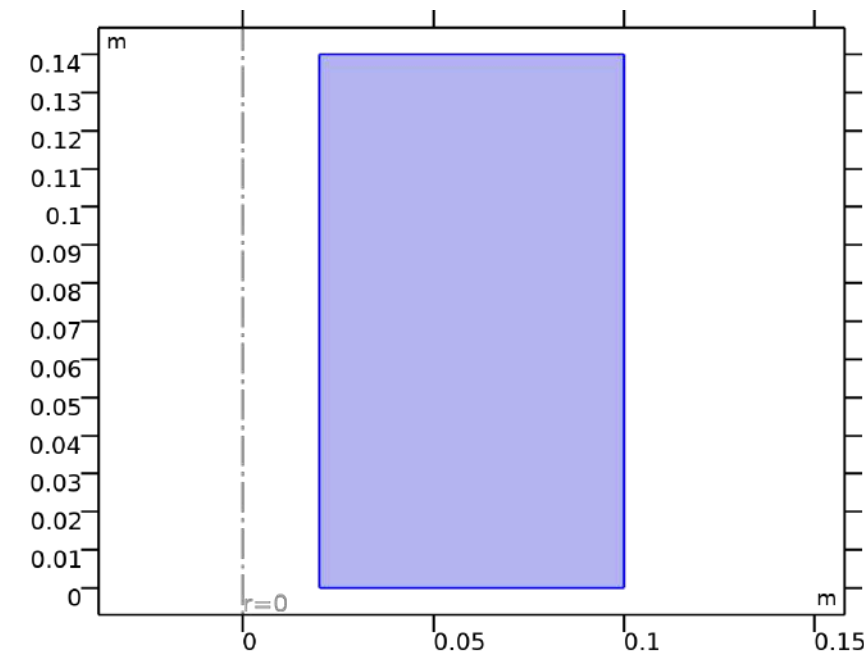
2.2. Geometry 1



Geometry 1

Units	
Length unit	m
Angular unit	deg

2.3. Heat Transfer in Solids



Heat Transfer in Solids

Equations

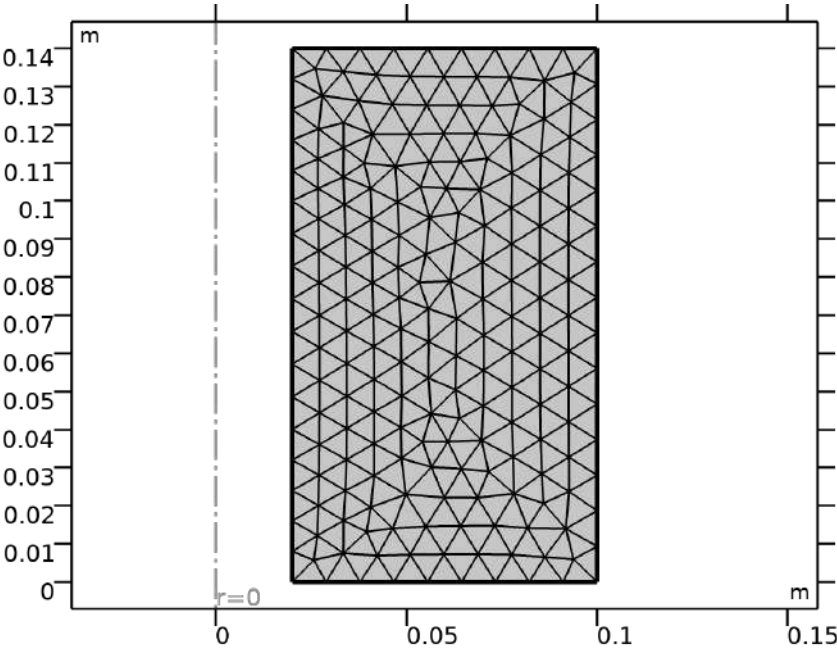
$$\rho C_p \mathbf{u} \cdot \nabla T + \nabla \cdot \mathbf{q} = Q + Q_{\text{ted}}$$

$$\mathbf{q} = -k \nabla T$$

Features

Name	Level
Solid 1	Domain
Initial Values 1	Domain
Axial Symmetry 1	Boundary
Thermal Insulation 1	Boundary
Temperature 1	Boundary
Heat Flux 1	Boundary

2.4. Mesh 1



Mesh 1

3. Study 1

Computation
information

Computation time	1 s
------------------	-----

3.1. Stationary

Study settings

Description	Value
Include geometric nonlinearity	Off

Physics and variables selection

Physics interface	Discretization
Heat Transfer in Solids (ht)	physics

Mesh selection

Geometry	Mesh
Geometry 1 (geom1)	mesh1

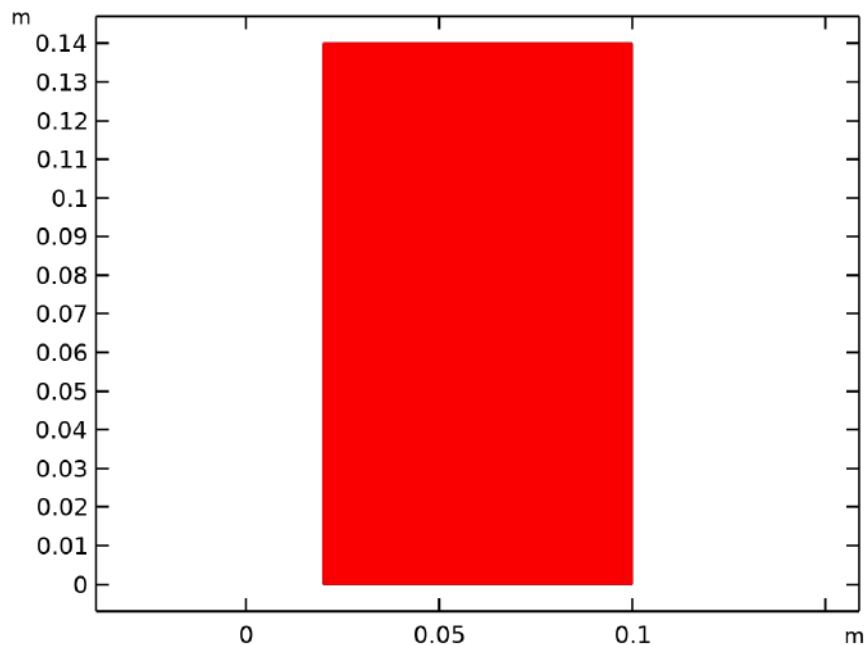
4. Results

4.1. Datasets

4.1.1. Study 1/Solution 1

Solution

Description	Value
Solution	Solution 1
Component	Component 1 (comp1)



Dataset: Study 1/Solution 1

4.1.2. Revolution 2D

Data

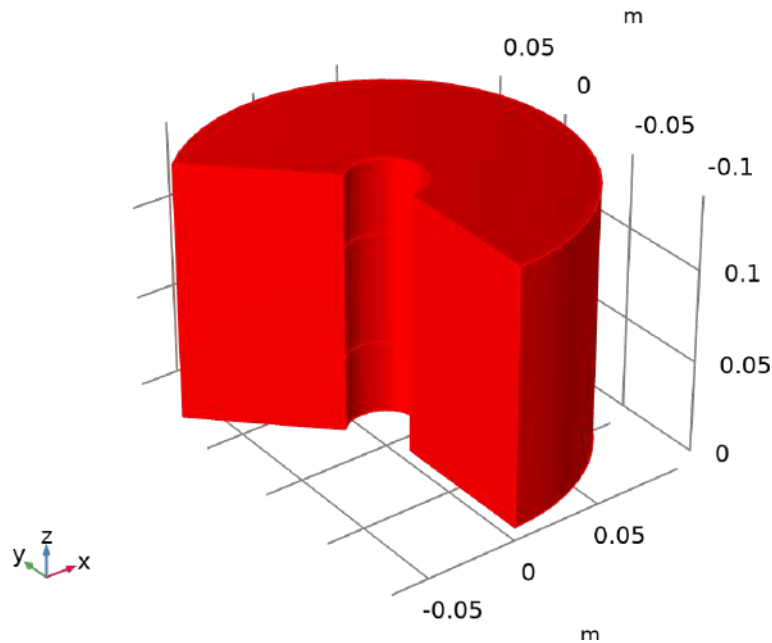
Description	Value
Dataset	Study 1/Solution 1

Axis data

Description	Value
Axis entry method	Two points
Points	{{0, 0}, {0, 1}}

Revolution layers

Description	Value
Start angle	-90
Revolution angle	225

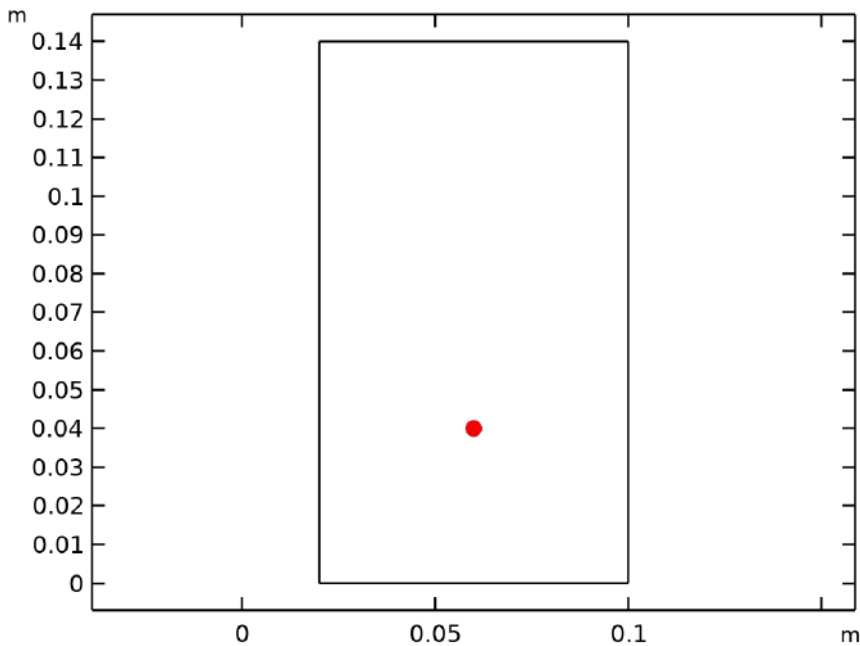


Dataset: Revolution 2D

4.1.3. Cut Point 2D A

Data	
Description	Value
Dataset	Study 1/Solution 1

Point data	
Description	Value
Entry method	Coordinates
R	0.06
Z	0.04



Dataset: Cut Point 2D A

4.1.4. Cut Point 2D B

Data

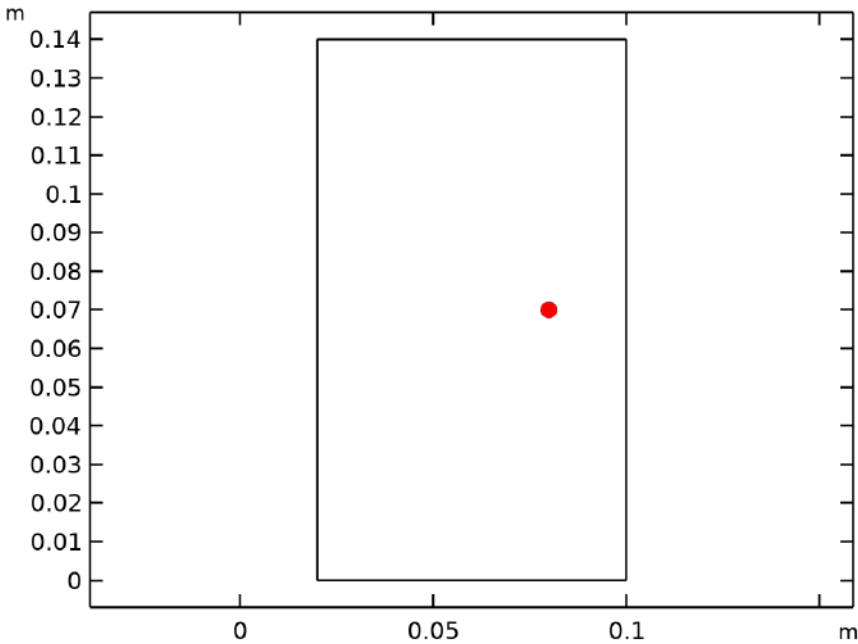
Description	Value
Dataset	Study 1/Solution 1

Point data

Description	Value
Entry method	Coordinates
R	0.08
Z	0.07

Advanced

Description	Value
Point number variable	cpt1n



Dataset: Cut Point 2D B

4.1.5. Cut Point 2D C

Data

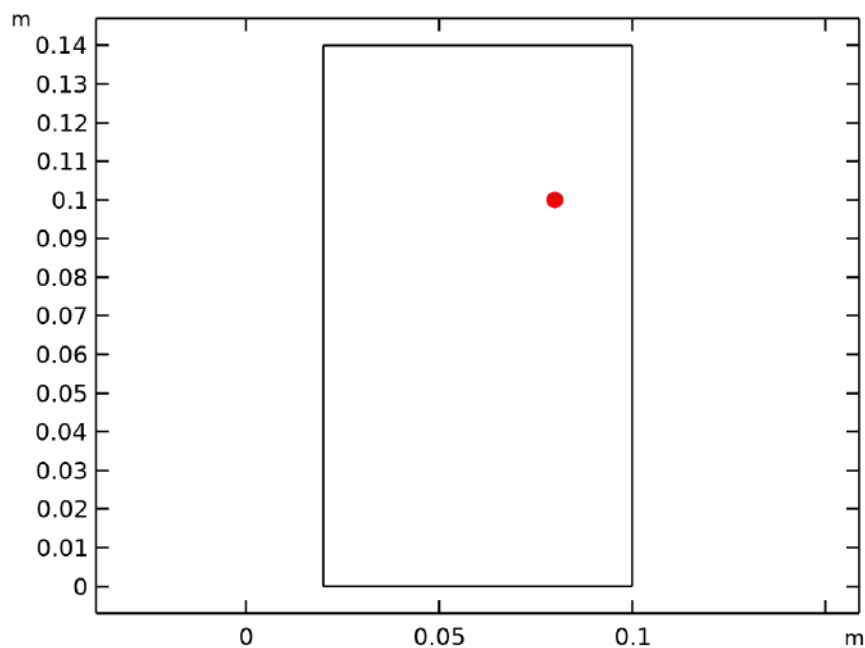
Description	Value
Dataset	Study 1/Solution 1

Point data

Description	Value
Entry method	Coordinates
R	0.08
Z	0.1

Advanced

Description	Value
Point number variable	cpt1n



Dataset: Cut Point 2D C

4.1.6. Cut Line 2D Z=0.05

Data

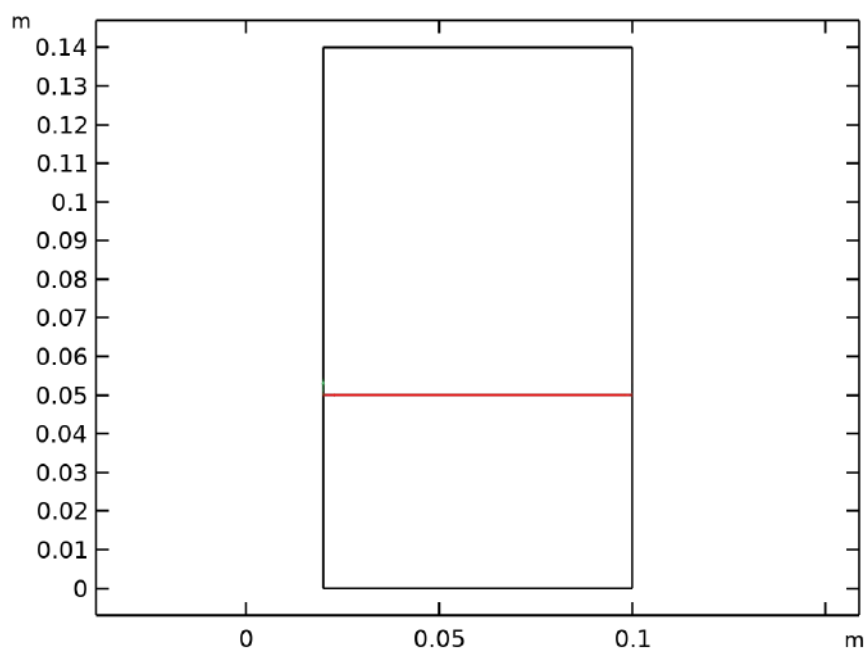
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.02, 0.05}, {0.1, 0.05}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D Z=0.05

4.1.7. Cut Line 2D Z=0.08

Data

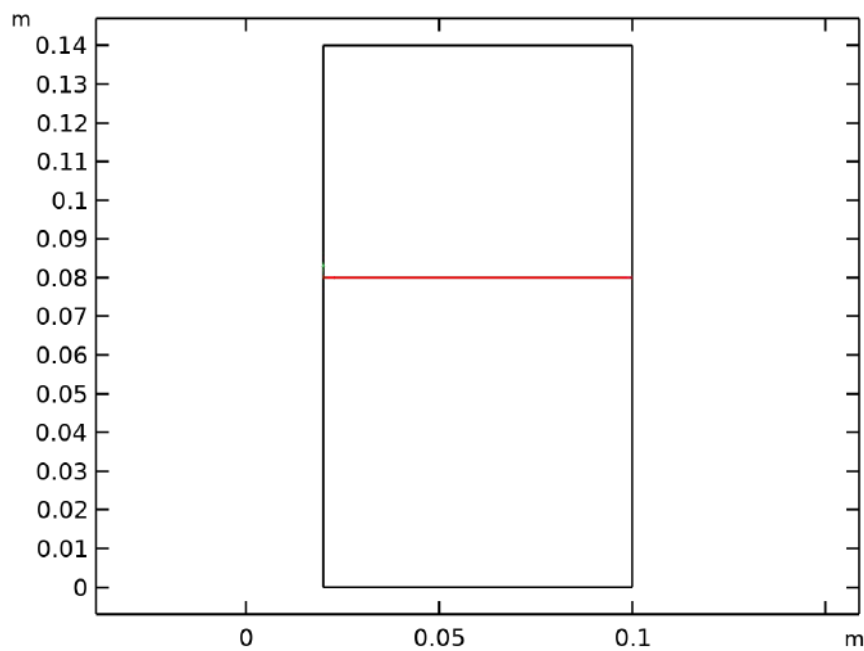
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.02, 0.08}, {0.1, 0.08}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D Z=0.08

4.1.8. Cut Line 2D Z=0.1

Data

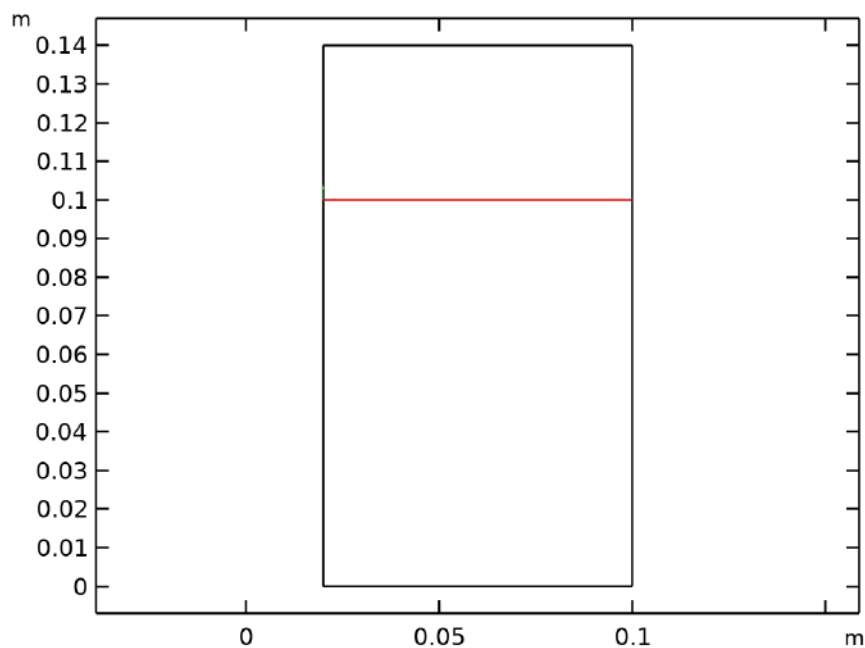
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.02, 0.1}, {0.1, 0.1}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D Z=0.1

4.1.9. Cut Line 2D R=0.03

Data

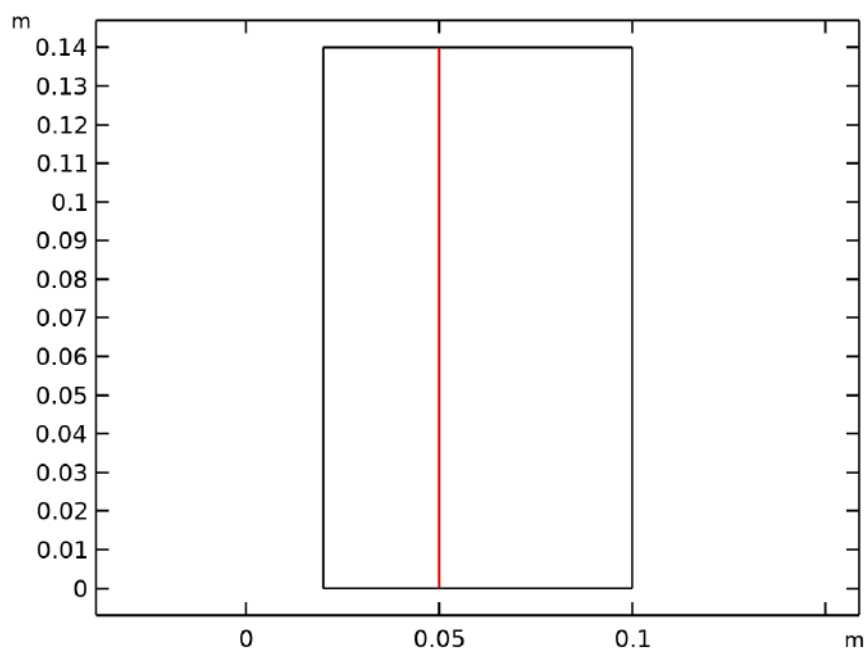
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.05, 0.0}, {0.05, 0.14}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D R=0.03

4.1.10. Cut Line 2D R=0.05

Data

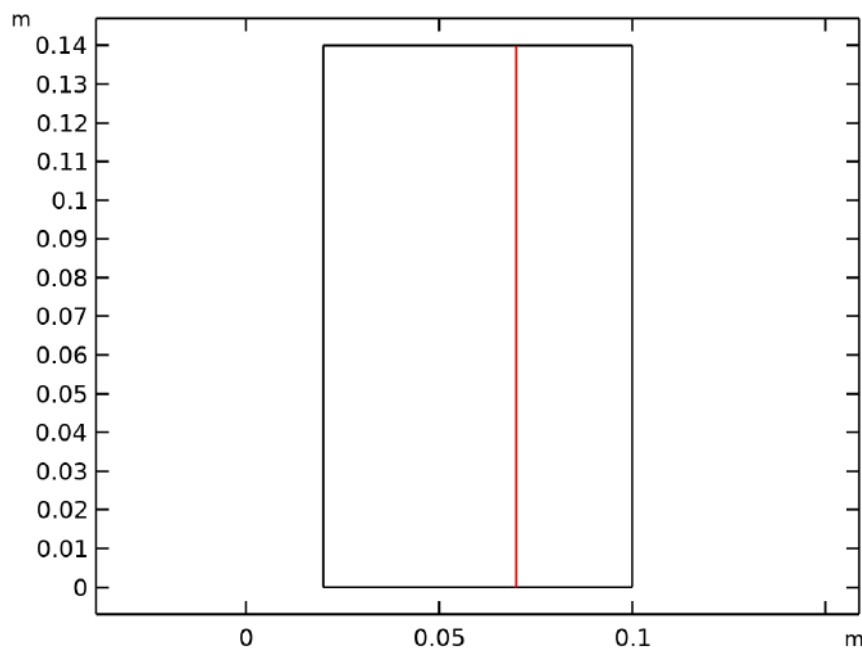
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.07, 0.0}, {0.07, 0.14}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D R=0.05

4.1.11. Cut Line 2D R=0.09

Data

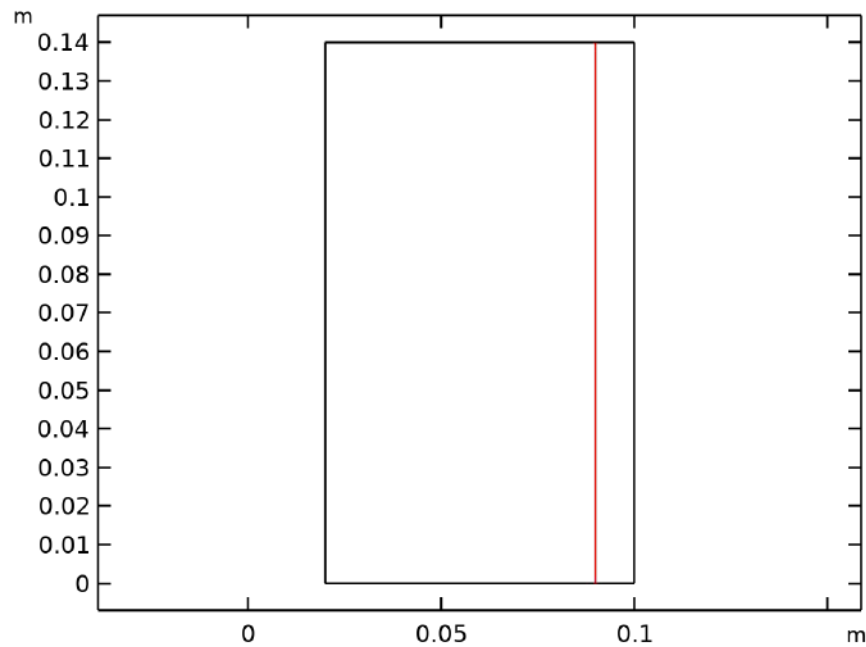
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.09, 0.0}, {0.09, 0.14}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D R=0.09

4.2. Derived Values

4.2.1. Point Evaluation A

Output		
Evaluated in		Table 2
Data		
Description	Value	
Dataset	Cut Point 2D A	
Expressions		
Expression	Unit	Description
T	K	Temperature

4.2.2. Point Evaluation B

Output

Evaluated in	Table 1
--------------	-------------------------

Data

Description	Value
Dataset	Cut Point 2D B

Expressions

Expression	Unit	Description
T	K	Temperature

4.2.3. Point Evaluation C

Output		
Evaluated in Table 1		
Data		
Description	Value	
Dataset	Cut Point 2D C	
Expressions		
Expression	Unit	Description

T	K	Temperature
---	---	-------------

4.3. Tables

4.3.1. Table 1

Point Evaluation A

Temperature (K), Point: (0.06, 0.04)	Temperature (K), Point: (0.08, 0.07)	Temperature (K), Point: (0.08, 0.1)
302.40	288.67	284.86

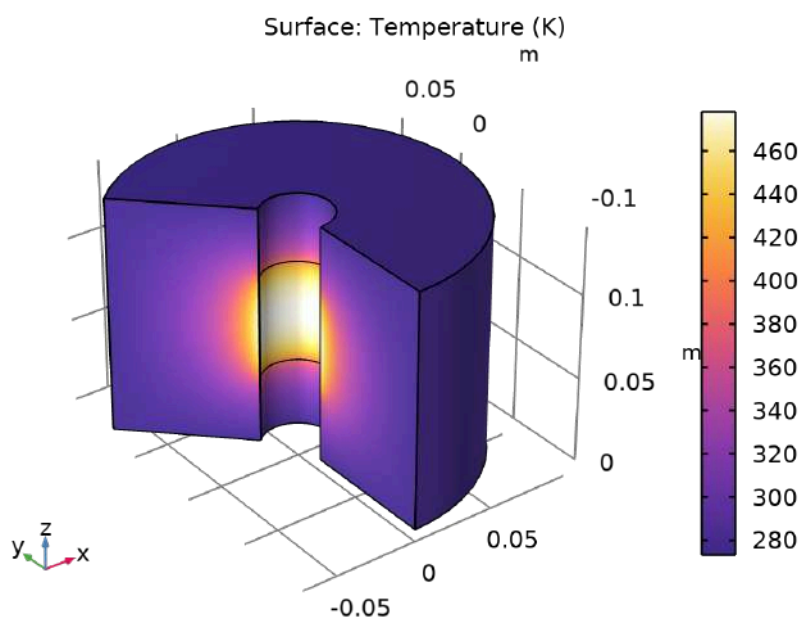
4.3.2. Table 2

Point Evaluation A

Temperature (K), Point: (0.06, 0.04)
302.40

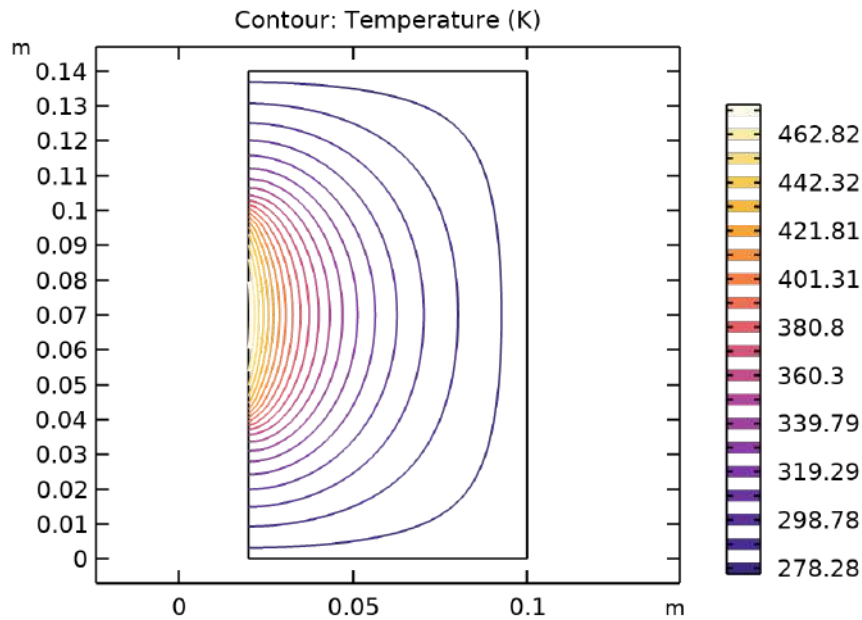
4.4. Plot Groups

4.4.1. Temperature, 3D (ht)



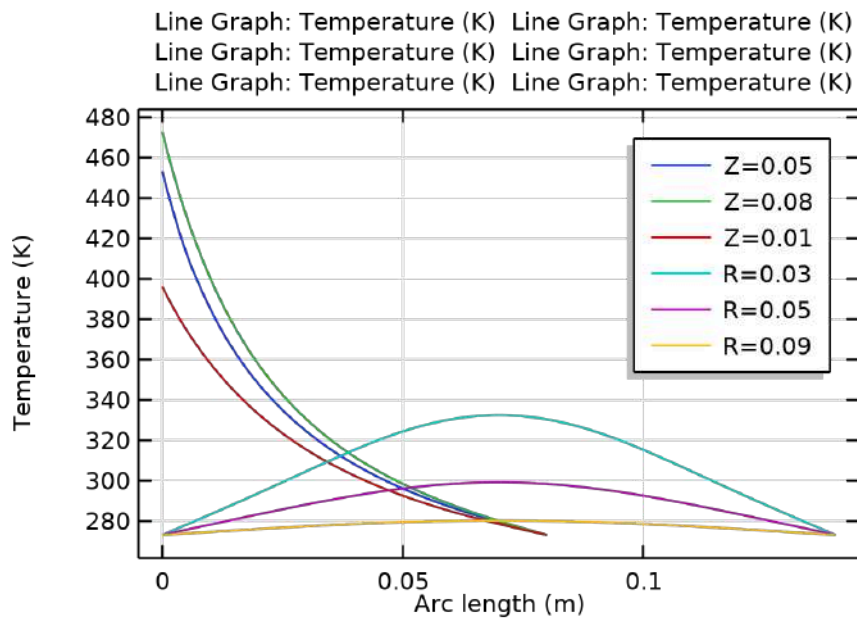
Surface: Temperature (K)

4.4.2. Isothermal Contours (ht)



Contour: Temperature (K)

4.4.3. 1D Plot Group 3



Line Graph: Temperature (K) Line Graph: Temperature (K) Line Graph: Temperature (K) Line Graph: Temperature (K) Line Graph: Temperature (K) Line Graph: Temperature (K)

A7Q1_Transiant

Report date

Oct 21, 2025, 12:34:01 AM

Contents

1. [Global Definitions](#)
 - 1.1. [Parameters](#)
2. [Component 1](#)
 - 2.1. [Definitions](#)
 - 2.2. [Geometry 1](#)
 - 2.3. [Heat Transfer in Solids](#)
 - 2.4. [Mesh 1](#)
3. [Study 2](#)
 - 3.1. [Time Dependent](#)
4. [Results](#)
 - 4.1. [Datasets](#)
 - 4.2. [Derived Values](#)
 - 4.3. [Tables](#)
 - 4.4. [Plot Groups](#)

1. Global Definitions

Date	Sep 21, 2025, 7:18:03 AM
------	--------------------------

Global settings

Name	A7Q1 Transiant.mph
Path	D:\4th Year\450\COMSOL\6.0\A7Q1\A7Q1_Transiant.mph
Version	COMSOL Multiphysics 6.0 (Build: 318)

Used products

COMSOL Multiphysics

Computer information

CPU	Intel64 Family 6 Model 141 Stepping 1, 6 cores
Operating system	Windows 10

1.1. Parameters

Parameters 1

Name	Expression	Value	Description
k	52 [W/(m*K)]	52 W/(m·K)	
rho	7850 [kg/m^3]	7850 kg/m ³	
Cp	460 [J/(kg*K)]	460 J/(kg·K)	
Q	0 [J]	0 J	
J	5e5 [W/m^2]	5E5 W/m ²	

2. Component 1

2.1. Definitions

2.1.1. Coordinate Systems

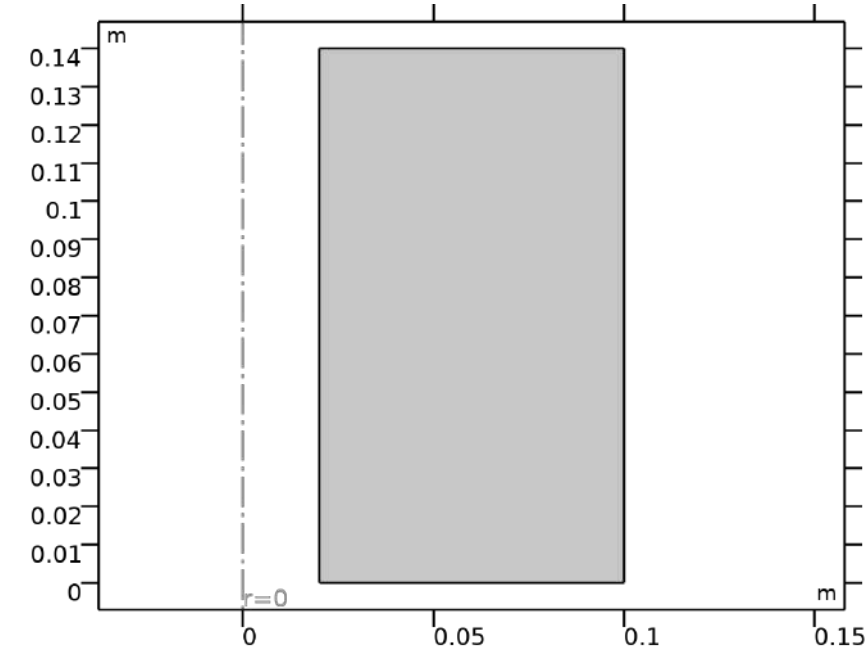
Boundary System 1

Coordinate system type	Boundary system
Tag	sys1

Coordinate names

First	Second	Third
t1	to	n

2.2. Geometry 1

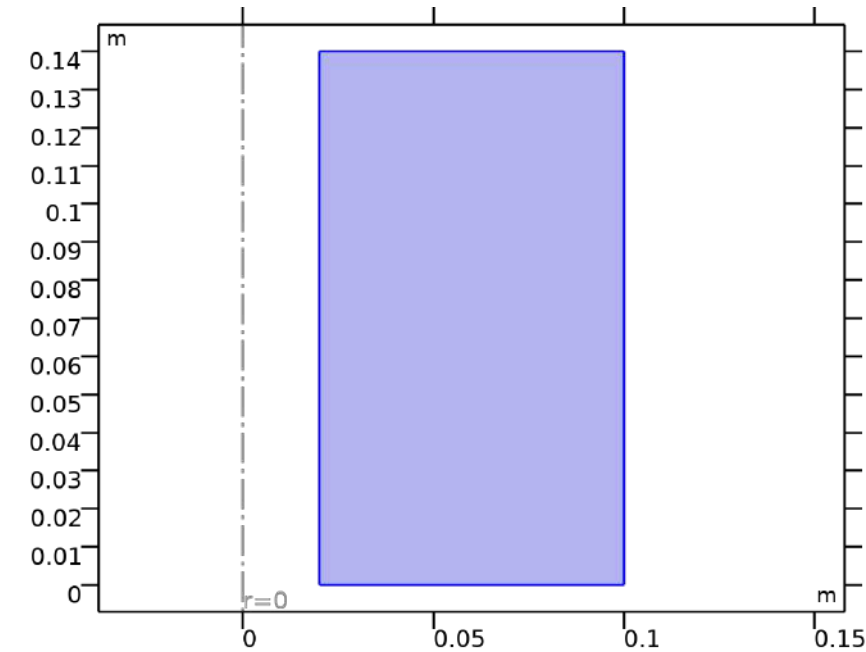


Geometry 1

Units

Length unit	m
Angular unit	deg

2.3. Heat Transfer in Solids



Heat Transfer in Solids

Equations

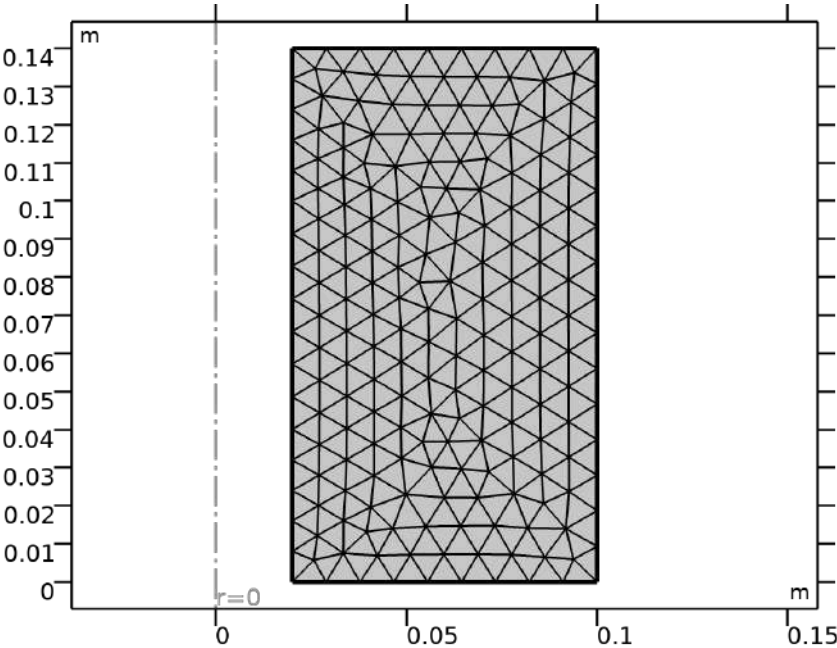
$$\rho C_p \frac{\partial T}{\partial t} + \rho C_p \mathbf{u} \cdot \nabla T + \nabla \cdot \mathbf{q} = Q + Q_{ted}$$

$$\mathbf{q} = -k \nabla T$$

Features

Name	Level
Solid 1	Domain
Initial Values 1	Domain
Axial Symmetry 1	Boundary
Thermal Insulation 1	Boundary
Temperature 1	Boundary
Heat Flux 1	Boundary

2.4. Mesh 1



Mesh 1

3. Study 2

Computation
information

Computation time	2 s
------------------	-----

3.1. Time Dependent

Times	Unit
range(0,1,120)	s

Study settings

Description	Value
Include geometric nonlinearity	Off

Study settings

Description	Value
Output times	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80,

81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120}
--

Physics and variables selection

Physics interface	Discretization
Heat Transfer in Solids (ht)	physics

Mesh selection

Geometry	Mesh
Geometry 1 (geom1)	mesh1

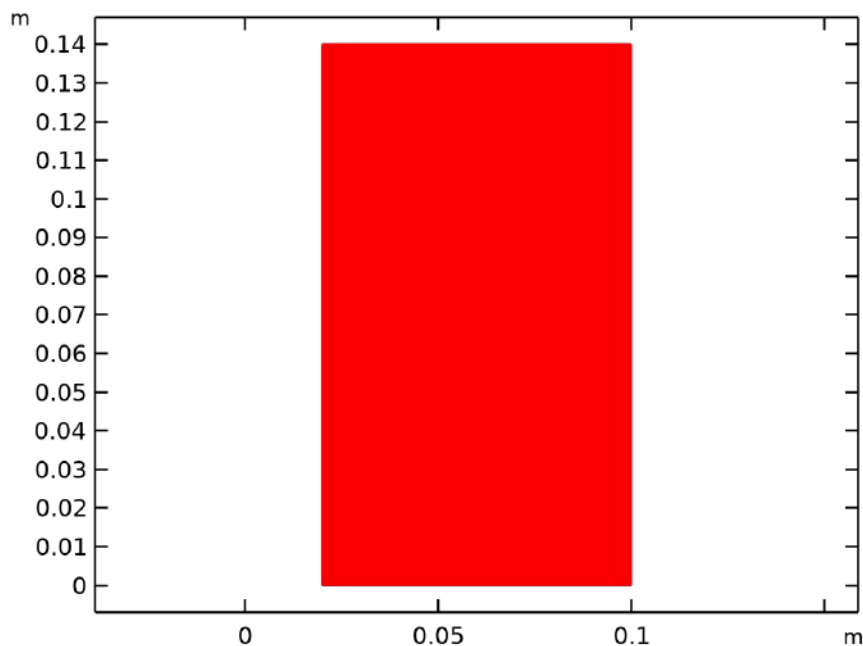
4. Results

4.1. Datasets

4.1.1. Study 2/Solution 1 (1)

Solution

Description	Value
Solution	Solution 1
Component	Component 1 (comp1)



Dataset: Study 2/Solution 1 (1)

4.1.2. Revolution 2D

Data

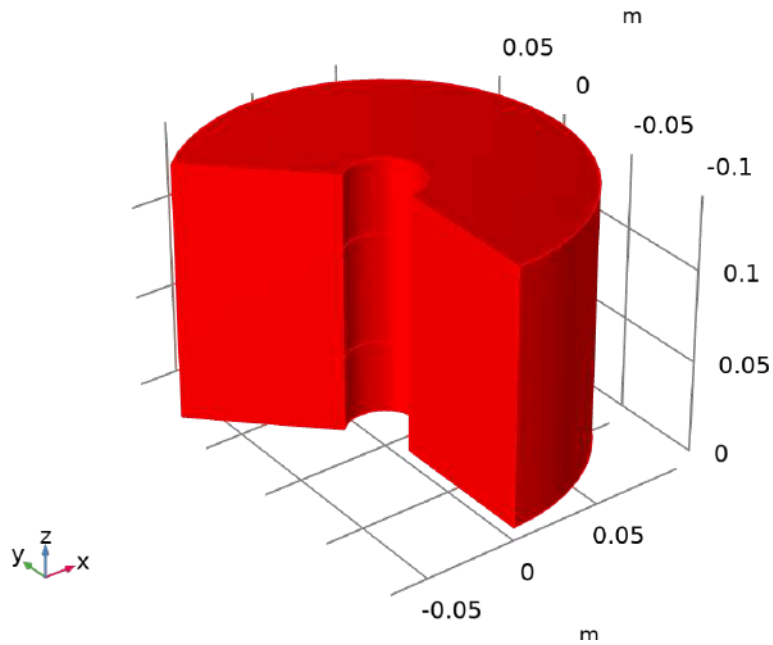
Description	Value
Dataset	Study 2/Solution 1 (1)

Axis data

Description	Value
Axis entry method	Two points
Points	{{0, 0}, {0, 1}}

Revolution layers

Description	Value
Start angle	-90
Revolution angle	225



Dataset: Revolution 2D

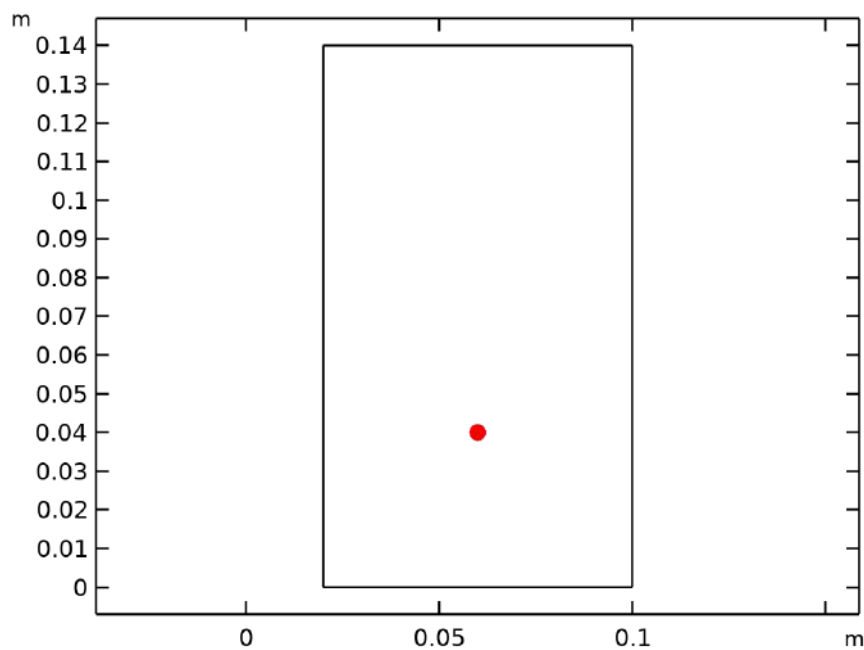
4.1.3. Cut Point 2D A

Data

Description	Value
Dataset	Study 2/Solution 1 (2)

Point data

Description	Value
Entry method	Coordinates
R	0.06
Z	0.04



Dataset: Cut Point 2D A

4.1.4. Cut Point 2D B

Data

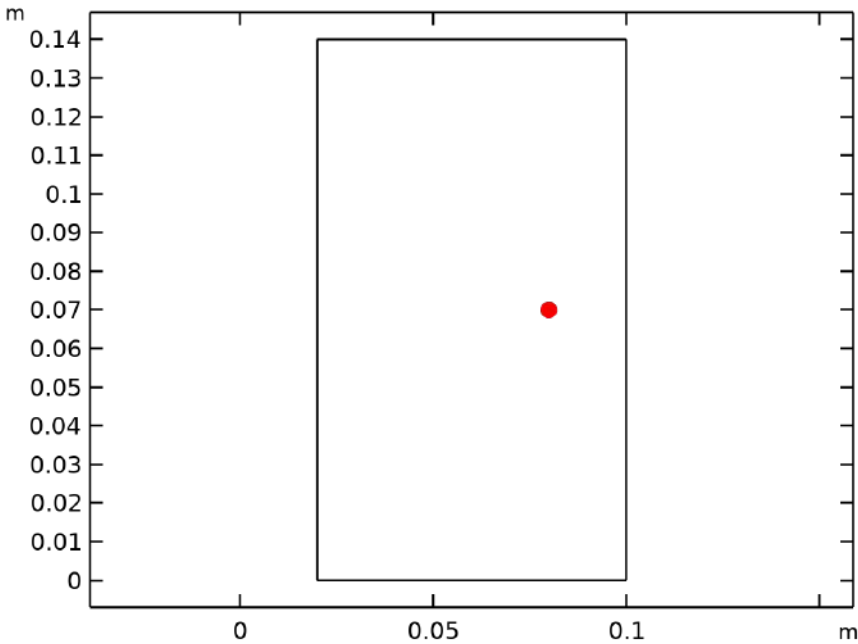
Description	Value
Dataset	Study 2/Solution 1 (2)

Point data

Description	Value
Entry method	Coordinates
R	0.08
Z	0.07

Advanced

Description	Value
Point number variable	cpt1n



Dataset: Cut Point 2D B

4.1.5. Cut Point 2D C

Data

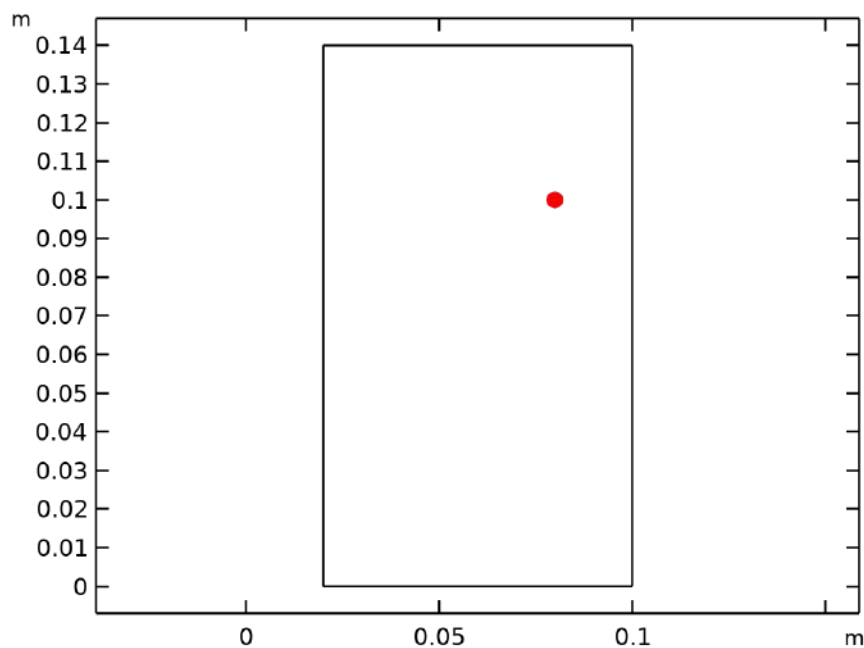
Description	Value
Dataset	Study 2/Solution 1 (1)

Point data

Description	Value
Entry method	Coordinates
R	0.08
Z	0.1

Advanced

Description	Value
Point number variable	cpt1n



Dataset: Cut Point 2D C

4.1.6. Cut Line 2D Z=0.05

Data

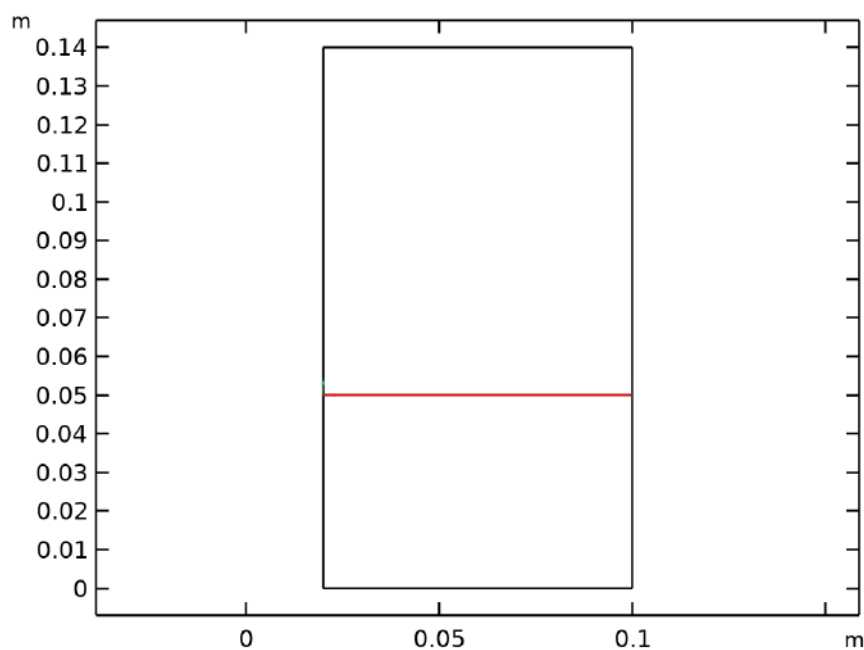
Description	Value
Dataset	Study 2/Solution 1 (1)

Line data

Description	Value
Line entry method	Two points
Points	{{0.02, 0.05}, {0.1, 0.05}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D Z=0.05

4.1.7. Cut Line 2D Z=0.08

Data

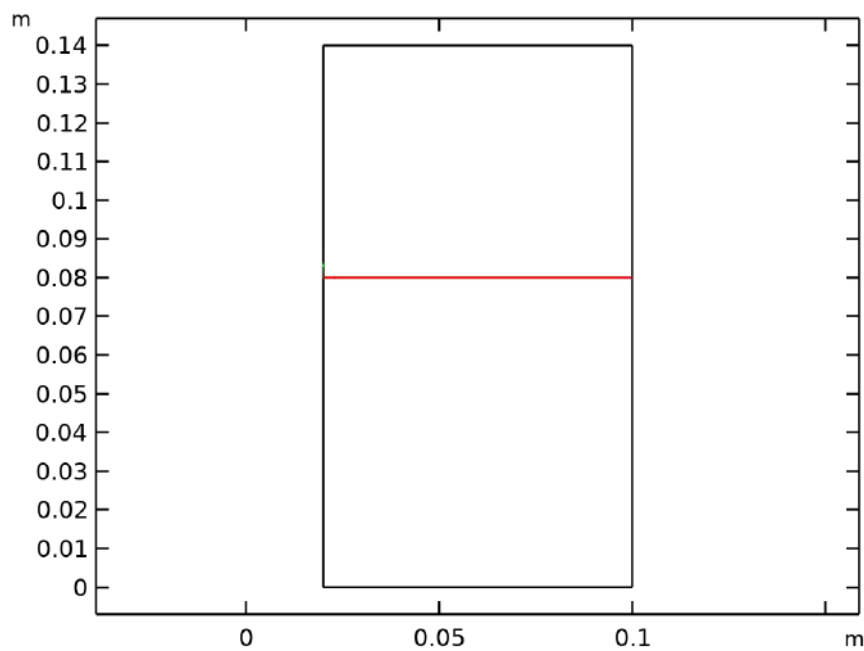
Description	Value
Dataset	Study 2/Solution 1 (1)

Line data

Description	Value
Line entry method	Two points
Points	{{0.02, 0.08}, {0.1, 0.08}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D Z=0.08

4.1.8. Cut Line 2D Z=0.1

Data

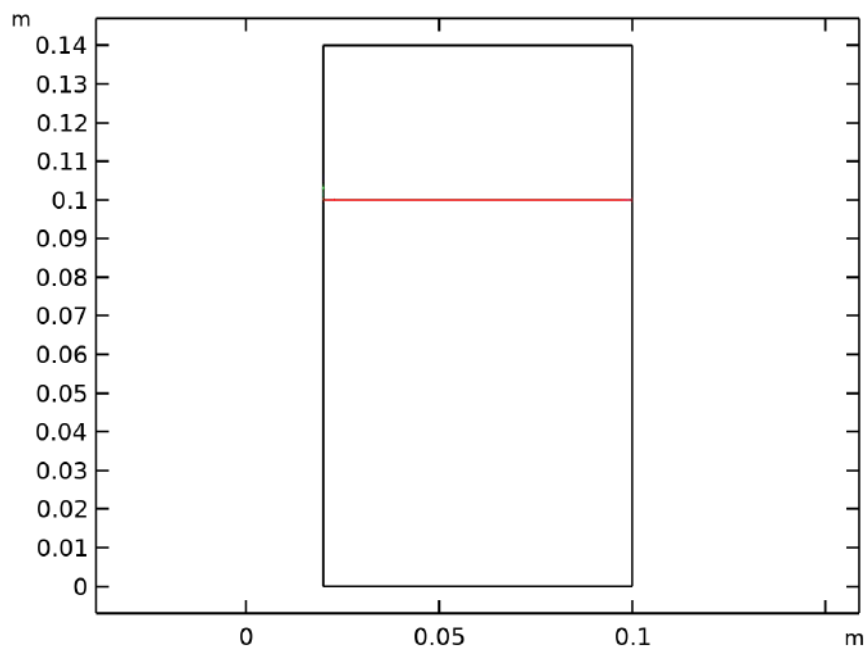
Description	Value
Dataset	Study 2/Solution 1 (1)

Line data

Description	Value
Line entry method	Two points
Points	{{0.02, 0.1}, {0.1, 0.1}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D Z=0.1

4.1.9. Cut Line 2D R=0.03

Data

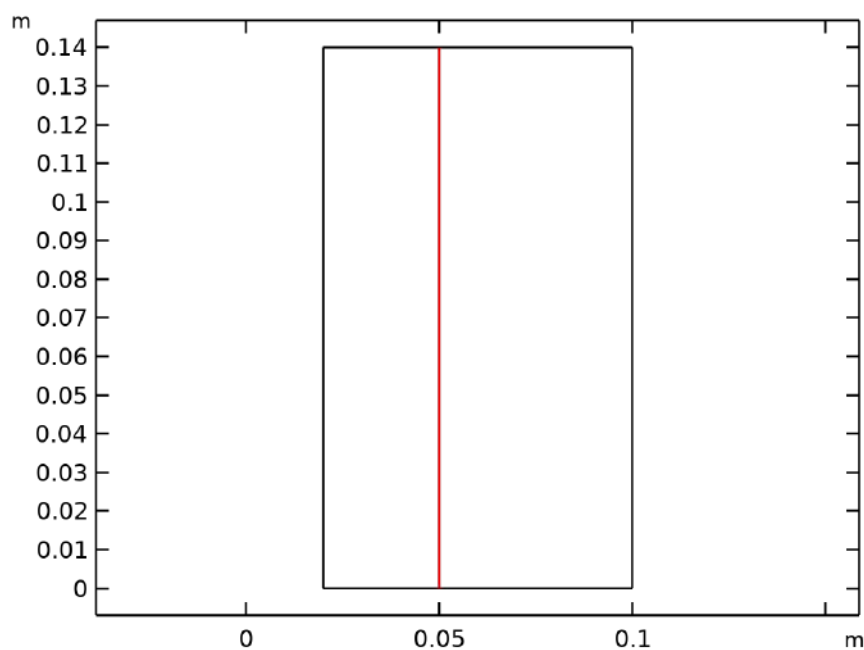
Description	Value
Dataset	Study 2/Solution 1 (1)

Line data

Description	Value
Line entry method	Two points
Points	{{0.05, 0.0}, {0.05, 0.14}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D R=0.03

4.1.10. Cut Line 2D R=0.05

Data

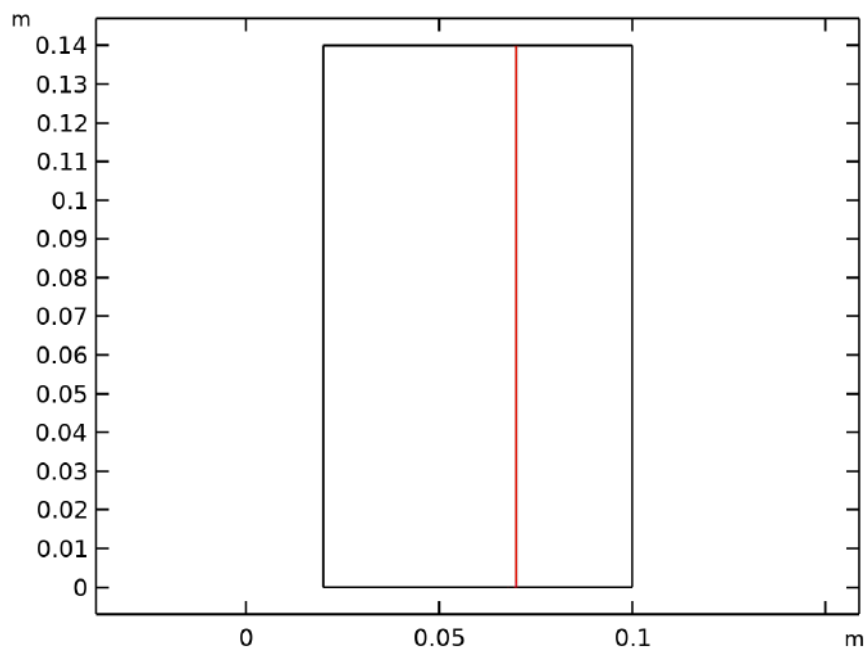
Description	Value
Dataset	Study 2/Solution 1 (1)

Line data

Description	Value
Line entry method	Two points
Points	{{0.07, 0.0}, {0.07, 0.14}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D R=0.05

4.1.11. Cut Line 2D R=0.09

Data

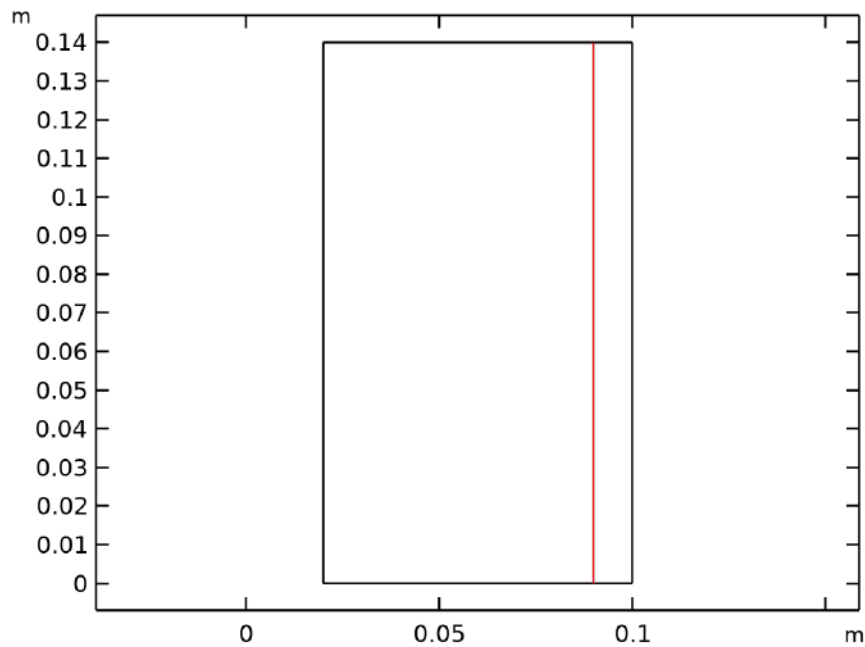
Description	Value
Dataset	Study 2/Solution 1 (1)

Line data

Description	Value
Line entry method	Two points
Points	{{0.09, 0.0}, {0.09, 0.14}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}

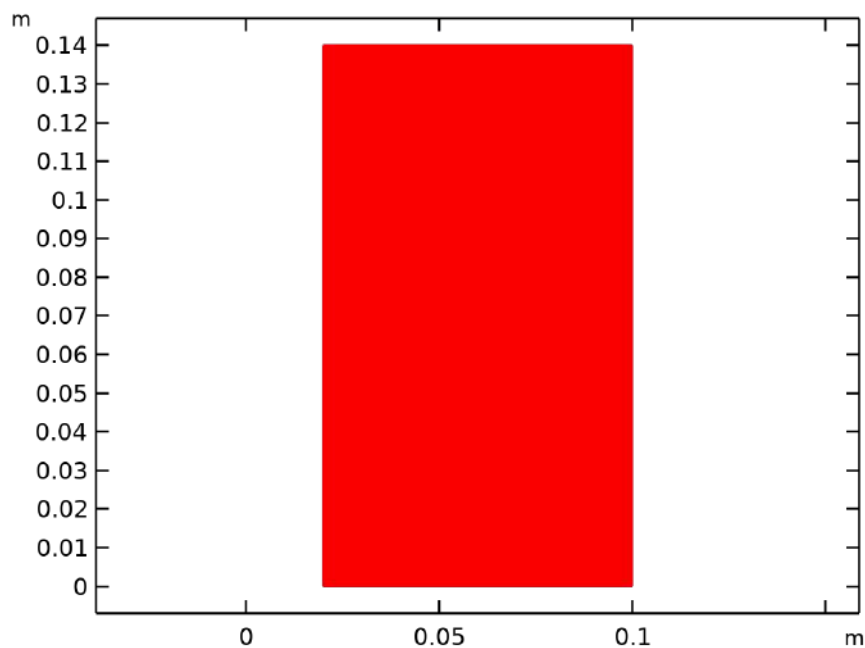


Dataset: Cut Line 2D R=0.09

4.1.12. Study 2/Solution 1 (2)

Solution

Description	Value
Solution	Solution 1
Component	Component 1 (comp1)



Dataset: Study 2/Solution 1 (2)

4.1.13. Revolution 2D 1

Data

Description	Value
Dataset	Study 2/Solution 1 (2)

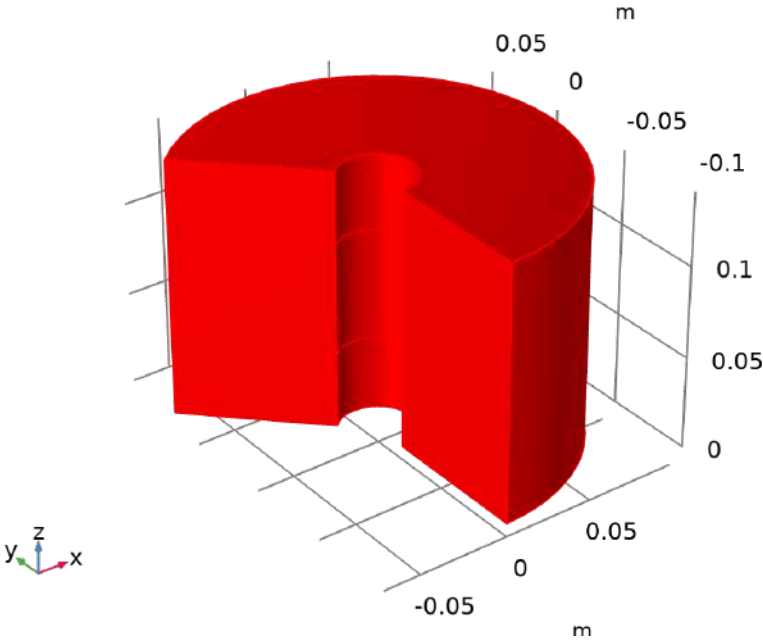
Axis data

Description	Value
-------------	-------

Axis entry method	Two points
Points	{{0, 0}, {0, 1}}

Revolution layers

Description	Value
Start angle	-90
Revolution angle	225



Dataset: Revolution 2D 1

4.2. Derived Values

4.2.1. Point Evaluation A

Output

Evaluated in	Table 2
--------------	-------------------------

Data

Description	Value
Dataset	Cut Point 2D A

Expressions

Expression	Unit	Description
T	K	Temperature

4.2.2. Point Evaluation B

Output

Evaluated in	Table 1
--------------	-------------------------

Data

Description	Value
Dataset	Cut Point 2D B

Expressions

Expression	Unit	Description
T	K	Temperature

4.2.3. Point Evaluation C

Output

Evaluated in [Table 1](#)

Data

Description	Value
Dataset	Cut Point 2D C

Expressions

Expression	Unit	Description
T	K	Temperature

4.3. Tables

4.3.1. Table 1

Point Evaluation A

Time (s)	Temperature (K), Point: (0.08, 0.07)	Temperature (K), Point: (0.08, 0.1)	Temperature (K), Point: (0.08, 0.1)
0.0000	293.15	293.15	293.15
1.0000	293.12	293.12	293.12
2.0000	292.87	292.87	292.87
3.0000	292.34	292.34	292.34
4.0000	291.71	291.69	291.69
5.0000	291.06	291.02	291.02
6.0000	290.42	290.33	290.33
7.0000	289.80	289.67	289.67
8.0000	289.22	289.03	289.03
9.0000	288.66	288.42	288.42
10.000	288.23	287.92	287.92
11.000	287.80	287.42	287.42
12.000	287.37	286.92	286.92
13.000	286.94	286.41	286.41
14.000	286.59	285.99	285.99
15.000	286.29	285.61	285.61
16.000	285.98	285.24	285.24
17.000	285.68	284.86	284.86
18.000	285.39	284.49	284.49
19.000	285.23	284.28	284.28
20.000	285.08	284.07	284.07
21.000	284.93	283.86	283.86
22.000	284.78	283.65	283.65
23.000	284.62	283.43	283.43
24.000	284.47	283.22	283.22
25.000	284.32	283.01	283.01
26.000	284.17	282.80	282.80
27.000	284.01	282.59	282.59
28.000	283.95	282.48	282.48
29.000	283.89	282.37	282.37
30.000	283.83	282.27	282.27
31.000	283.77	282.16	282.16
32.000	283.71	282.05	282.05

33.000	283.65	281.95	281.95
34.000	283.59	281.84	281.84
35.000	283.53	281.74	281.74
36.000	283.47	281.63	281.63
37.000	283.45	281.58	281.58
38.000	283.45	281.54	281.54
39.000	283.44	281.50	281.50
40.000	283.43	281.46	281.46
41.000	283.43	281.42	281.42
42.000	283.42	281.37	281.37
43.000	283.41	281.33	281.33
44.000	283.41	281.29	281.29
45.000	283.40	281.25	281.25
46.000	283.42	281.24	281.24
47.000	283.45	281.24	281.24
48.000	283.48	281.25	281.25
49.000	283.50	281.25	281.25
50.000	283.53	281.25	281.25
51.000	283.56	281.25	281.25
52.000	283.59	281.26	281.26
53.000	283.62	281.26	281.26
54.000	283.64	281.26	281.26
55.000	283.67	281.26	281.26
56.000	283.70	281.27	281.27
57.000	283.73	281.27	281.27
58.000	283.77	281.29	281.29
59.000	283.81	281.31	281.31
60.000	283.85	281.33	281.33
61.000	283.90	281.35	281.35
62.000	283.94	281.38	281.38
63.000	283.98	281.40	281.40
64.000	284.02	281.42	281.42
65.000	284.07	281.45	281.45
66.000	284.11	281.47	281.47
67.000	284.15	281.49	281.49
68.000	284.20	281.52	281.52
69.000	284.24	281.54	281.54
70.000	284.28	281.57	281.57
71.000	284.33	281.60	281.60
72.000	284.38	281.63	281.63
73.000	284.42	281.66	281.66
74.000	284.47	281.69	281.69
75.000	284.52	281.72	281.72
76.000	284.56	281.75	281.75
77.000	284.61	281.79	281.79
78.000	284.66	281.82	281.82
79.000	284.71	281.85	281.85
80.000	284.75	281.88	281.88

81.000	284.80	281.91	281.91
82.000	284.85	281.94	281.94
83.000	284.89	281.97	281.97
84.000	284.94	282.01	282.01
85.000	284.99	282.04	282.04
86.000	285.03	282.07	282.07
87.000	285.08	282.10	282.10
88.000	285.13	282.14	282.14
89.000	285.17	282.17	282.17
90.000	285.22	282.20	282.20
91.000	285.27	282.24	282.24
92.000	285.31	282.27	282.27
93.000	285.36	282.31	282.31
94.000	285.40	282.34	282.34
95.000	285.45	282.37	282.37
96.000	285.49	282.40	282.40
97.000	285.54	282.44	282.44
98.000	285.58	282.47	282.47
99.000	285.63	282.50	282.50
100.00	285.67	282.54	282.54
101.00	285.72	282.57	282.57
102.00	285.76	282.60	282.60
103.00	285.80	282.63	282.63
104.00	285.85	282.67	282.67
105.00	285.89	282.70	282.70
106.00	285.93	282.73	282.73
107.00	285.97	282.76	282.76
108.00	286.01	282.79	282.79
109.00	286.05	282.82	282.82
110.00	286.09	282.86	282.86
111.00	286.13	282.89	282.89
112.00	286.17	282.92	282.92
113.00	286.21	282.95	282.95
114.00	286.25	282.98	282.98
115.00	286.29	283.01	283.01
116.00	286.33	283.03	283.03
117.00	286.37	283.06	283.06
118.00	286.40	283.09	283.09
119.00	286.44	283.12	283.12
120.00	286.47	283.15	283.15

4.3.2. Table 2

Point Evaluation A

Time (s)	Temperature (K), Point: (0.06, 0.04)
0.0000	293.15
1.0000	293.15
2.0000	293.15

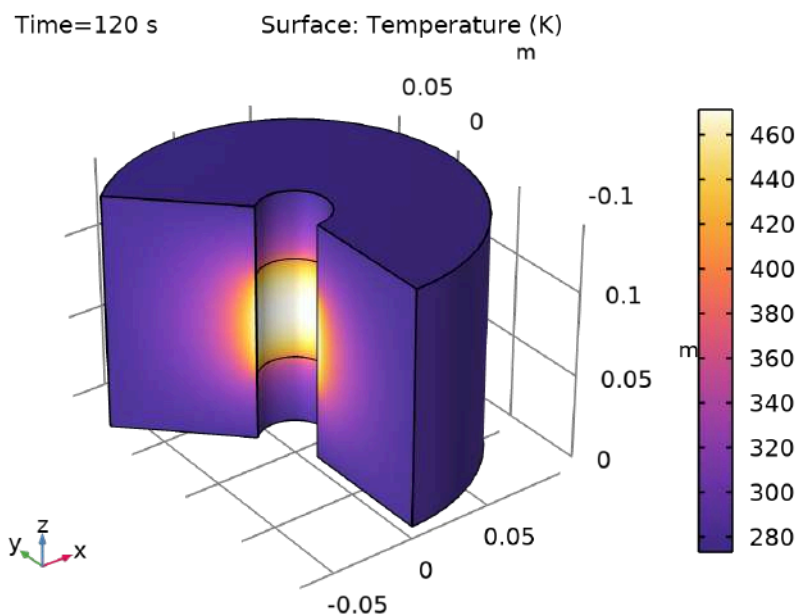
3.0000	293.14
4.0000	293.11
5.0000	293.05
6.0000	292.97
7.0000	292.88
8.0000	292.77
9.0000	292.66
10.000	292.54
11.000	292.42
12.000	292.30
13.000	292.18
14.000	292.08
15.000	291.99
16.000	291.90
17.000	291.81
18.000	291.72
19.000	291.70
20.000	291.69
21.000	291.67
22.000	291.66
23.000	291.64
24.000	291.62
25.000	291.61
26.000	291.59
27.000	291.58
28.000	291.62
29.000	291.66
30.000	291.70
31.000	291.75
32.000	291.79
33.000	291.83
34.000	291.87
35.000	291.91
36.000	291.96
37.000	292.03
38.000	292.11
39.000	292.19
40.000	292.26
41.000	292.34
42.000	292.42
43.000	292.50
44.000	292.57
45.000	292.65
46.000	292.74
47.000	292.84
48.000	292.93
49.000	293.03
50.000	293.12

51.000	293.22
52.000	293.31
53.000	293.41
54.000	293.51
55.000	293.60
56.000	293.70
57.000	293.79
58.000	293.89
59.000	293.99
60.000	294.08
61.000	294.18
62.000	294.28
63.000	294.38
64.000	294.47
65.000	294.57
66.000	294.67
67.000	294.76
68.000	294.86
69.000	294.96
70.000	295.05
71.000	295.14
72.000	295.23
73.000	295.32
74.000	295.42
75.000	295.51
76.000	295.60
77.000	295.69
78.000	295.78
79.000	295.87
80.000	295.96
81.000	296.05
82.000	296.14
83.000	296.23
84.000	296.32
85.000	296.40
86.000	296.49
87.000	296.57
88.000	296.66
89.000	296.74
90.000	296.83
91.000	296.91
92.000	296.99
93.000	297.07
94.000	297.15
95.000	297.23
96.000	297.31
97.000	297.39
98.000	297.47

99.000	297.54
100.00	297.62
101.00	297.69
102.00	297.77
103.00	297.84
104.00	297.91
105.00	297.99
106.00	298.06
107.00	298.13
108.00	298.19
109.00	298.26
110.00	298.33
111.00	298.39
112.00	298.46
113.00	298.52
114.00	298.59
115.00	298.65
116.00	298.71
117.00	298.77
118.00	298.83
119.00	298.89
120.00	298.95

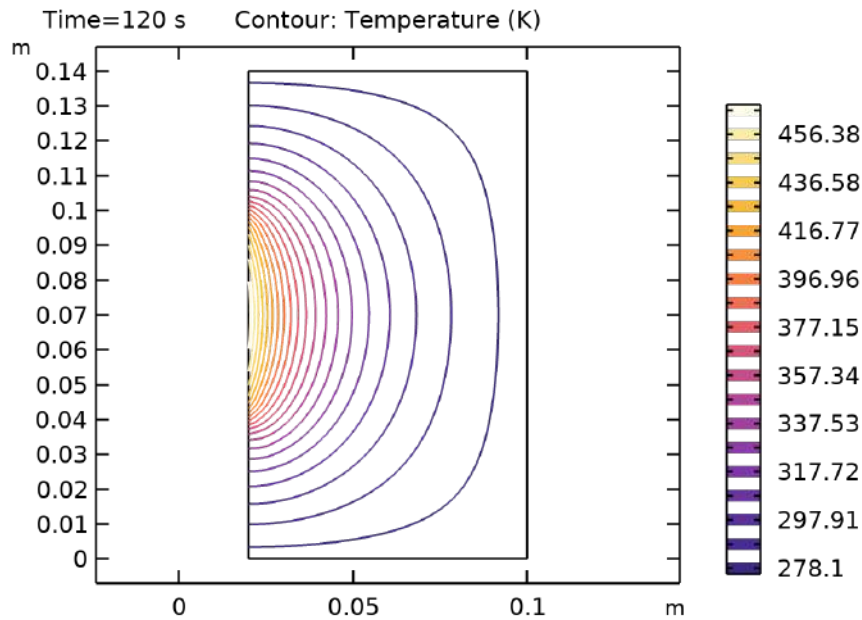
4.4. Plot Groups

4.4.1. Temperature, 3D (ht)



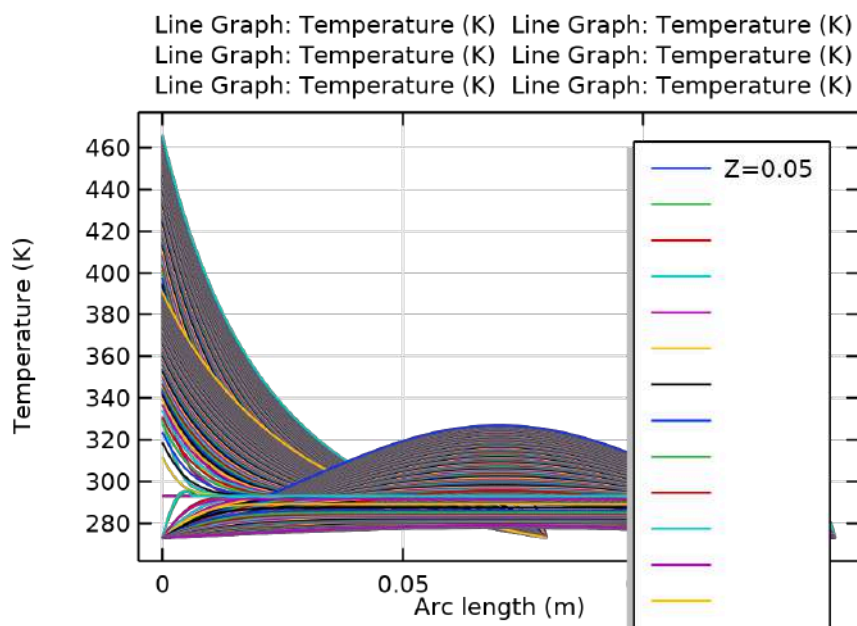
Surface: Temperature (K)

4.4.2. Isothermal Contours (ht)



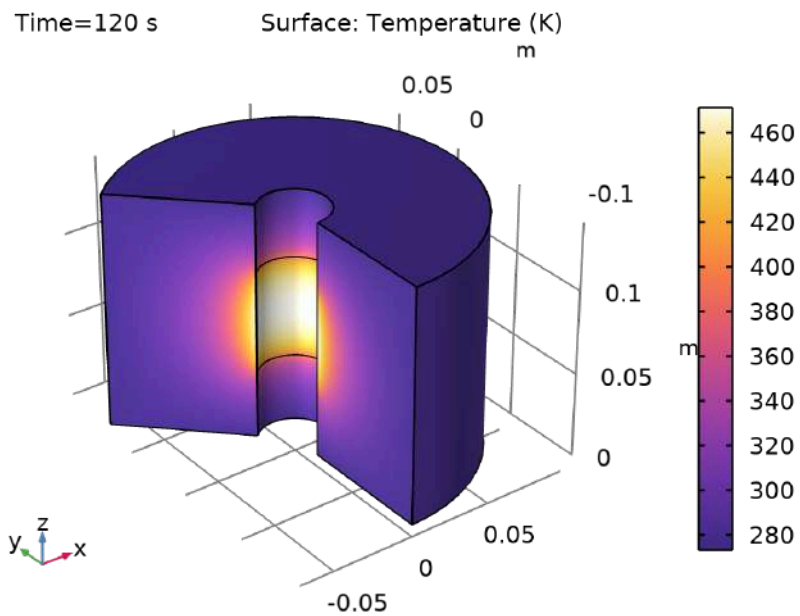
Contour: Temperature (K)

4.4.3. 1D Plot Group 3



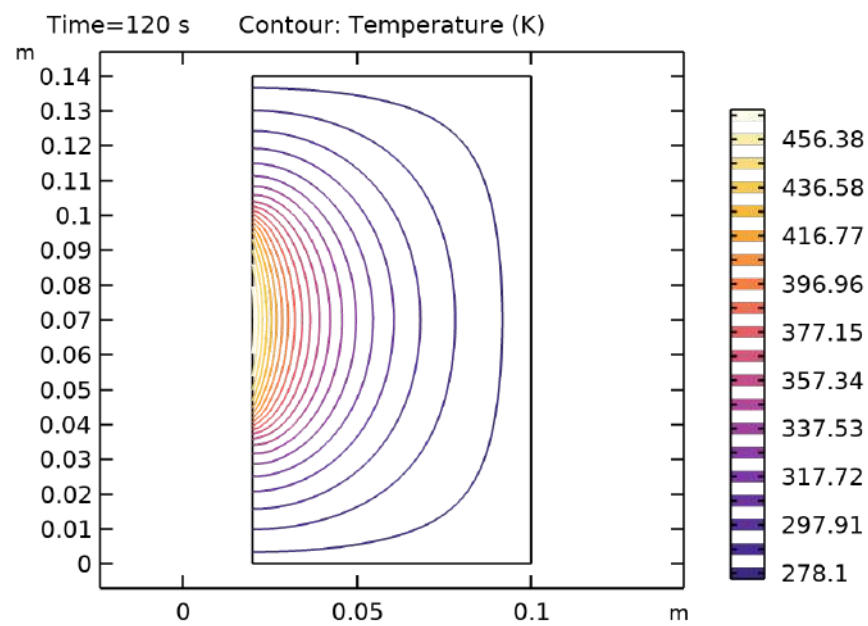
Line Graph: Temperature (K) Line Graph: Temperature (K) Line Graph: Temperature (K) Line Graph: Temperature (K)
Line Graph: Temperature (K) Line Graph: Temperature (K) Line Graph: Temperature (K)

4.4.4. Temperature, 3D (ht) 1



Surface: Temperature (K)

4.4.5. Isothermal Contours (ht) 1



Contour: Temperature (K)

A7Q2

Report date

Oct 21, 2025, 12:41:06 AM

Contents

1. [Global Definitions](#)
 - 1.1. [Parameters](#)
2. [Component 1](#)
 - 2.1. [Definitions](#)
 - 2.2. [Geometry 1](#)
 - 2.3. [Laminar Flow](#)
 - 2.4. [Mesh 1](#)
3. [Study 1](#)
 - 3.1. [Parametric Sweep](#)
 - 3.2. [Time Dependent](#)
4. [Results](#)
 - 4.1. [Datasets](#)
 - 4.2. [Plot Groups](#)

1. Global Definitions

Date	Sep 21, 2025, 7:56:52 AM
------	--------------------------

Global settings

Name	A7Q2.mph
Path	D:\4th Year\450\COMSOL\6.0\A7Q2\A7Q2.mph
Version	COMSOL Multiphysics 6.0 (Build: 318)

Used products

COMSOL Multiphysics

Computer information

CPU	Intel64 Family 6 Model 141 Stepping 1, 6 cores
Operating system	Windows 10

1.1. Parameters

Parameters 1

Name	Expression	Value	Description
rho	1080 [kg/m^3]	1080 kg/m ³	
mu	0.00325 [Pa*s]	0.00325 Pa·s	
u_max	0.5 [m/s]	0.5 m/s	

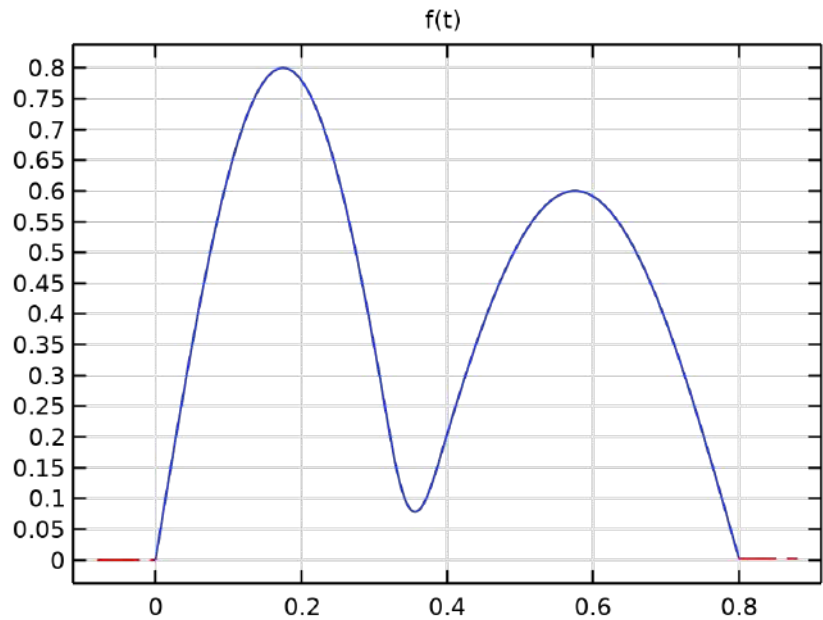
2. Component 1

2.1. Definitions

2.1.1. Functions

Piecewise 1

Function name	f
Function type	Piecewise



Piecewise 1

2.1.2. Coordinate Systems

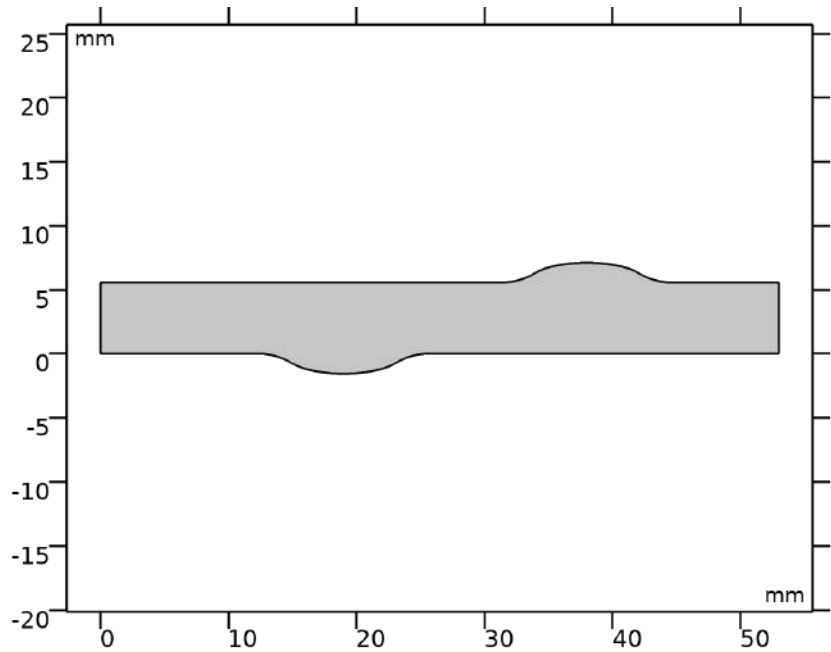
Boundary System 1

Coordinate system type	Boundary system
Tag	sys1

Coordinate names

First	Second	Third
t1	n	to

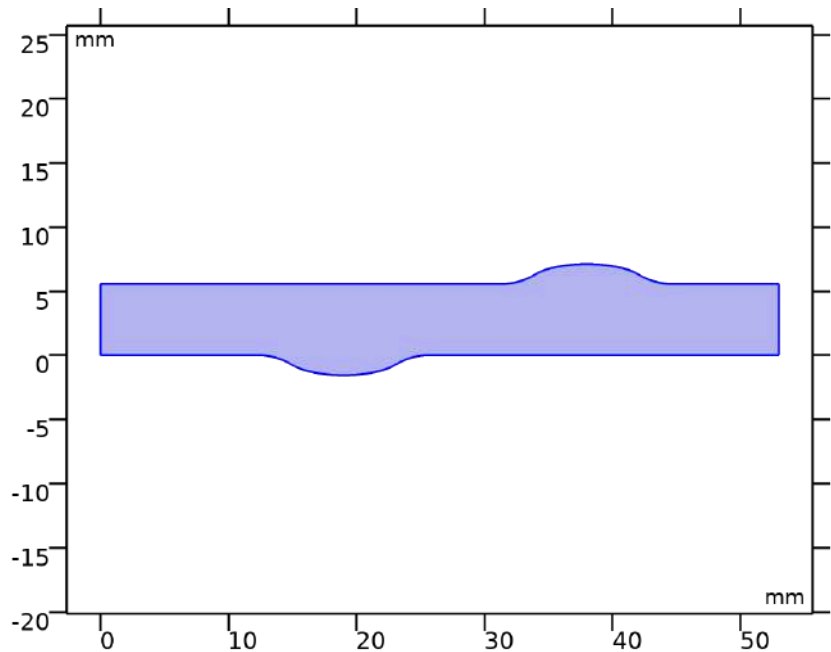
2.2. Geometry 1



Geometry 1

Units	
Length unit	mm
Angular unit	deg

2.3. Laminar Flow



Laminar Flow

Equations

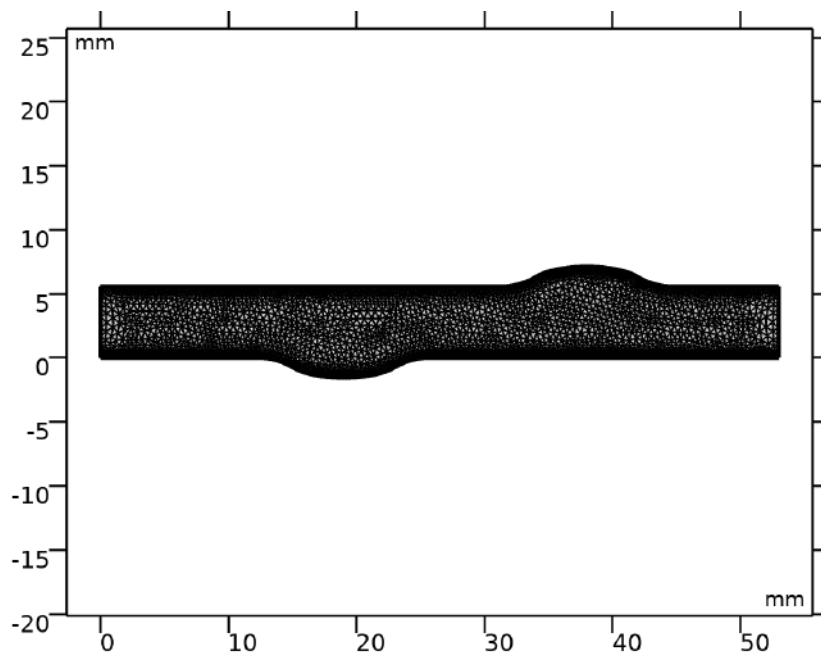
$$\rho \frac{\partial \mathbf{u}}{\partial t} + \rho (\mathbf{u} \cdot \nabla) \mathbf{u} = \nabla \cdot [-p\mathbf{I} + \mathbf{K}] + \mathbf{F}$$

$$\rho \nabla \cdot \mathbf{u} = 0$$

Features

Name	Level
Fluid Properties 1	Domain
Initial Values 1	Domain
Wall 1	Boundary
Inlet 1	Boundary
Outlet 1	Boundary

2.4. Mesh 1



Mesh 1

3. Study 1

Computation
information

Computation time	53 s
------------------	------

3.1. Parametric Sweep

Parameter name	Parameter value list	Parameter unit
u_max	0.25 0.45 0.5 0.65 0.85	m/s

Study settings

Description	Value
Sweep type	Specified combinations
Parameter name	u_max
Unit	m/s

Parameters

Parameter name	Parameter value list	Parameter unit
u_max	0.25 0.45 0.5 0.65 0.85	m/s

3.2. Time Dependent

Times	Unit
range(0,0.1,0.8)	s

Study settings

Description	Value
Include geometric nonlinearity	Off

Study settings

Description	Value
Output times	{0, 0.1, 0.2, 0.30000000000000004, 0.4, 0.5, 0.6000000000000001, 0.7000000000000001, 0.8}

Physics and variables selection

Physics interface	Discretization
Laminar Flow (spf)	physics

Mesh selection

Geometry	Mesh
Geometry 1 (geom1)	mesh1

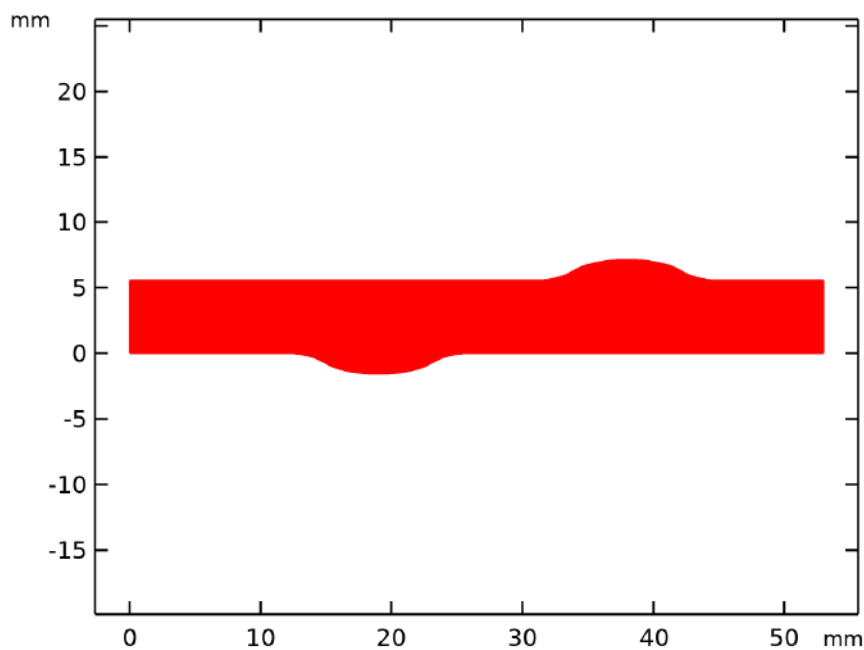
4. Results

4.1. Datasets

4.1.1. Study 1/Solution 1

Solution

Description	Value
Solution	Solution 1
Component	Component 1 (comp1)



Dataset: Study 1/Solution 1

4.1.2. Cut Line 2D X=8 mm

Data

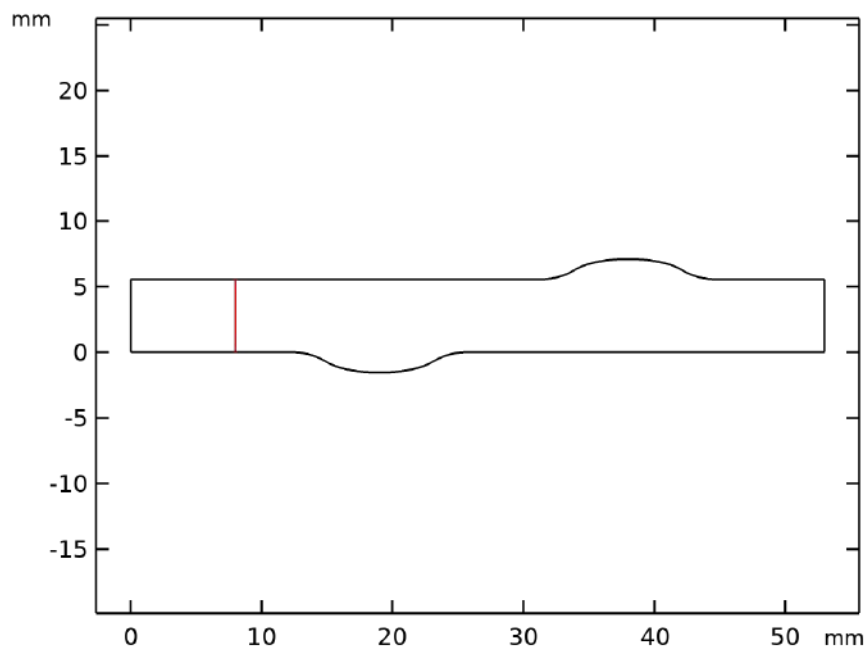
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{8, 0}, {8, 5.57}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D X=8 mm

4.1.3. Cut Line 2D X=15mm

Data

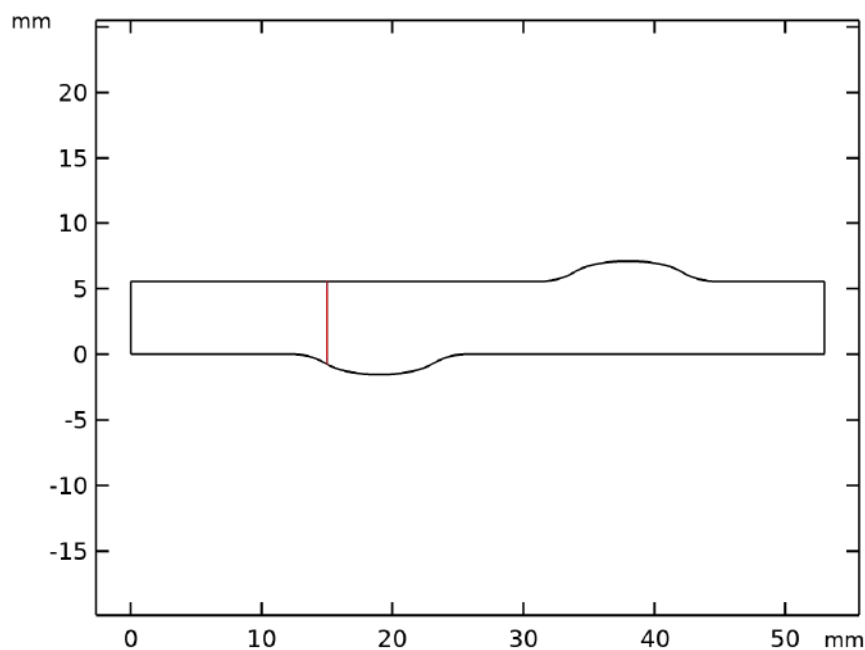
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{15, 5.57}, {15, -0.8}}

Advanced

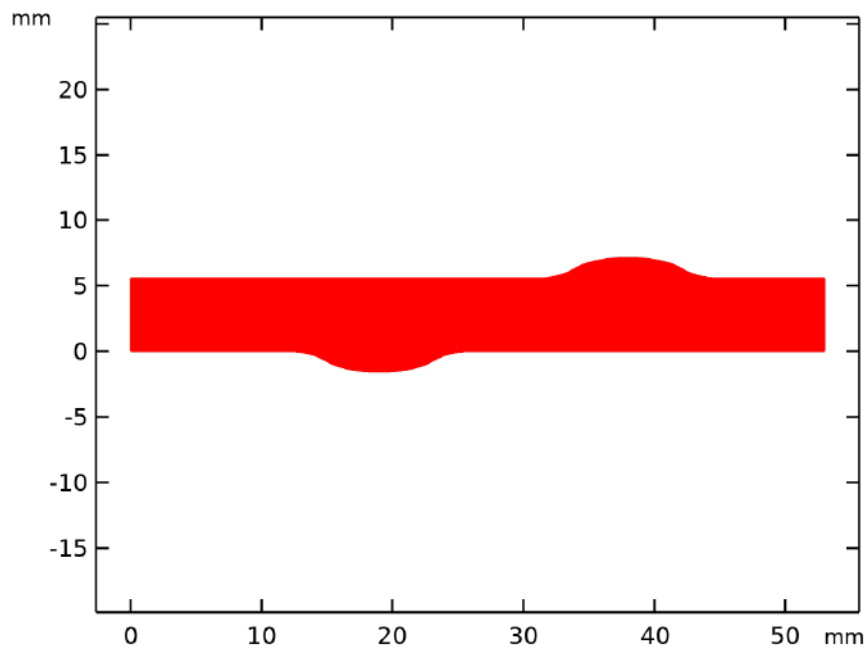
Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D X=15mm

4.1.4. Study 1/Parametric Solutions 1

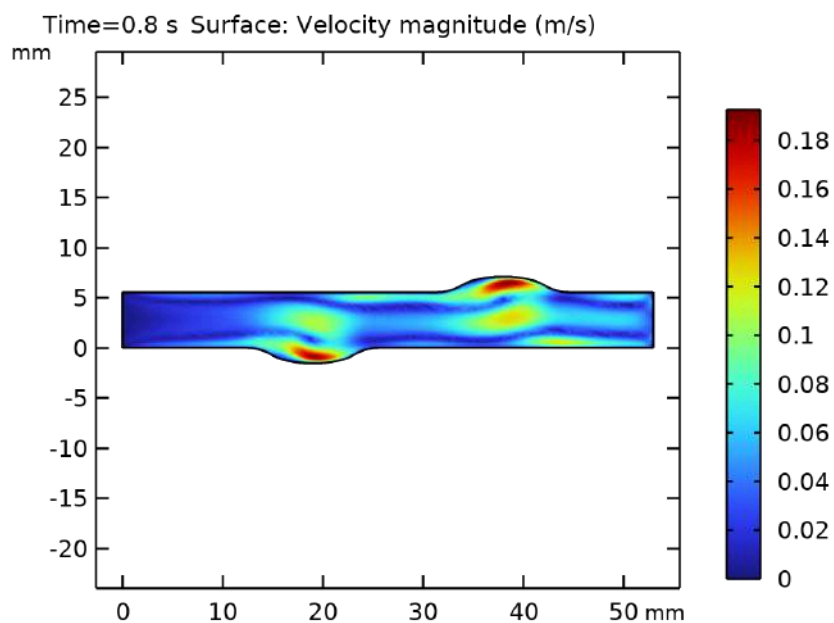
Solution	
Description	Value
Solution	Parametric Solutions 1
Component	Component 1 (comp1)



Dataset: Study 1/Parametric Solutions 1

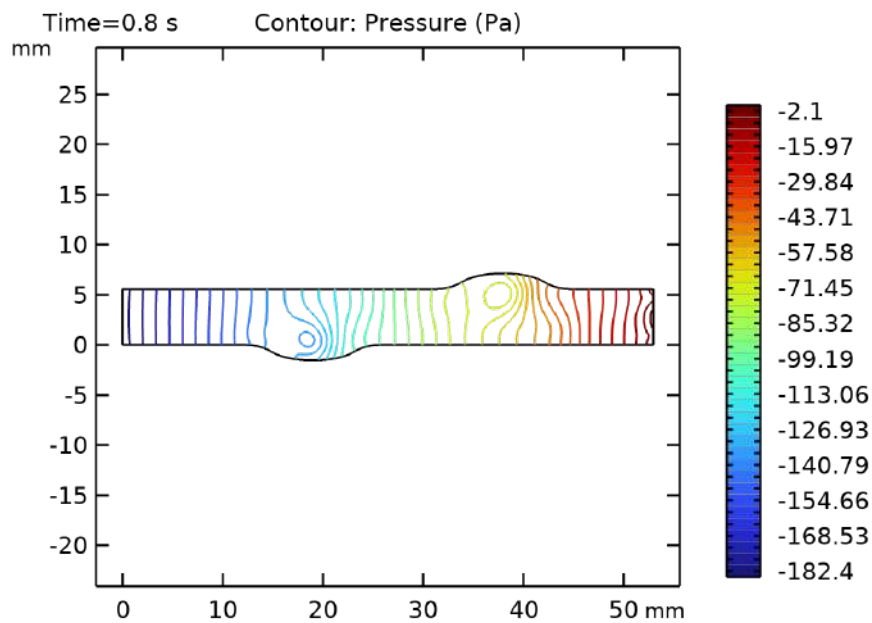
4.2. Plot Groups

4.2.1. Velocity (spf)



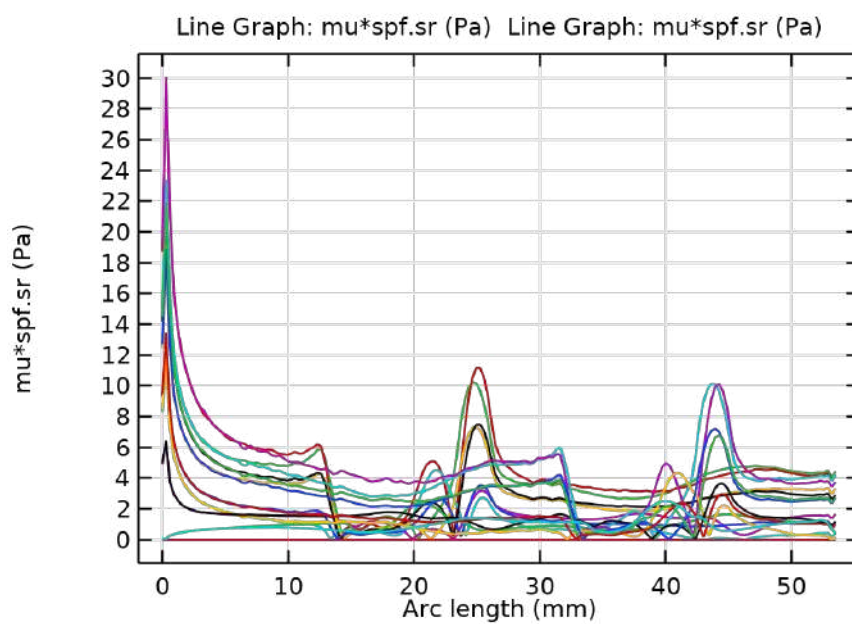
Surface: Velocity magnitude (m/s)

4.2.2. Pressure (spf)



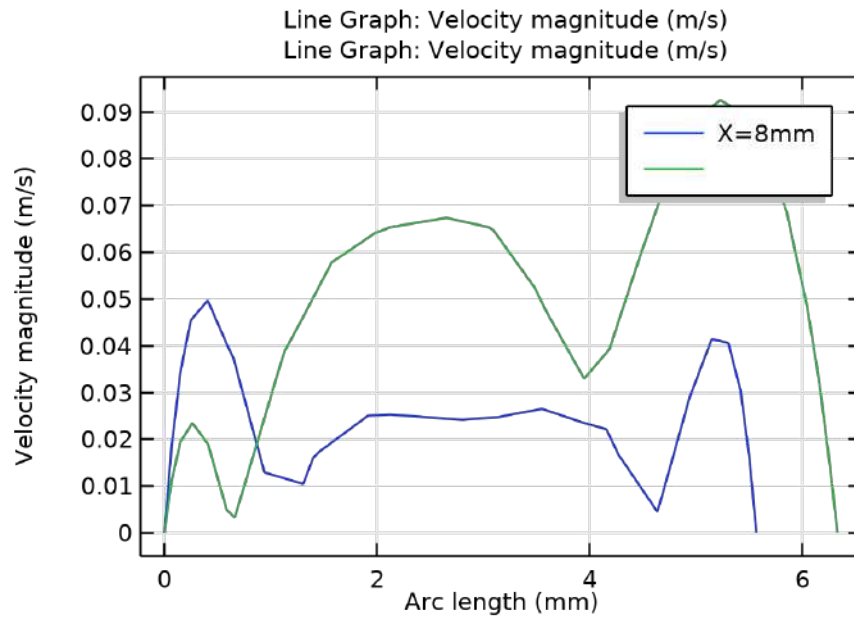
Contour: Pressure (Pa)

4.2.3. WSS



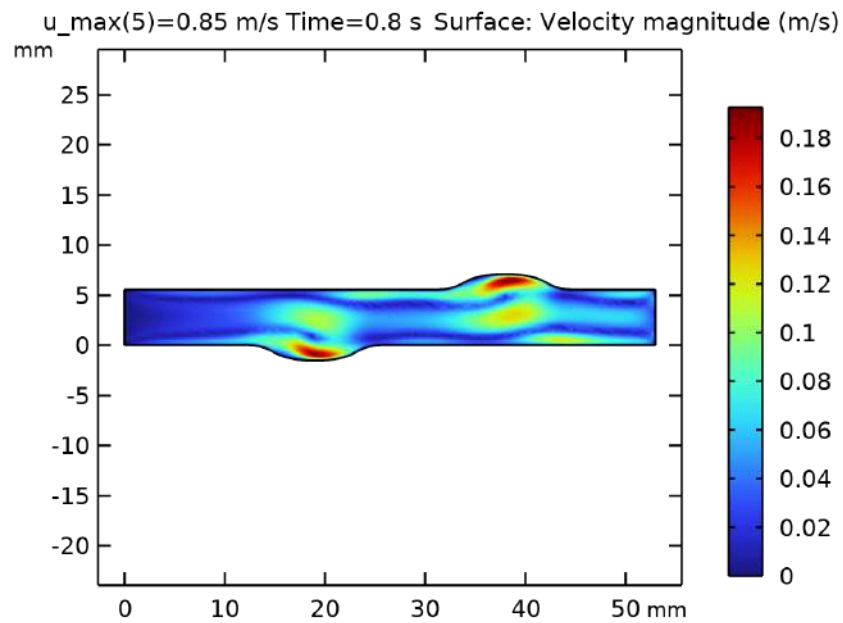
Line Graph: $\mu \cdot \text{spf.sr}$ (Pa) Line Graph: $\mu \cdot \text{spf.sr}$ (Pa)

4.2.4. Velocity Profiles



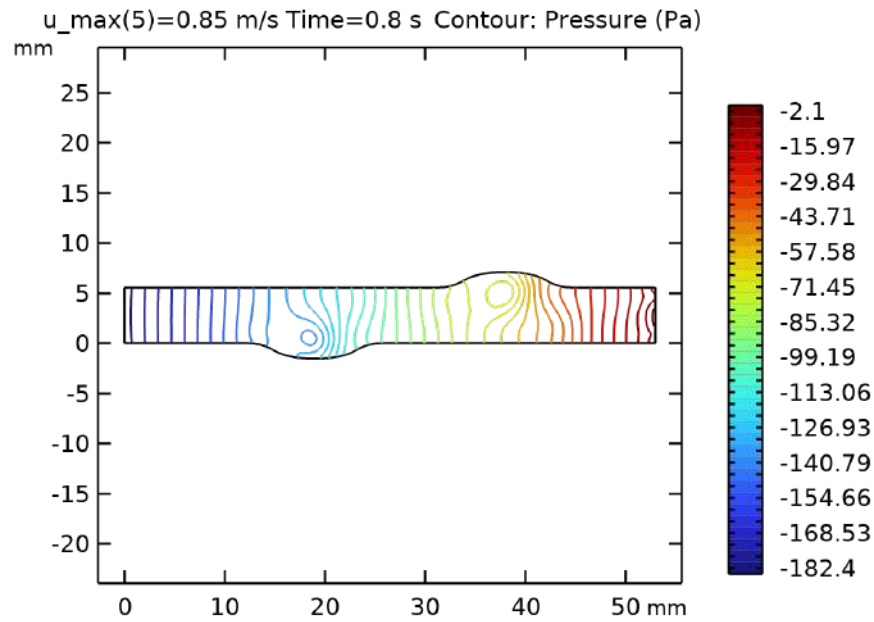
Line Graph: Velocity magnitude (m/s) Line Graph: Velocity magnitude (m/s)

4.2.5. Velocity (spf) 1



Surface: Velocity magnitude (m/s)

4.2.6. Pressure (spf) 1



Contour: Pressure (Pa)

A7Q3

Report date

Oct 21, 2025, 12:51:56 AM

Contents

1. [Global Definitions](#)
 - 1.1. [Parameters](#)
2. [Component 1](#)
 - 2.1. [Definitions](#)
 - 2.2. [Geometry 1](#)
 - 2.3. [Transport of Diluted Species](#)
 - 2.4. [Mesh 1](#)
3. [Study 1](#)
 - 3.1. [Time Dependent](#)
4. [Results](#)
 - 4.1. [Datasets](#)
 - 4.2. [Derived Values](#)
 - 4.3. [Tables](#)
 - 4.4. [Plot Groups](#)

1. Global Definitions

Date	Sep 21, 2025, 8:34:01 AM
------	--------------------------

Global settings

Name	A7Q3.mph
Path	D:\4th Year\450\COMSOL\6.0\A7Q3\A7Q3.mph
Version	COMSOL Multiphysics 6.0 (Build: 318)

Used products

COMSOL Multiphysics

Computer information

CPU	Intel64 Family 6 Model 141 Stepping 1, 6 cores
Operating system	Windows 10

1.1. Parameters

Parameters 1

Name	Expression	Value	Description
c0	3 [mol/m^3]	3 mol/m ³	
c1	0 [mole/m^3]	0 mol/m ³	
a	1000	1000	
D	1e-5 [m^2 /s]	1E-5 m ² /s	
km	5 [m/s]	5 m/s	
R	0	0	

2. Component 1

2.1. Definitions

2.1.1. Coordinate Systems

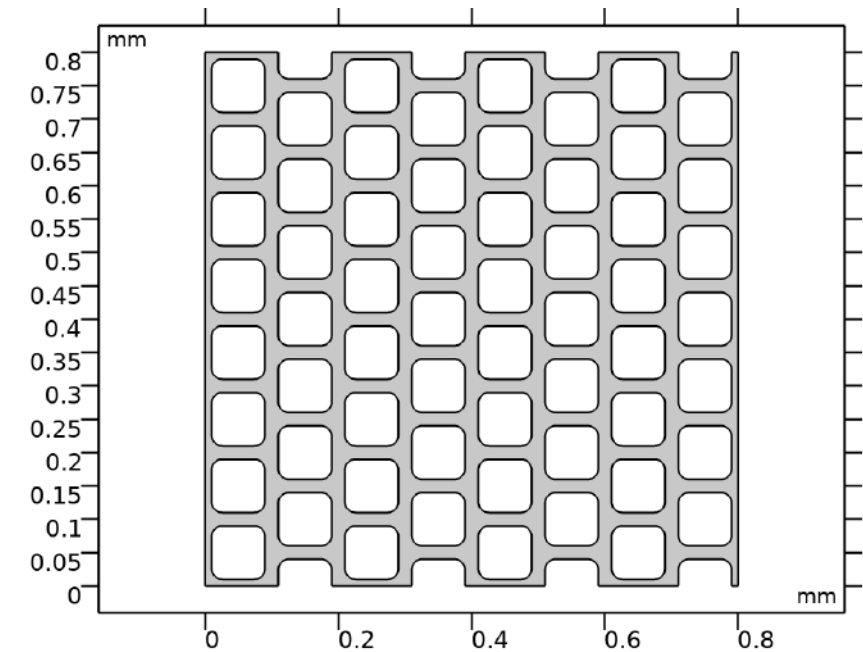
Boundary System 1

Coordinate system type	Boundary system
Tag	sys1

Coordinate names

First	Second	Third
t1	n	to

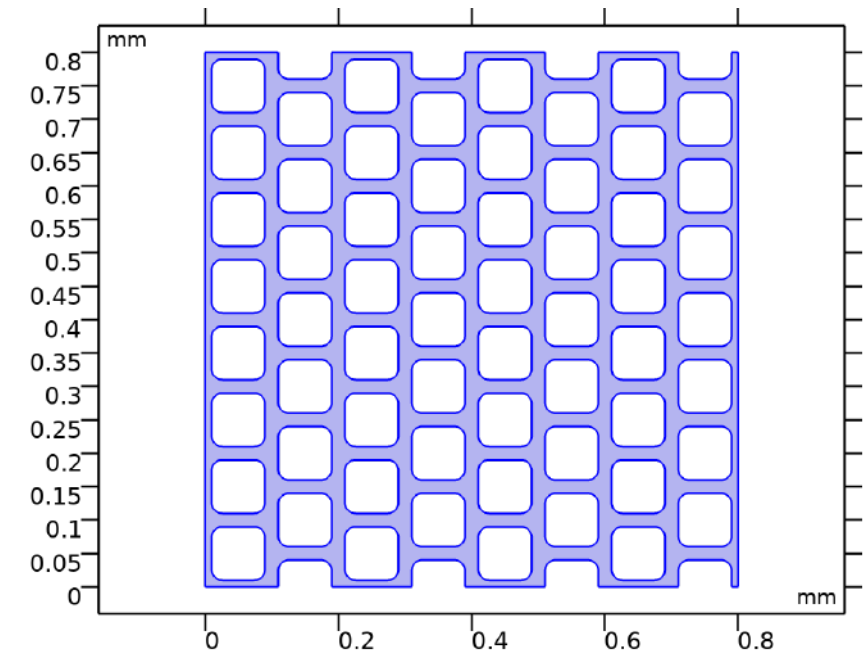
2.2. Geometry 1



Geometry 1

Units	
Length unit	mm
Angular unit	deg

2.3. Transport of Diluted Species



Transport of Diluted Species

Equations

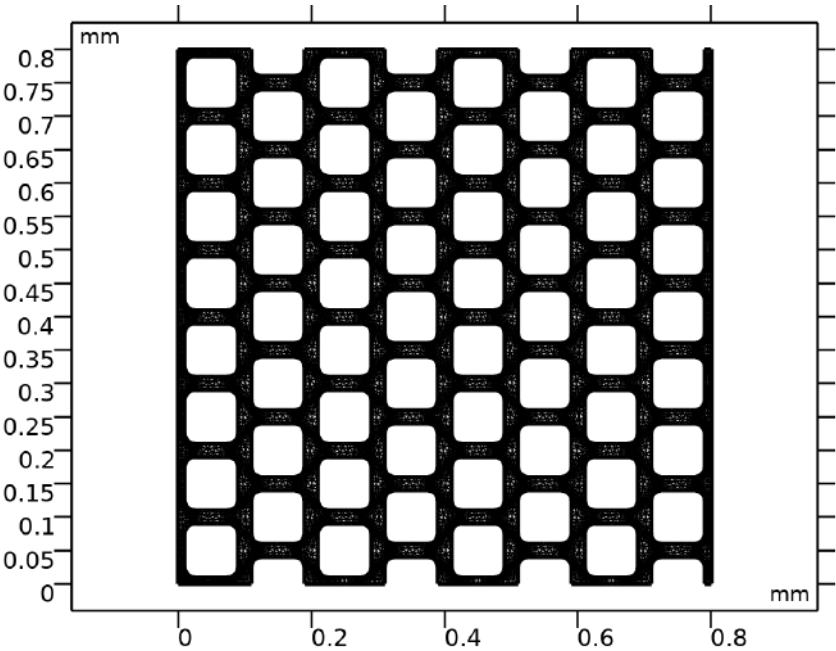
$$\frac{\partial c_i}{\partial t} + \nabla \cdot \mathbf{J}_i + \mathbf{u} \cdot \nabla c_i = R_i$$

$$\mathbf{J}_i = -D_i \nabla c_i$$

Features

Name	Level
Transport Properties 1	Domain
No Flux 1	Boundary
Initial Values 1	Domain
Concentration 1	Boundary
Flux 1	Boundary

2.4. Mesh 1



Mesh 1

3. Study 1

Computation
information

Computation time	3 s
------------------	-----

3.1. Time Dependent

Times	Unit
range(0,01,20)	s

Study settings

Description	Value
Include geometric nonlinearity	Off

Study settings

Description	Value
Output times	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}

Physics and variables selection

Physics interface	Discretization
Transport of Diluted Species (tds)	physics

Mesh selection

Geometry	Mesh
Geometry 1 (geom1)	mesh1

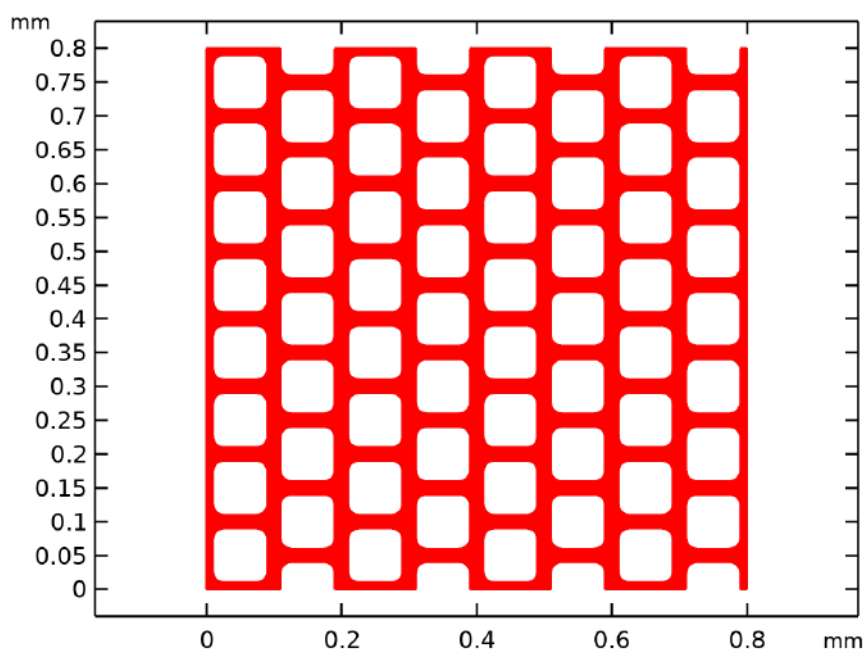
4. Results

4.1. Datasets

4.1.1. Study 1/Solution 1

Solution

Description	Value
Solution	Solution 1
Component	Component 1 (comp1)



Dataset: Study 1/Solution 1

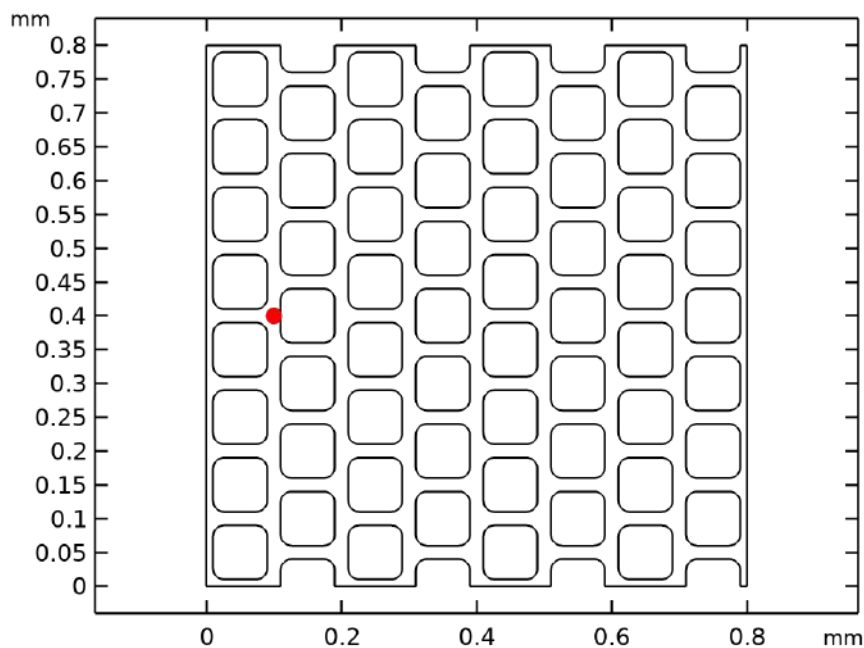
4.1.2. Cut Point 2D A

Data

Description	Value
Dataset	Study 1/Solution 1

Point data

Description	Value
Entry method	Coordinates
X	0.1
Y	0.4



Dataset: Cut Point 2D A

4.1.3. Cut Point 2D B

Data

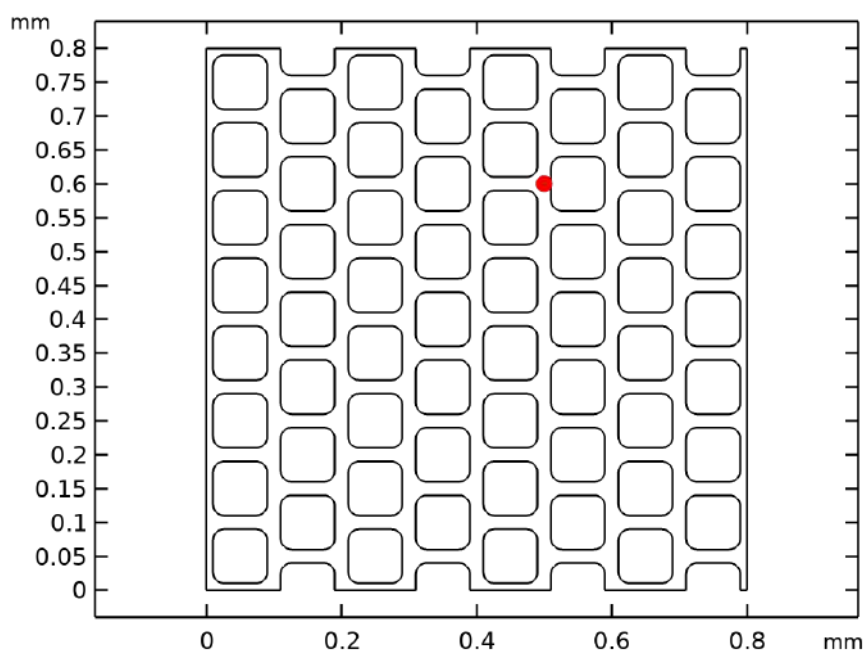
Description	Value
Dataset	Study 1/Solution 1

Point data

Description	Value
Entry method	Coordinates
X	0.5
Y	0.6

Advanced

Description	Value
Point number variable	cpt1n



Dataset: Cut Point 2D B

4.1.4. Cut Point 2D C

Data

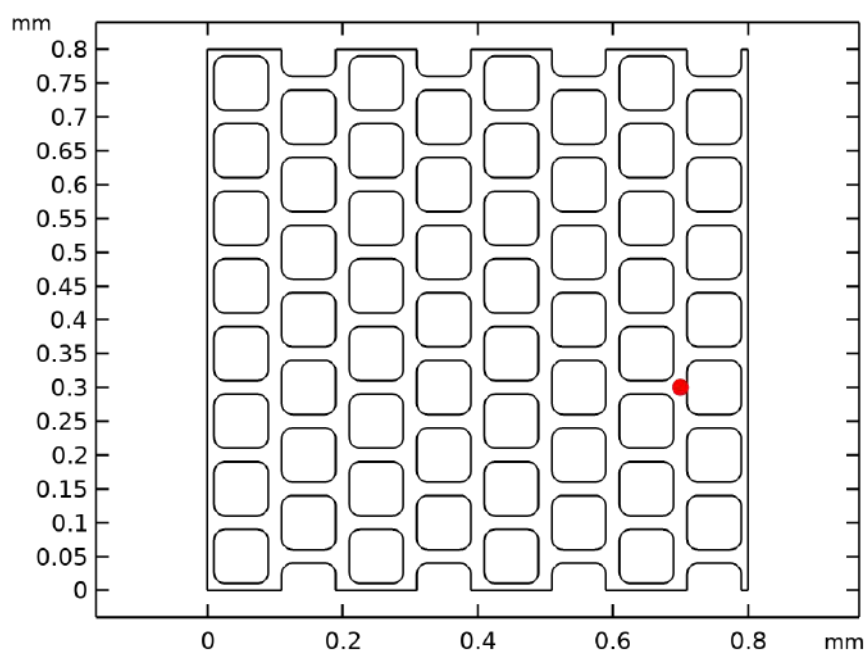
Description	Value
Dataset	Study 1/Solution 1

Point data

Description	Value
Entry method	Coordinates
X	0.7
Y	0.3

Advanced

Description	Value
Point number variable	cpt1n



Dataset: Cut Point 2D C

4.1.5. Cut Line 2D X=0.1

Data

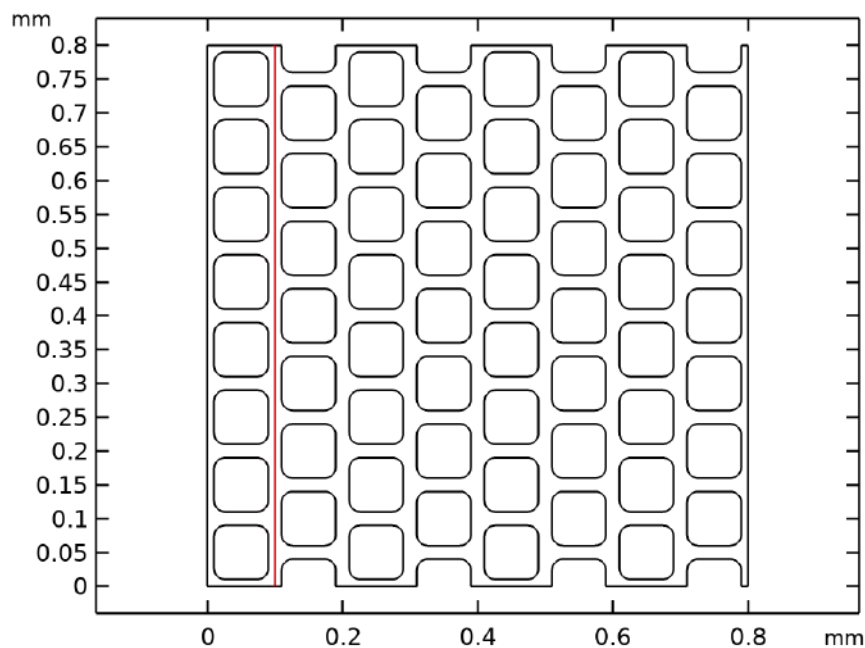
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.1, 0}, {0.1, 0.8}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D $X=0.1$

4.1.6. Cut Line 2D $X=0.2$

Data

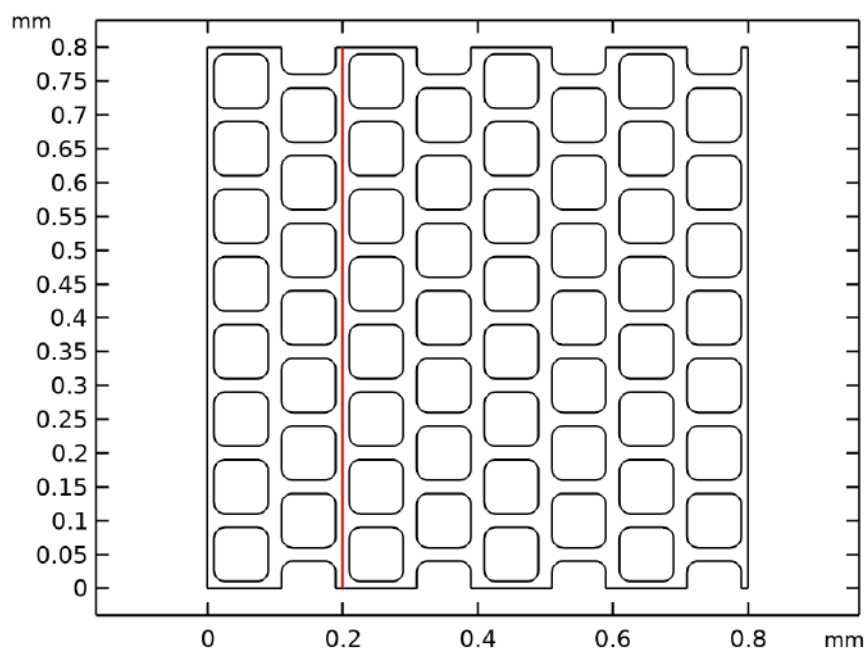
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.2, 0}, {0.2, 0.8}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D $X=0.2$

4.1.7. Cut Line 2D X=0.7

Data

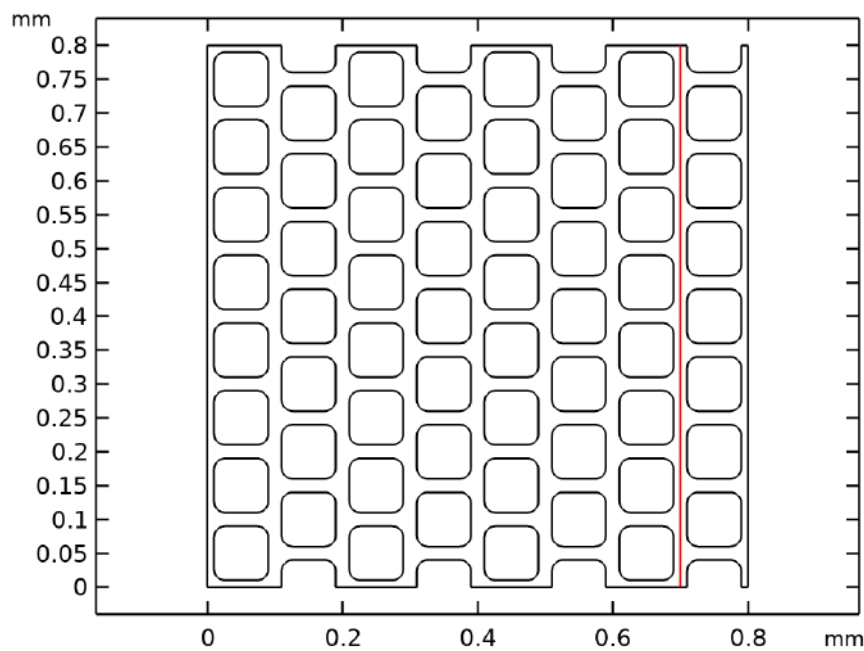
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.7, 0}, {0.7, 0.8}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D X=0.7

4.1.8. Cut Line 2D X=0.8

Data

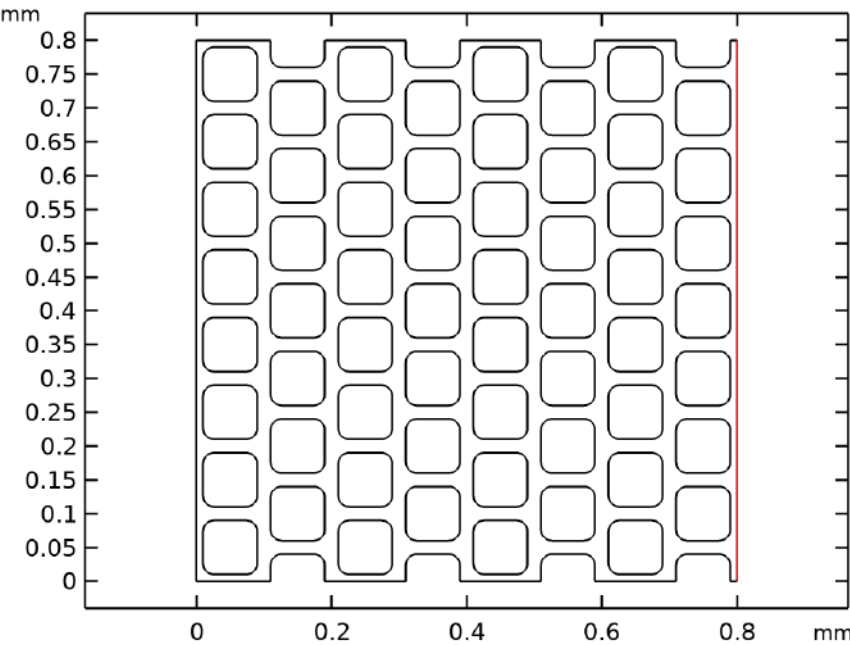
Description	Value
Dataset	Study 1/Solution 1

Line data

Description	Value
Line entry method	Two points
Points	{{0.8, 0}, {0.8, 0.8}}

Advanced

Description	Value
Space variable	cln1x
Normal variables	{cln1nx, cln1ny}
Tangent variables	{cln1tx, cln1ty}



Dataset: Cut Line 2D X=0.8

4.2. Derived Values

4.2.1. Point Evaluation A

Output		
Evaluated in Table 1		
Data		
Description	Value	
Dataset	Cut Point 2D A	
Expressions		
Expression	Unit	Description
c	mol/m^3	Concentration

4.2.2. Point Evaluation B

Output		
Evaluated in Table 1		
Data		
Description	Value	
Dataset	Cut Point 2D B	
Expressions		
Expression	Unit	Description
c	mol/m^3	Concentration

4.2.3. Point Evaluation C

Output		
Evaluated in Table 1		
Data		
Description	Value	
Dataset	Cut Point 2D C	
Expressions		
Expression	Unit	Description

c	mol/m ³	Concentration
---	--------------------	---------------

4.3. Tables

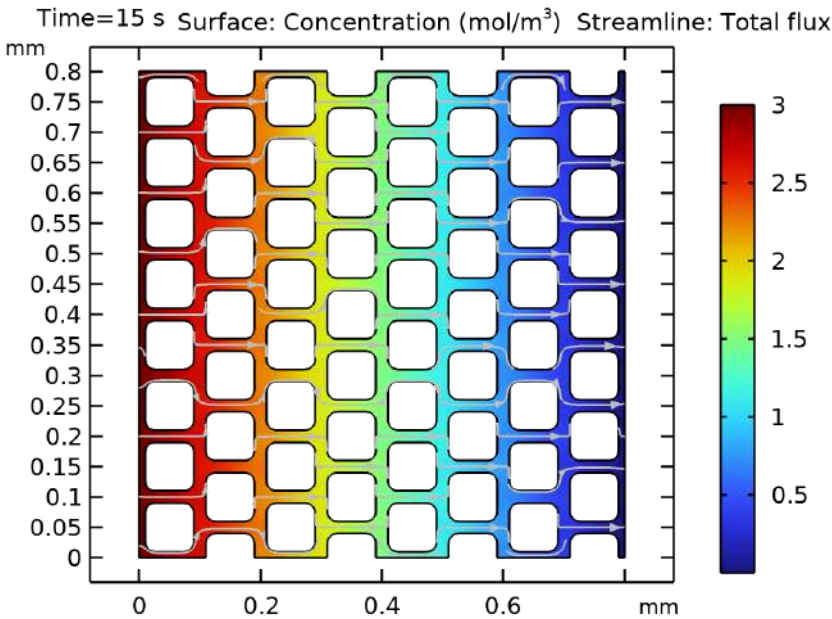
4.3.1. Table 1

Point Evaluation A

Time (s)	Concentration (mol/m ³), Point: (0.1, 0.4)	Concentration (mol/m ³), Point: (0.5, 0.6)	Concentration (mol/m ³), Point: (0.7, 0.3)
20.000	2.6660	1.1494	0.39094

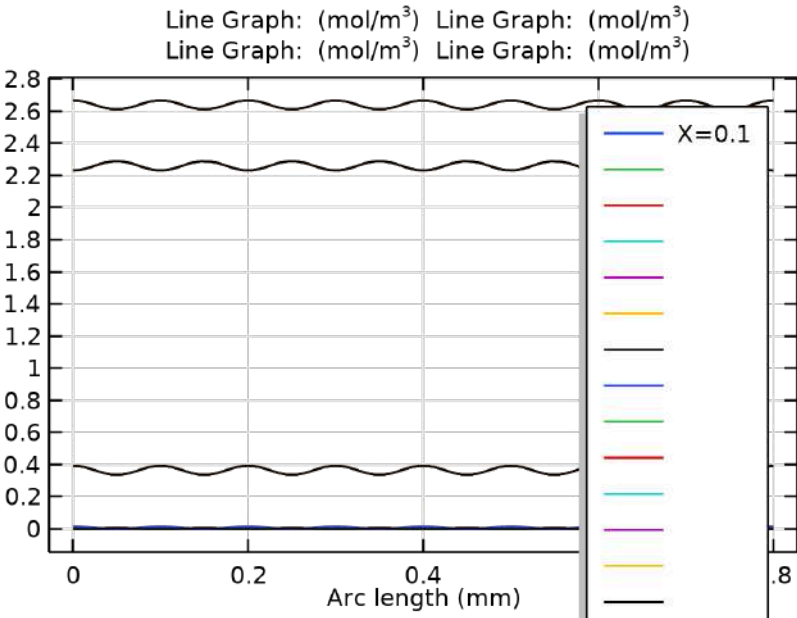
4.4. Plot Groups

4.4.1. Concentration (tds)



Surface: Concentration (mol/m³) Streamline: Total flux

4.4.2. 1D Plot Group 2



A7Q4

Report date

Oct 21, 2025, 12:59:45 AM

Contents

1. [Global Definitions](#)
 - 1.1. [Parameters](#)
2. [Component 1](#)
 - 2.1. [Definitions](#)
 - 2.2. [Geometry 1](#)
 - 2.3. [Laminar Flow](#)
 - 2.4. [Mesh 1](#)
3. [Study 1](#)
 - 3.1. [Parametric Sweep](#)
 - 3.2. [Stationary](#)
4. [Results](#)
 - 4.1. [Datasets](#)
 - 4.2. [Plot Groups](#)

1. Global Definitions

Date	Sep 21, 2025, 8:51:13 AM
------	--------------------------

Global settings

Name	A7Q4.mph
Path	D:\4th Year\450\COMSOL\6.0\A7Q4\A7Q4.mph
Version	COMSOL Multiphysics 6.0 (Build: 318)

Used products

COMSOL Multiphysics

Computer information

CPU	Intel64 Family 6 Model 141 Stepping 1, 6 cores
Operating system	Windows 10

1.1. Parameters

Parameters 1

Name	Expression	Value	Description
Re	1e3	1000	
mu	1/Re	0.001	

2. Component 1

2.1. Definitions

2.1.1. Coordinate Systems

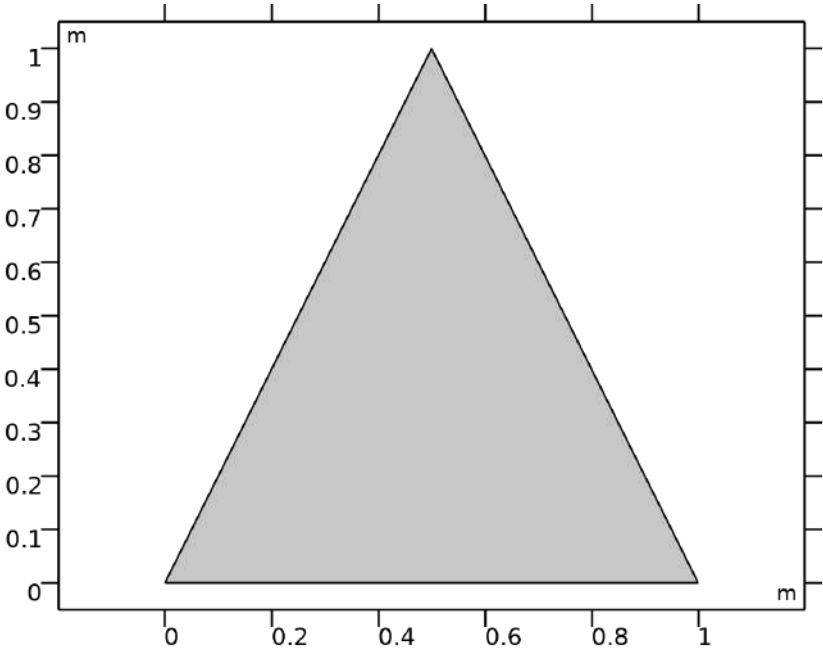
Boundary System 1

Coordinate system type	Boundary system
Tag	sys1

Coordinate names

First	Second	Third
t1	n	to

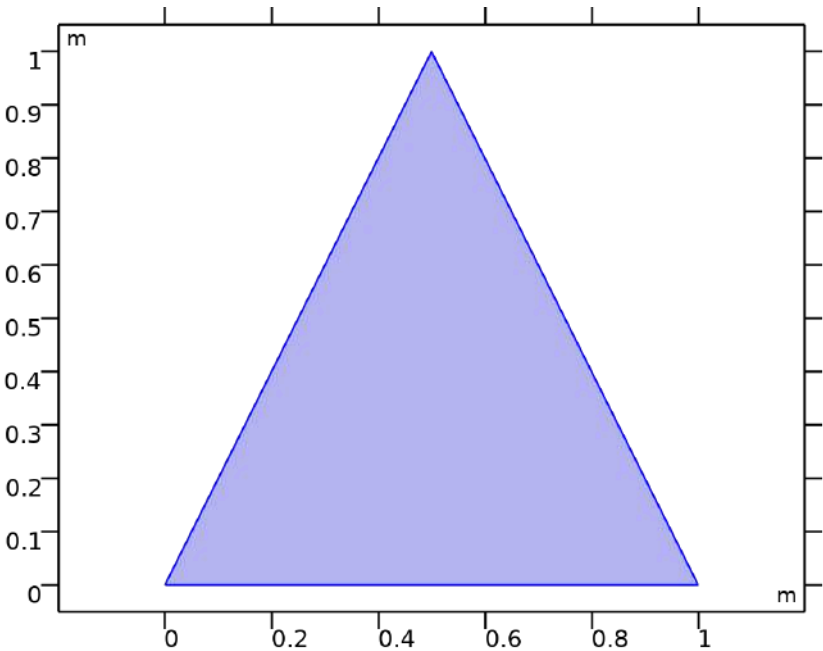
2.2. Geometry 1



Geometry 1

Units	
Length unit	m
Angular unit	deg

2.3. Laminar Flow



Laminar Flow

Equations

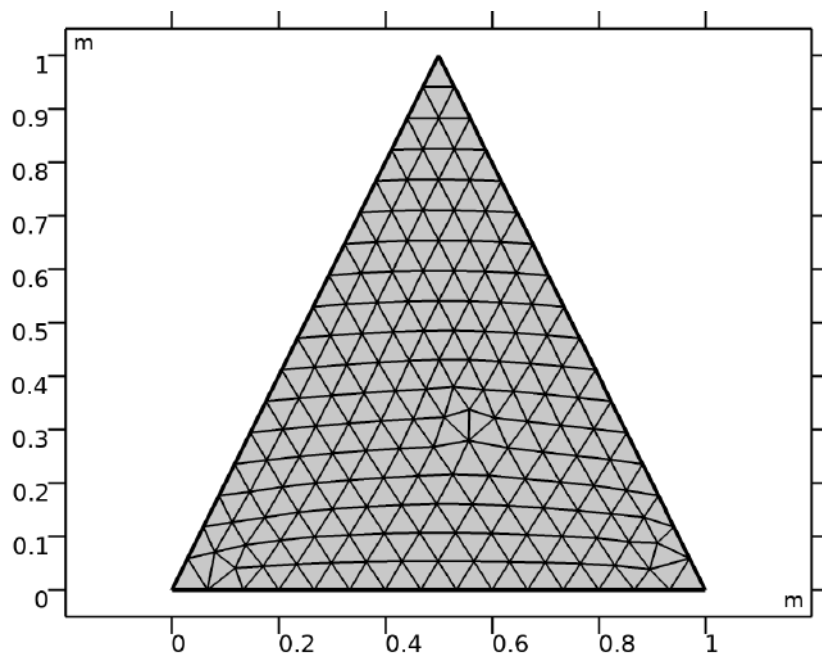
$$\rho(\mathbf{u} \cdot \nabla)\mathbf{u} = \nabla \cdot [-p\mathbf{I} + \mathbf{K}] + \mathbf{F}$$

$$\rho \nabla \cdot \mathbf{u} = 0$$

Features

Name	Level
Fluid Properties 1	Domain
Initial Values 1	Domain
Wall 1	Boundary
Pressure Point Constraint 1	Point

2.4. Mesh 1



Mesh 1

3. Study 1

Computation
information

Computation time	1 s
------------------	-----

3.1. Parametric Sweep

Parameter name	Parameter value list
Re	1e3 1e4 1e5 1e7

Study settings

Description	Value
Sweep type	Specified combinations
Parameter name	Re
Unit	

Parameters

Parameter name	Parameter value list	Parameter unit
Re	1e3 1e4 1e5 1e7	

3.2. Stationary

Study settings

Description	Value
Include geometric nonlinearity	Off

Physics and variables selection

Physics interface	Discretization
Laminar Flow (spf)	physics

Mesh selection

Geometry	Mesh
Geometry 1 (geom1)	mesh1

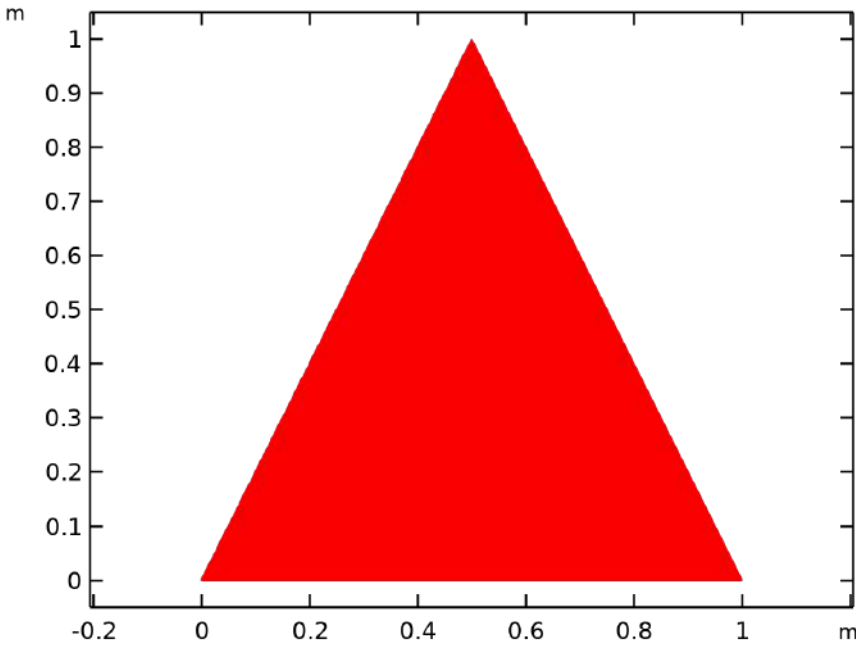
4. Results

4.1. Datasets

4.1.1. Study 1/Solution 1

Solution

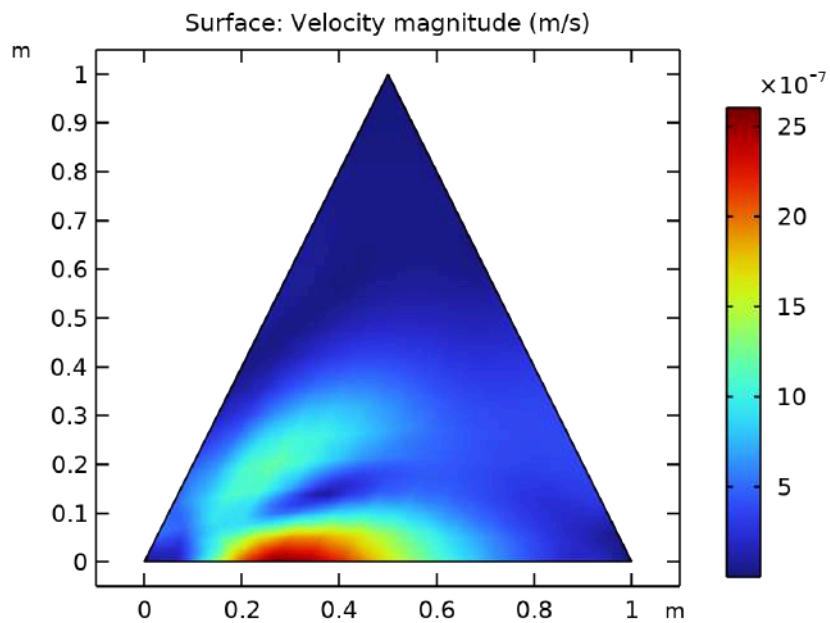
Description	Value
Solution	Solution 1
Component	Component 1 (comp1)



Dataset: Study 1/Solution 1

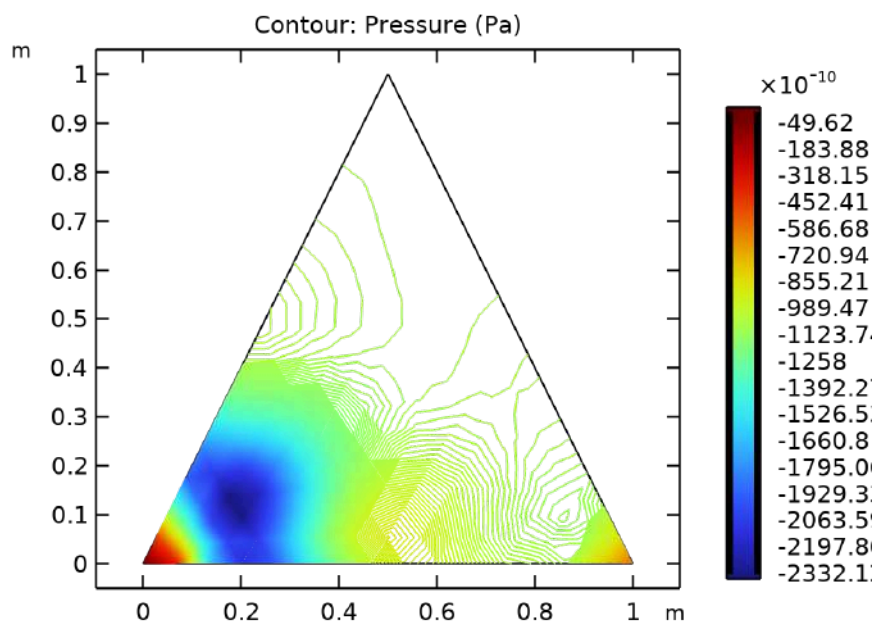
4.2. Plot Groups

4.2.1. Velocity (spf)



Surface: Velocity magnitude (m/s)

4.2.2. Pressure (spf)



Contour: Pressure (Pa)

A7Q5

Report date

Oct 21, 2025, 1:11:18 AM

Contents

1. [Global Definitions](#)
 - 1.1. [Parameters](#)
2. [Component 1](#)
 - 2.1. [Definitions](#)
 - 2.2. [Geometry 1](#)
 - 2.3. [Laminar Flow](#)
 - 2.4. [Heat Transfer in Fluids](#)
 - 2.5. [Mesh 1](#)
3. [Study 1](#)
 - 3.1. [Parametric Sweep](#)
 - 3.2. [Time Dependent](#)
4. [Study 2](#)
 - 4.1. [Parametric Sweep](#)
 - 4.2. [Time Dependent](#)
5. [Results](#)
 - 5.1. [Datasets](#)
 - 5.2. [Derived Values](#)
 - 5.3. [Tables](#)
 - 5.4. [No slip](#)
 - 5.5. [Free slip](#)

1. Global Definitions

Date	Sep 20, 2025, 5:53:08 PM
------	--------------------------

Global settings

Name	A7Q5 V2.mph
Path	D:\4th Year\450\COMSOL\6.0\A7Q5\A7Q5_V2.mph
Version	COMSOL Multiphysics 6.0 (Build: 318)

Used products

COMSOL Multiphysics

Computer information

CPU	Intel64 Family 6 Model 141 Stepping 1, 6 cores
Operating system	Windows 10

1.1. Parameters

Parameters 1

Name	Expression	Value	Description
Re	10^2	100	
Gr	10^3	1000	
Pr	6.9	6.9	

2. Component 1

2.1. Definitions

2.1.1. Coordinate Systems

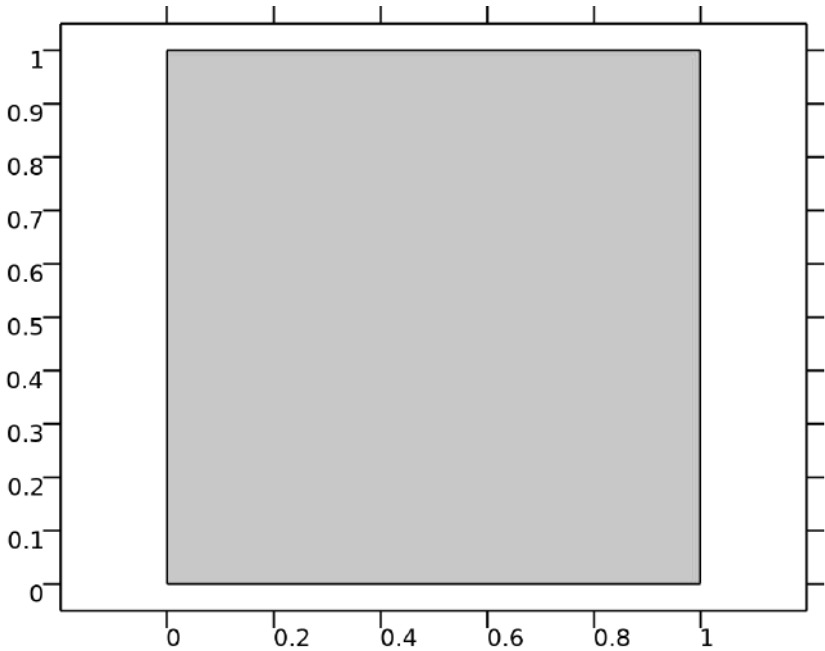
Boundary System 1

Coordinate system type	Boundary system
Tag	sys1

Coordinate names

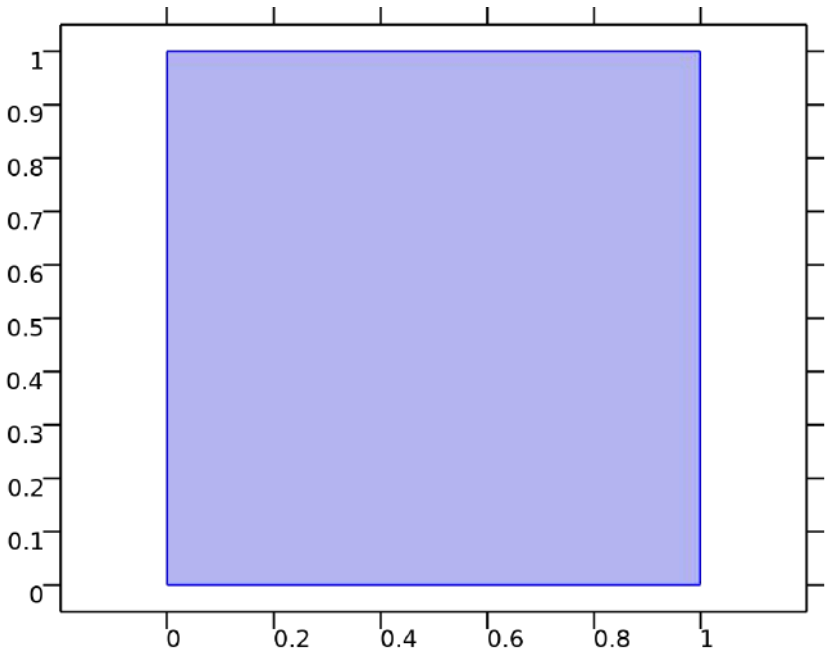
First	Second	Third
t1	n	to

2.2. Geometry 1



Geometry 1

2.3. Laminar Flow



Laminar Flow

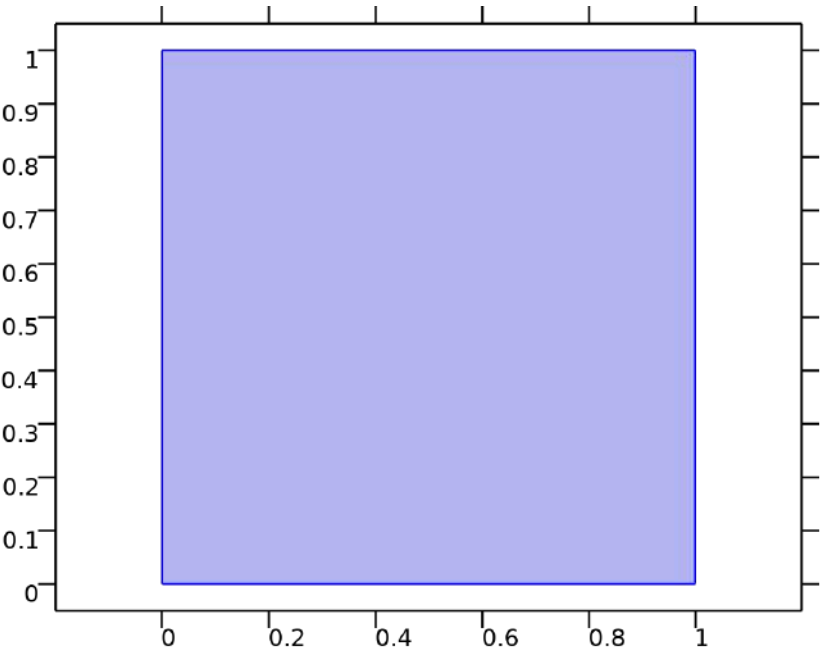
Equations

$$\rho \frac{\partial \mathbf{u}}{\partial t} + \rho(\mathbf{u} \cdot \nabla)\mathbf{u} = \nabla \cdot [-p\mathbf{I} + \mathbf{K}] + \mathbf{F}$$
$$\rho \nabla \cdot \mathbf{u} = 0$$

Features

Name	Level
Fluid Properties 1	Domain
Initial Values 1	Domain
Wall 1	Boundary
Volume Force 1	Domain
Pressure Point Constraint 1	Point

2.4. Heat Transfer in Fluids



Heat Transfer in Fluids

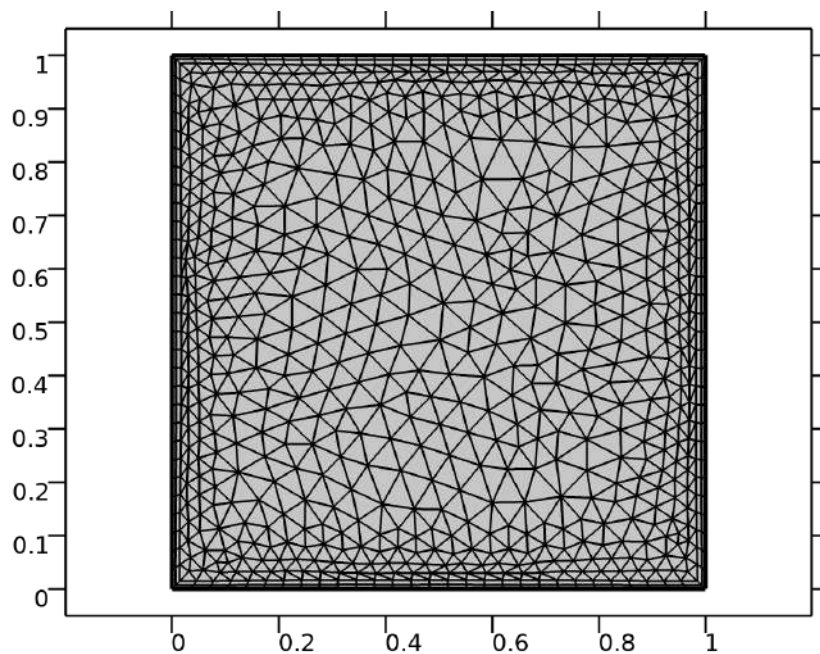
Equations

$$d_z \rho C_p \frac{\partial T}{\partial t} + d_z \rho C_p \mathbf{u} \cdot \nabla T + \nabla \cdot \mathbf{q} = d_z Q + q_0 + d_z Q_p + d_z Q_{vd}$$
$$\mathbf{q} = -d_z k \nabla T$$

Features

Name	Level
Fluid 1	Domain
Initial Values 1	Domain
Thermal Insulation 1	Boundary
Temperature 1	Boundary
Temperature 2	Boundary

2.5. Mesh 1



Mesh 1

3. Study 1

Computation
information

Computation time	31 s
------------------	------

3.1. Parametric Sweep

Parameter name	Parameter value list
Re	1e3 1e4 1e5
Gr	1e2 1e4 1e6

Study settings

Description	Value
Sweep type	Specified combinations
Parameter name	{Re, Gr}
Unit	{, }

Parameters

Parameter name	Parameter value list	Parameter unit
Re	1e3 1e4 1e5	
Gr	1e2 1e4 1e6	

3.2. Time Dependent

Times	Unit
range(0,1,30)	s

Study settings

Description	Value
Include geometric nonlinearity	Off

Study settings

Description	Value
Output times	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30}

Physics and variables selection

Physics interface	Discretization
Laminar Flow (spf)	physics
Heat Transfer in Fluids (ht)	physics

Mesh selection

Geometry	Mesh
Geometry 1 (geom1)	mesh1

4. Study 2

Computation
information

Computation time	26 s
------------------	------

4.1. Parametric Sweep

Parameter name	Parameter value list
Re	1e3 1e4 1e5
Gr	1e2 1e4 1e6

Study settings

Description	Value
Sweep type	Specified combinations
Parameter name	{Re, Gr}
Unit	{, }

Parameters

Parameter name	Parameter value list	Parameter unit
Re	1e3 1e4 1e5	
Gr	1e2 1e4 1e6	

4.2. Time Dependent

Times	Unit
range(0,1,30)	s

Study settings

Description	Value
Include geometric nonlinearity	Off

Study settings

Description	Value
Output times	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30}

Physics and variables selection

Physics interface	Discretization
Laminar Flow (spf)	physics
Heat Transfer in Fluids (ht)	physics

Mesh selection

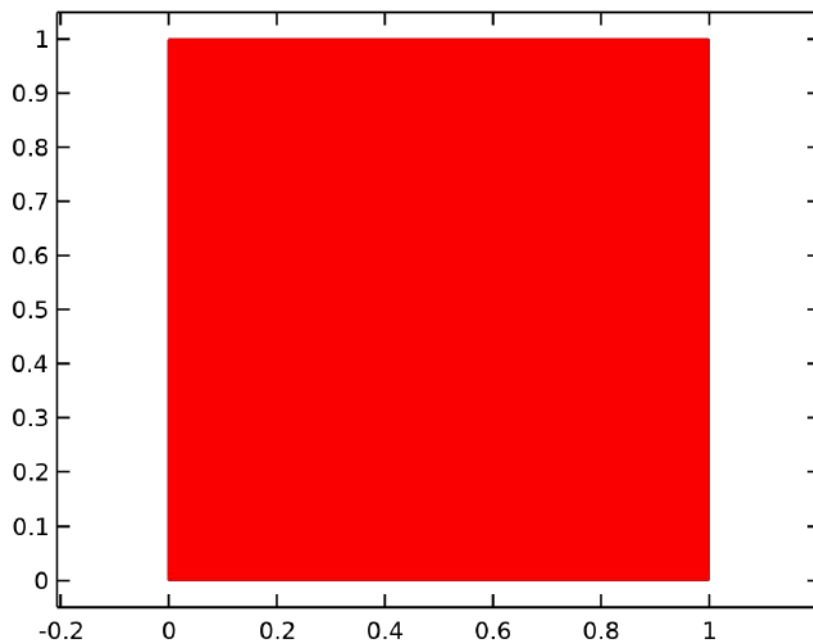
Geometry	Mesh
Geometry 1 (geom1)	mesh1

5. Results

5.1. Datasets

5.1.1. Study 1/Solution 1

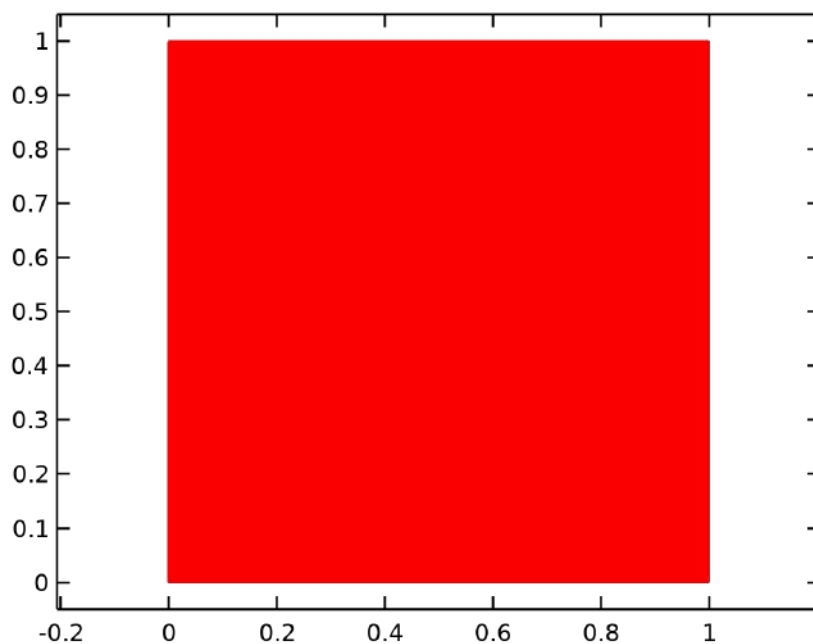
Solution	
Description	Value
Solution	Solution 1
Component	Component 1 (comp1)



Dataset: Study 1/Solution 1

5.1.2. Study 2/Solution 2

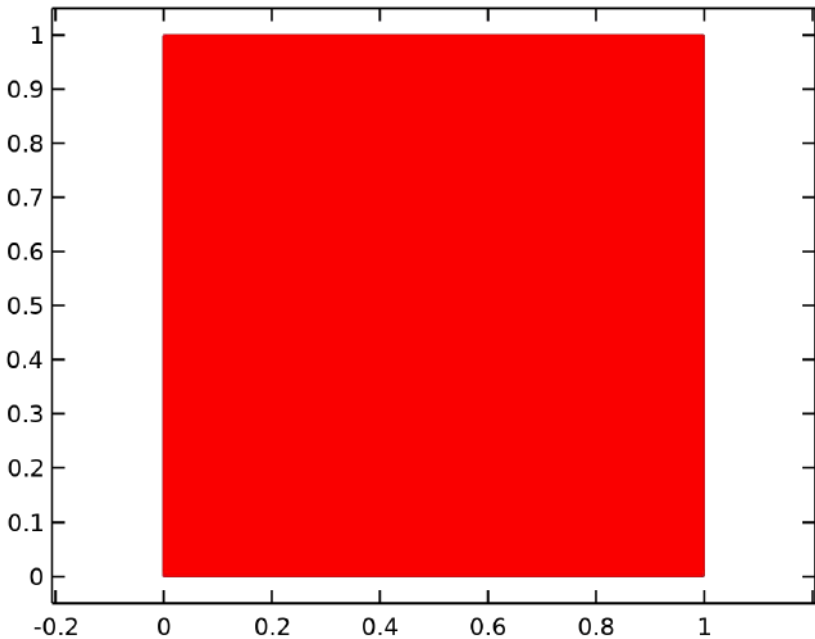
Solution	
Description	Value
Solution	Solution 2
Component	Component 1 (comp1)



Dataset: Study 2/Solution 2

5.1.3. Study 2/Parametric Solutions 1

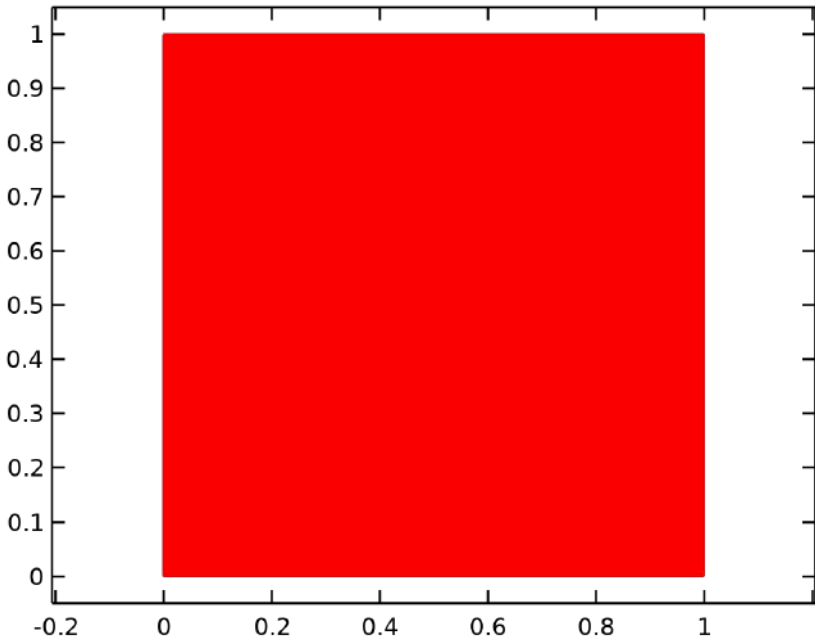
Solution	
Description	Value
Solution	Parametric Solutions 1
Component	Component 1 (comp1)



Dataset: Study 2/Parametric Solutions 1

5.1.4. Study 1/Parametric Solutions 2

Solution	
Description	Value
Solution	Parametric Solutions 2
Component	Component 1 (comp1)



Dataset: Study 1/Parametric Solutions 2

5.2. Derived Values

5.2.1. No slip Nusselt Number

Output

Evaluated in	Table 1
--------------	-------------------------

Data

Description	Value
Dataset	Study 1/Solution 1

Expressions

Expression	Unit	Description
$1 \cdot \text{ht.gradTy}/(1-0)$		Nusselt Number

Integration settings

Description	Value
Integration order	4

5.2.2. Free slip Nusselt Number

Output

Evaluated in	Table 2
--------------	-------------------------

Data

Description	Value
Dataset	Study 2/Solution 2

Expressions

Expression	Unit	Description
$1 \cdot \text{ht.gradTy}/(1-0)$		Nusselt Number

Integration settings

Description	Value
Integration order	4

5.3. Tables

5.3.1. Table 1

Line Integration 1

Time (s)	Nusselt Number
0.0000	-4.6961E-16
1.0000	-6.1628E-15
2.0000	-1.1349E-14
3.0000	-1.4088E-15
4.0000	-5.7126E-14
5.0000	-2.2428E-12
6.0000	-6.4630E-11
7.0000	-7.0683E-10
8.0000	-6.4695E-9
9.0000	-1.0280E-7
10.000	-5.3994E-7
11.000	-1.9948E-6
12.000	-6.3377E-6
13.000	-1.6076E-5
14.000	-3.5623E-5
15.000	-7.4622E-5

16.000	-1.4283E-4
17.000	-2.5284E-4
18.000	-4.2267E-4
19.000	-6.7997E-4
20.000	-0.0010407
21.000	-0.0015361
22.000	-0.0022255
23.000	-0.0030940
24.000	-0.0041758
25.000	-0.0055039
26.000	-0.0070541
27.000	-0.0088914
28.000	-0.011017
29.000	-0.013414
30.000	-0.016146

5.3.2. Table 2

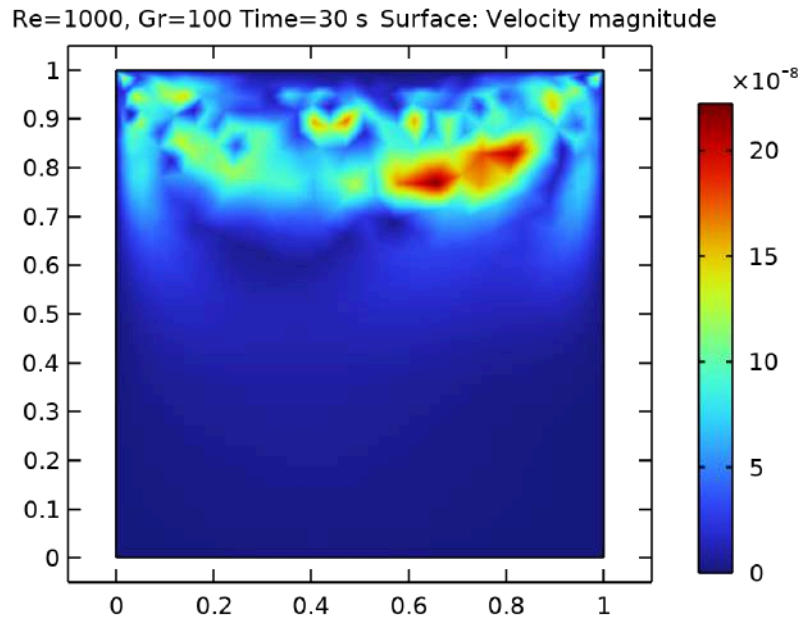
Line Integration 2

Time (s)	Nusselt Number
0.0000	7.3062E-13
1.0000	-4.7612E-13
2.0000	-3.0015E-13
3.0000	2.4768E-13
4.0000	-9.6693E-14
5.0000	-1.5151E-11
6.0000	-3.3835E-10
7.0000	-3.4615E-9
8.0000	-2.3401E-8
9.0000	-1.1856E-7
10.000	-1.0763E-6
11.000	-4.6314E-6
12.000	-1.2506E-5
13.000	-3.1737E-5
14.000	-6.1449E-5
15.000	-1.2448E-4
16.000	-2.1182E-4
17.000	-3.6750E-4
18.000	-5.7791E-4
19.000	-8.9310E-4
20.000	-0.0013129
21.000	-0.0018770
22.000	-0.0026068
23.000	-0.0035310
24.000	-0.0047036
25.000	-0.0060951
26.000	-0.0077469
27.000	-0.0096969

28.000	-0.011914
29.000	-0.014433
30.000	-0.017284

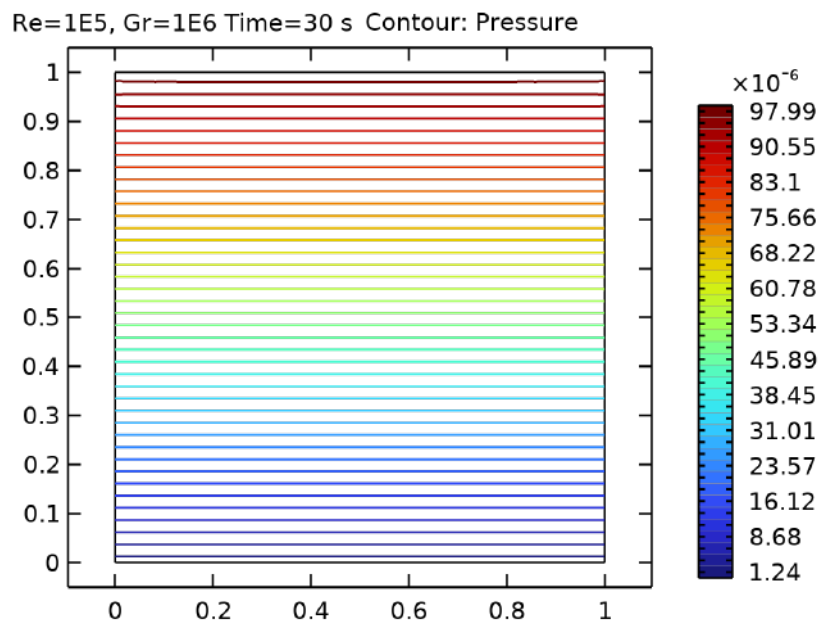
5.4. No slip

5.4.1. Velocity (spf) 3



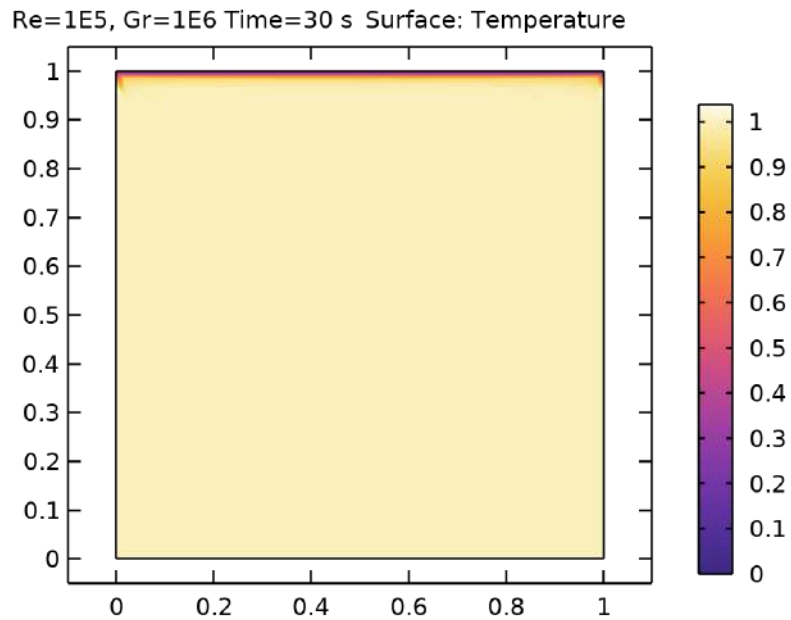
Surface: Velocity magnitude

5.4.2. Pressure (spf) 3



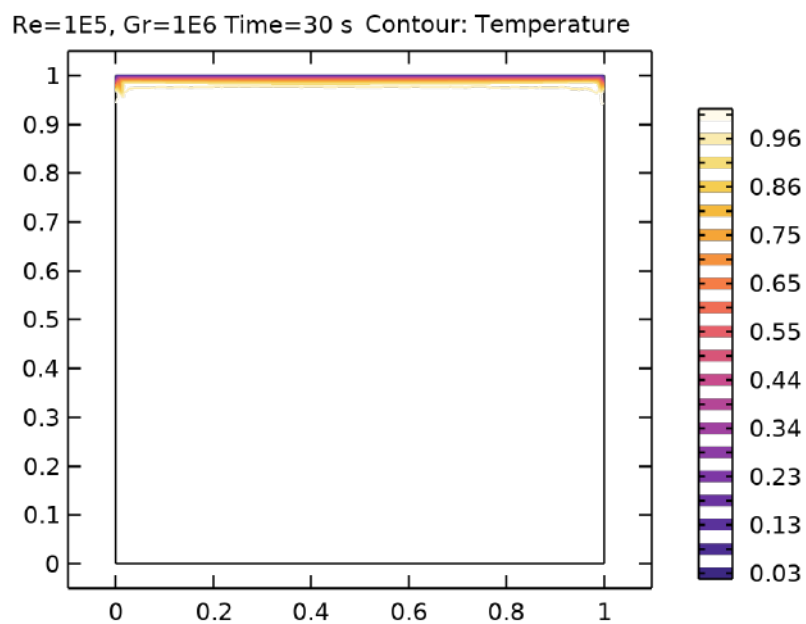
Contour: Pressure

5.4.3. Temperature (ht) 3



Surface: Temperature

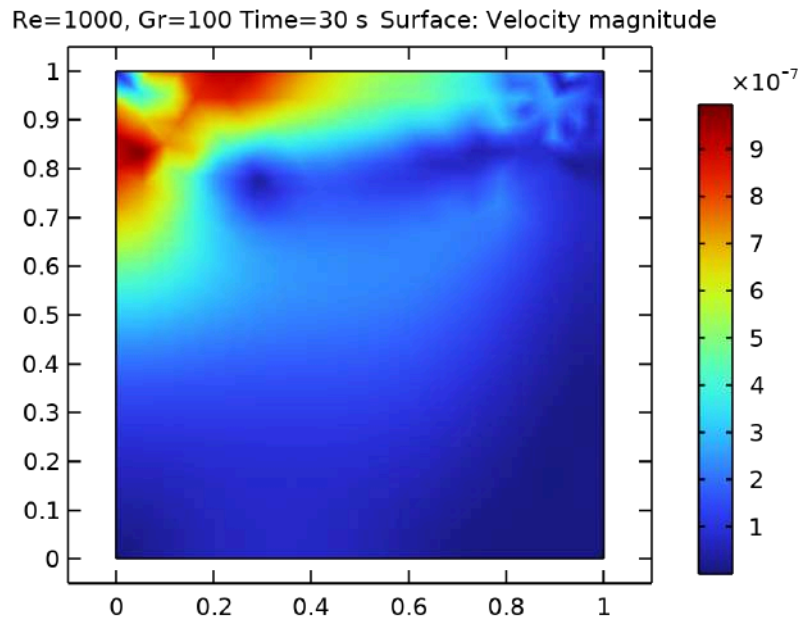
5.4.4. Isothermal Contours (ht) 3



Contour: Temperature

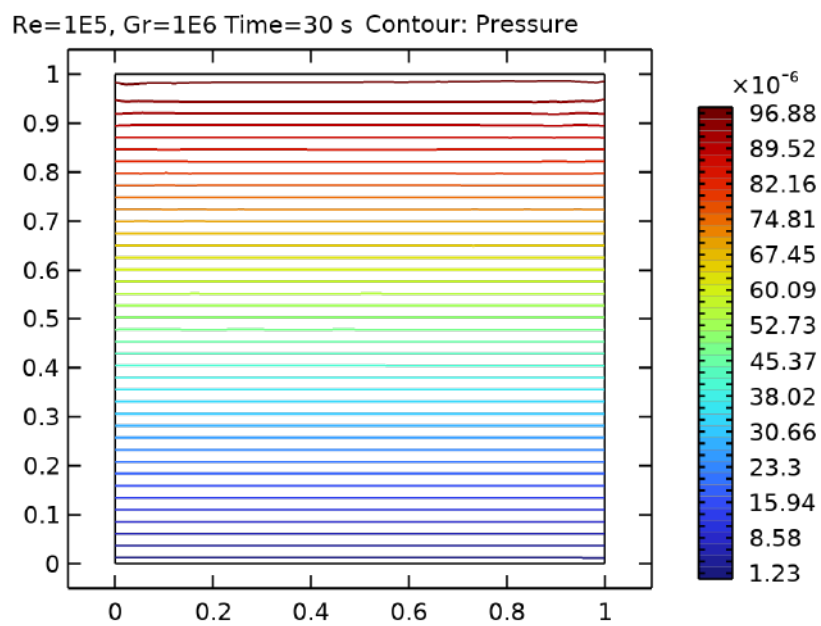
5.5. Free slip

5.5.1. Velocity (spf) 2



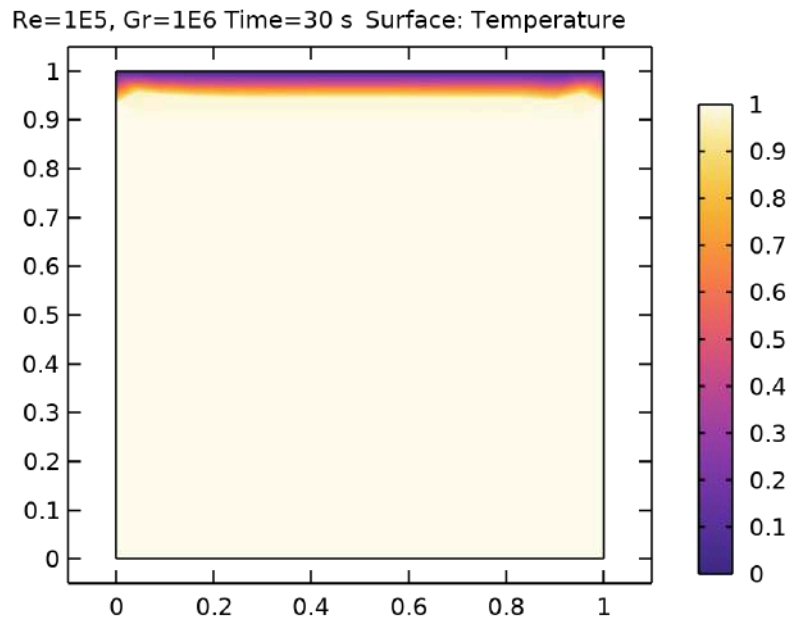
Surface: Velocity magnitude

5.5.2. Pressure (spf) 2



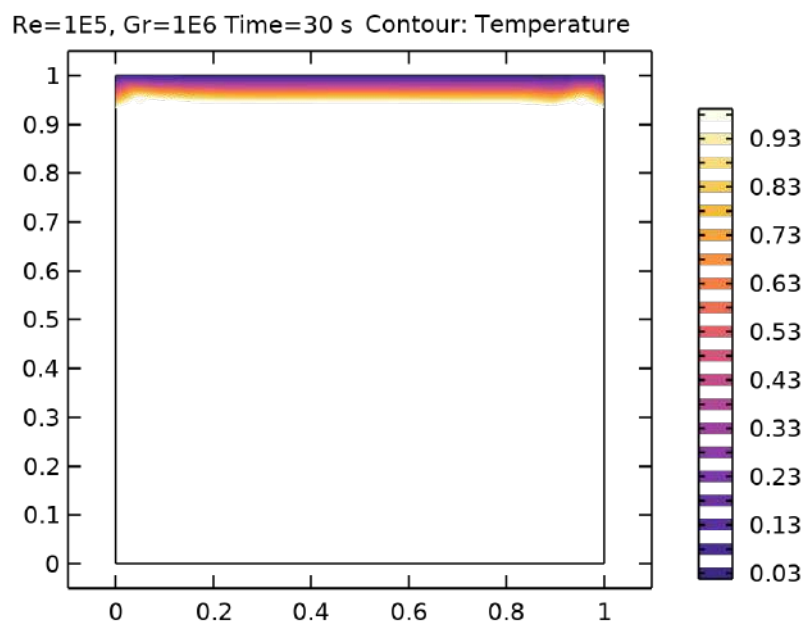
Contour: Pressure

5.5.3. Temperature (ht) 2



Surface: Temperature

5.5.4. Isothermal Contours (ht) 2



Contour: Temperature

A7Q6

Report date

Oct 21, 2025, 1:21:48 AM

Contents

1. [Global Definitions](#)
 - 1.1. [Parameters](#)
2. [Component 1](#)
 - 2.1. [Definitions](#)
 - 2.2. [Geometry 1](#)
 - 2.3. [Darcy's Law](#)
 - 2.4. [Transport of Diluted Species in Porous Media](#)
 - 2.5. [Mesh 1](#)
3. [Study 1](#)
 - 3.1. [Parametric Sweep](#)
 - 3.2. [Time Dependent](#)
4. [Results](#)
 - 4.1. [Datasets](#)
 - 4.2. [Plot Groups](#)

1. Global Definitions

Date	Oct 15, 2025, 1:29:42 AM
------	--------------------------

Global settings

Name	A7Q6 V2.mph
Path	D:\4th Year\450\COMSOL\6.0\A7Q6\A7Q6_V2.mph
Version	COMSOL Multiphysics 6.0 (Build: 318)

Used products

COMSOL Multiphysics
Battery Design Module

Computer information

CPU	Intel64 Family 6 Model 141 Stepping 1, 6 cores
Operating system	Windows 10

1.1. Parameters

Parameters 1

Name	Expression	Value	Description
L	0.08[m]	0.08 m	
H	0.02[m]	0.02 m	
R	2	2	
U	1[mm/s]	0.001 m/s	
mu1	1[mPa*s]	0.001 Pa·s	
rho	1000[kg/m^3]	1000 kg/m ³	
D	4e-8[m^2/s]	4E-8 m ² /s	
kappa	1e-6[m^2]	1E-6 m ²	
x0	0.01[m]	0.01 m	
delta	1e-4[m]	1E-4 m	
epsilon	0.1	0.1	

c2	10[mol/m ³]	10 mol/m ³	
----	-------------------------	-----------------------	--

2. Component 1

2.1. Definitions

2.1.1. Coordinate Systems

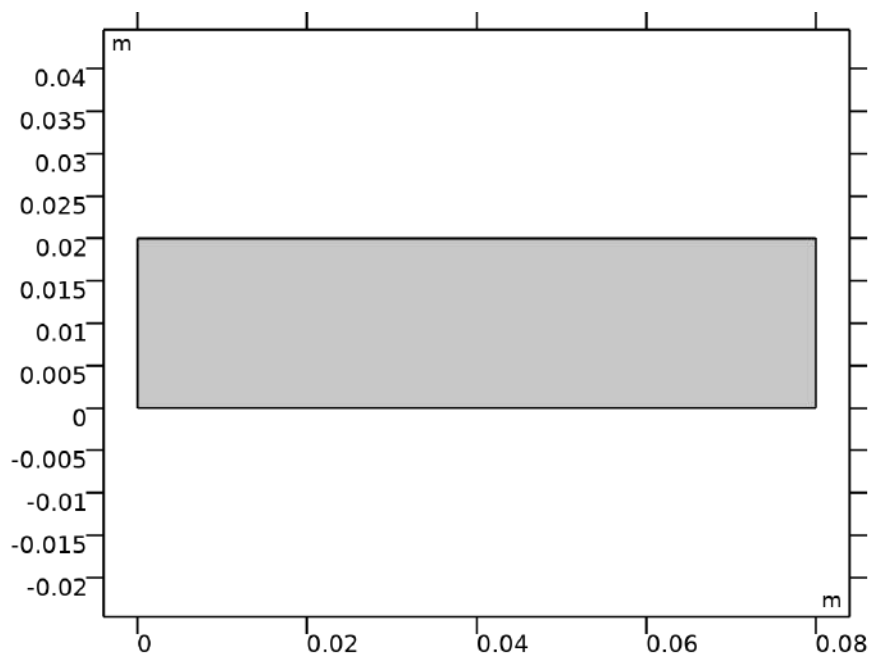
Boundary System 1

Coordinate system type	Boundary system
Tag	sys1

Coordinate names

First	Second	Third
t1	n	to

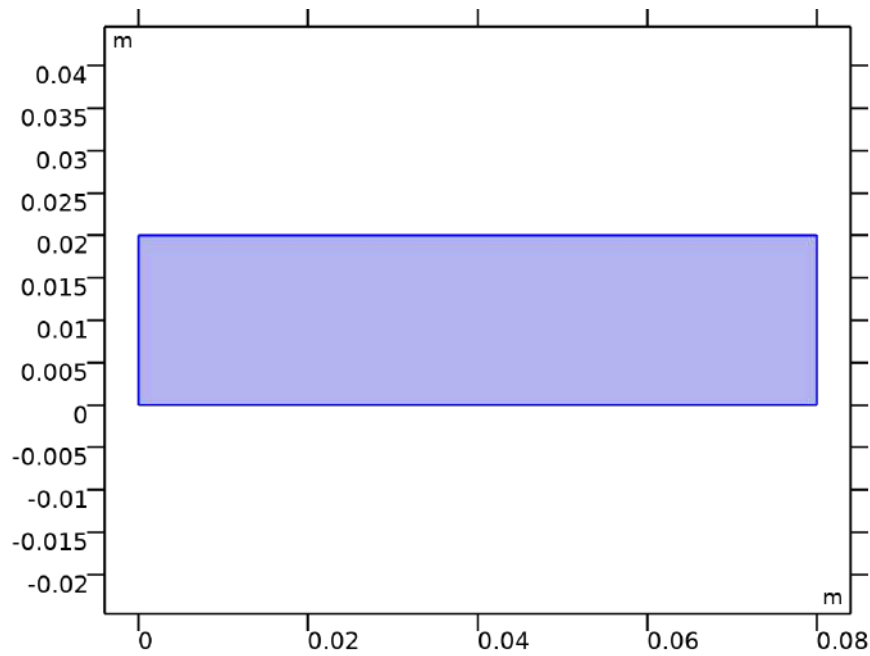
2.2. Geometry 1



Geometry 1

Units	
Length unit	m
Angular unit	deg

2.3. Darcy's Law



Darcy's Law

Equations

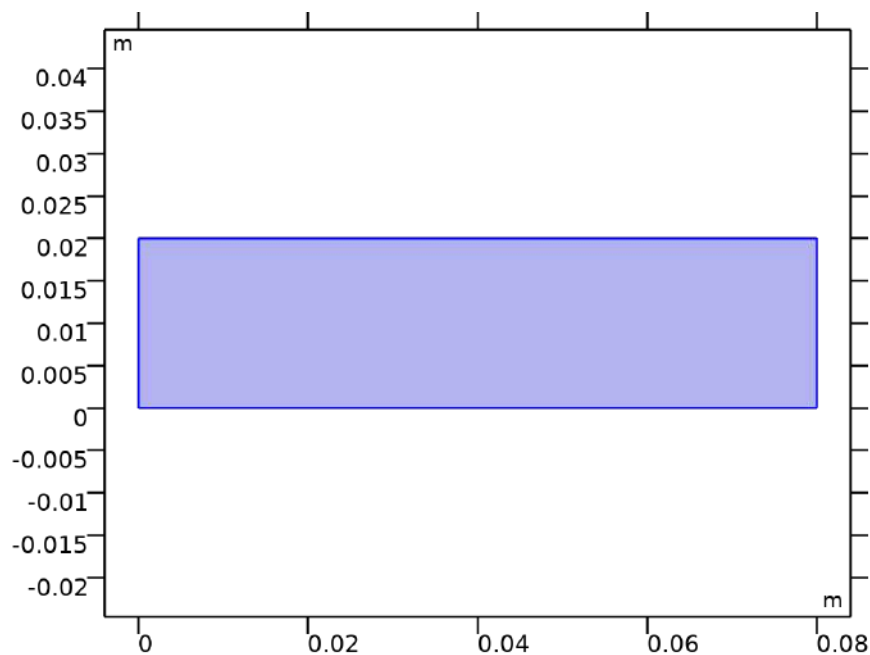
$$\frac{\partial}{\partial t}(\epsilon_p \rho) + \nabla \cdot (\rho \mathbf{u}) = Q_m$$

$$\mathbf{u} = -\frac{\kappa}{\mu} \nabla p$$

Features

Name	Level
Porous Medium 1	Domain
No Flow 1	Boundary
Initial Values 1	Domain
Inlet 1	Boundary
Outlet 1	Boundary

2.4. Transport of Diluted Species in Porous Media



Transport of Diluted Species in Porous Media

Equations

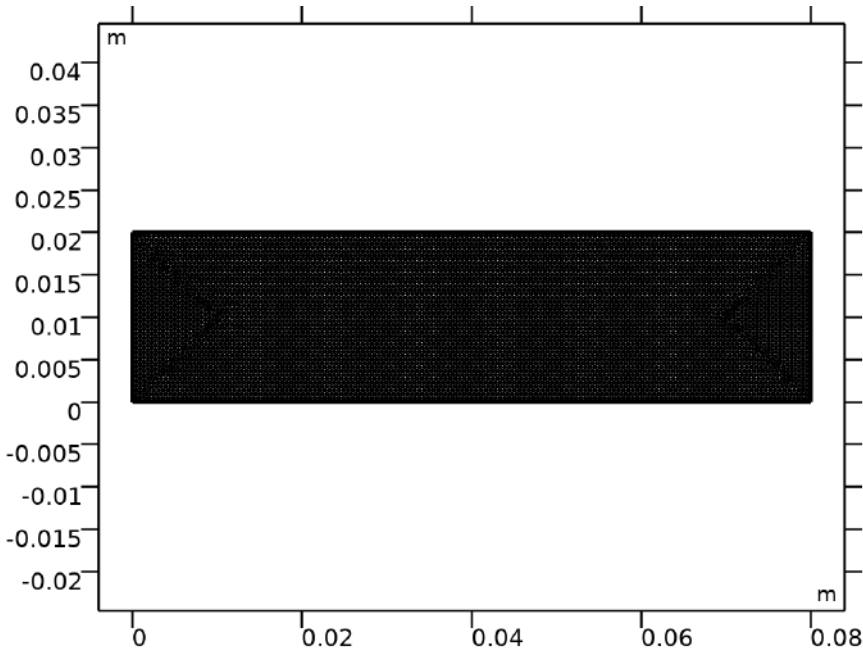
$$\frac{\partial(\epsilon_p c_i)}{\partial t} + \frac{\partial(\rho c_{p,i})}{\partial t} + \nabla \cdot \mathbf{J}_i + \mathbf{u} \cdot \nabla c_i = R_i + S_i$$

$$\mathbf{J}_i = -(D_{D,i} + D_{e,i}) \nabla c_i$$

Features

Name	Level
Porous Medium 1	Domain
No Flux 1	Boundary
Initial Values 1	Domain
Inflow 1	Boundary
Outflow 1	Boundary

2.5. Mesh 1



Mesh 1

3. Study 1

Computation information

Computation time	3 min 59 s
------------------	------------

3.1. Parametric Sweep

Parameter name	Parameter value list
epsilon	0.1 0.5 0.75

Study settings

Description	Value
Sweep type	Specified combinations
Parameter name	epsilon
Unit	

Parameters

Parameter name	Parameter value list	Parameter unit
epsilon	0.1 0.5 0.75	

3.2. Time Dependent

Times	Unit
range(0,0.1,15)	s

Study settings

Description	Value
Include geometric nonlinearity	Off

Study settings

Description	Value
Output times	{0, 0.1, 0.2, 0.30000000000000004, 0.4, 0.5, 0.6000000000000001, 0.7000000000000001, 0.8, 0.9, 1, 1.1, 1.2000000000000002, 1.3, 1.4000000000000001, 1.5, 1.6, 1.7000000000000002, 1.8, 1.9000000000000001, 2, 2.1, 2.2, 2.3000000000000003, 2.4000000000000004, 2.5, 2.6, 2.7, 2.8000000000000003, 2.9000000000000004, 3, 3.1, 3.2, 3.3000000000000003, 3.4000000000000004, 3.5, 3.6, 3.7, 3.8000000000000003, 3.9000000000000004, 4, 4.1000000000000005, 4.2, 4.3, 4.4, 4.5, 4.6000000000000005, 4.7, 4.8000000000000001, 4.9, 5, 5.1000000000000005, 5.2, 5.3000000000000001, 5.4, 5.5, 5.6000000000000005, 5.7, 5.8000000000000001, 5.9, 6, 6.1000000000000005, 6.2, 6.3000000000000001, 6.4, 6.5, 6.6000000000000005, 6.7, 6.8000000000000001, 6.9, 7, 7.1000000000000005, 7.2, 7.3000000000000001, 7.4, 7.5, 7.6000000000000005, 7.7, 7.8000000000000001, 7.9, 8, 8.1, 8.2000000000000001, 8.3, 8.4, 8.5, 8.6, 8.7000000000000001, 8.8, 8.9, 9, 9.1, 9.2000000000000001, 9.3, 9.4, 9.5, 9.6000000000000001, 9.7000000000000001, 9.8, 9.9, 10, 10.1000000000000001, 10.2000000000000001, 10.3, 10.4, 10.5, 10.6000000000000001, 10.7000000000000001, 10.8, 10.9, 11, 11.1000000000000001, 11.2000000000000001, 11.3, 11.4, 11.5, 11.6000000000000001, 11.7000000000000001, 11.8, 11.9, 12, 12.1000000000000001, 12.2000000000000001, 12.3, 12.4, 12.5, 12.6000000000000001, 12.7000000000000001, 12.8, 12.9, 13, 13.1000000000000001, 13.2000000000000001, 13.3, 13.4, 13.5, 13.6000000000000001, 13.7000000000000001, 13.8, 13.9, 14, 14.1000000000000001, 14.2000000000000001, 14.3, 14.4, 14.5, 14.6000000000000001, 14.7000000000000001, 14.8, 14.9, 15}

Physics and variables selection

Physics interface	Discretization
Darcy's Law (dl)	physics
Transport of Diluted Species in Porous Media (tds)	physics

Mesh selection

Geometry	Mesh
Geometry 1 (geom1)	mesh1

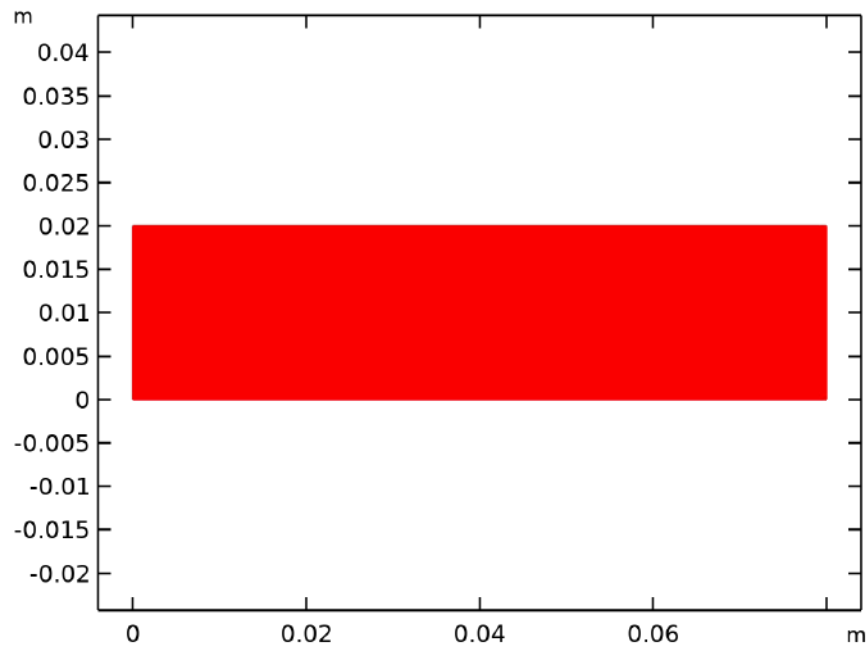
4. Results

4.1. Datasets

4.1.1. Study 1/Solution 1

Solution

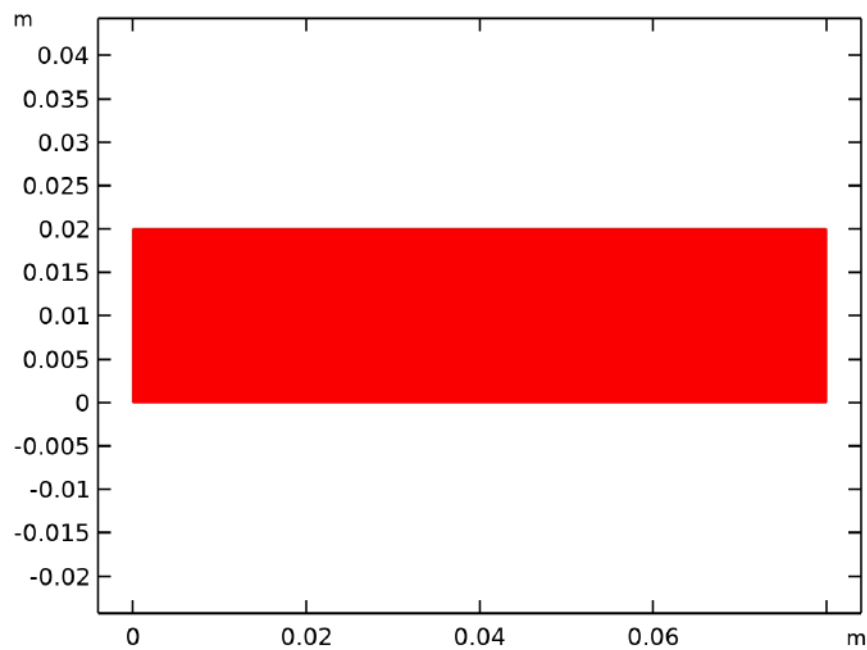
Description	Value
Solution	Solution 1
Component	Component 1 (comp1)



Dataset: Study 1/Solution 1

4.1.2. Study 1/Parametric Solutions 1

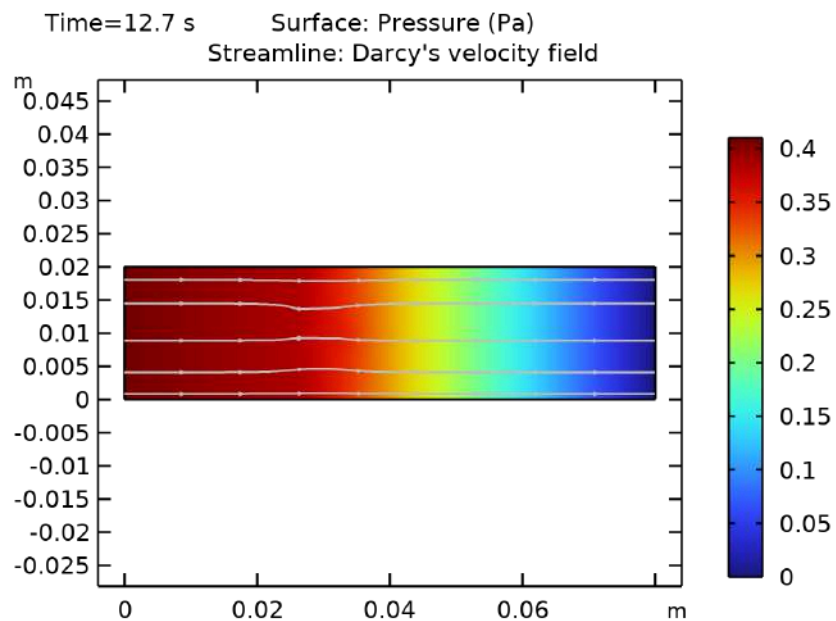
Solution	
Description	Value
Solution	Parametric Solutions 1
Component	Component 1 (comp1)



Dataset: Study 1/Parametric Solutions 1

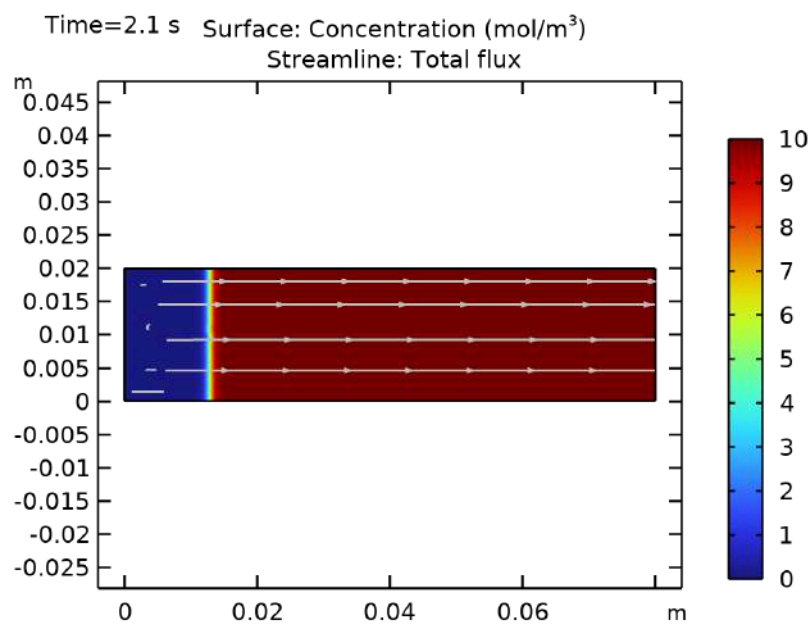
4.2. Plot Groups

4.2.1. Pressure (dl)



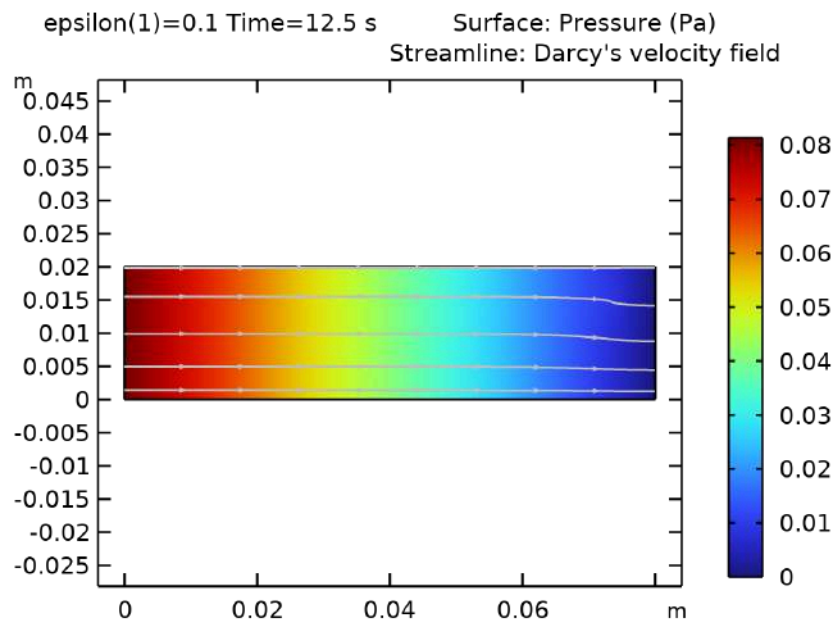
Surface: Pressure (Pa) Streamline: Darcy's velocity field

4.2.2. Concentration (tds)



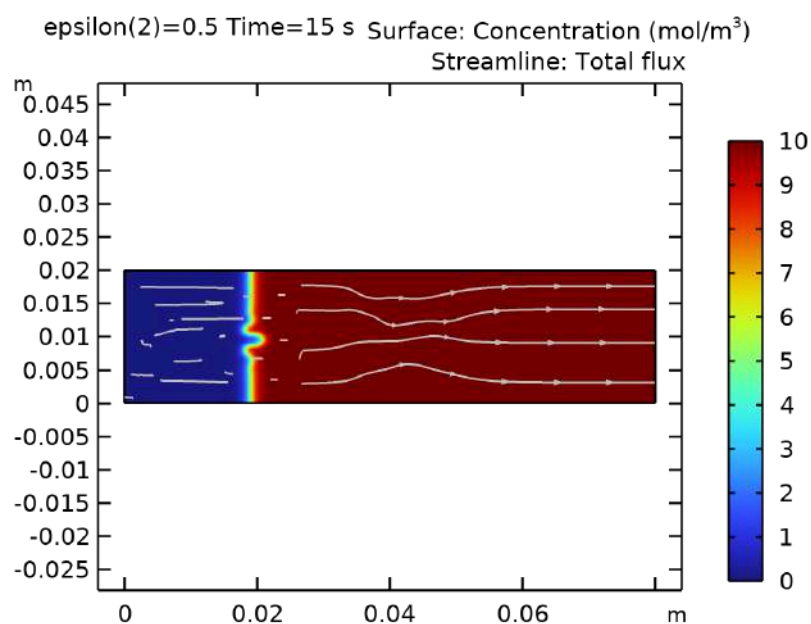
Surface: Concentration (mol/m³) Streamline: Total flux

4.2.3. Pressure (dl) 1



Surface: Pressure (Pa) Streamline: Darcy's velocity field

4.2.4. Concentration (tds) 1



Surface: Concentration (mol/m³) Streamline: Total flux